AndorSDK3 Camera Control Software

User Guide

By Techit Sitthiwong

Quantum Atom Optics Laboratory

July 2025

สารบัญ

สารบัญ 1

สารบัญภาพ 4

Andor Sona CMOS Information 5

การใช้ Python สำหรับควบคุมกล้อง Andor Sona sCMOS 6

การเชื่อมต่อให้ Python กับกล้อง SDK3 (Andor Sona sCMOS) 6

คำสั่ง Python ถ่ายภาพแบบ snap ด้วยกล้อง sCMOS 8

คำสั่ง Python ถ่ายภาพแบบ Live ด้วยกล้อง sCMOS 12

วิธีใช้งานซอฟท์แวร์ AndorSDK3 camera control สำหรับควบคุมกล้อง sCMOS 16

1. การเชื่อมต่อกับกล้อง sCMOS 18

3. การตั้งค่าพารามิเตอร์ต่างๆก่อนเริ่มการถ่ายภาพด้วยกล้อง sCMOS 21

4. การตั้งค่าเปิดโหมด FFT และ Filter 25

5. แถบ Toolbar สำหรับสั่งใช้กล้อง sCMOS ให้ถ่ายภาพ 25

7. ตัวอย่างการถ่ายภาพในโหมดต่างๆ 27

คำสั่ง Python ทั้งหมดของซอฟท์แวร์ AndorSDK3 Camera Control 30

Import library ทั้งหมดสำหรับซอฟท์แวร์ AndorSDK3 Camera Control 30

คำสั่ง Python กำหนดตัวแปรต่างๆแบบ global เพื่อรองรับพารามิเตอร์ในการควบคุมกล้อง sCMOS 30

ฟังก์ชันคำสั่ง Python สำหรับ import ไฟล์ DLL ที่จะเป็นของกล้อง sCMOS 31

ฟังก์ชันคำสั่ง Python สำหรับจัดเรียงตำแหน่งและแถวภายใน Right Panel 32

ฟังก์ชันคำสั่ง Python สำหรับเชื่อมต่อและตัดการเชื่อมต่อกับกล้อง sCMOS 32

ฟังก์ชันคำสั่ง Python สำหรับ เปิด/ปิด ระบบทำความเย็นของกล้อง sCMOS 33

ฟังก์ชันคำสั่ง Python สำหรับตั้งค่าอุณหภูมิและเปิดระบบทำความเย็นของกล้อง sCMOS 34

ฟังก์ชันคำสั่ง Python สำหรับแสดงอุณหภูมิของกล้อง sCMOS บน Status bar 35

ฟังก์ชันคำสั่ง Python สำหรับบันทึกการตั้งค่าและพารามิเตอร์ที่ใช้ถ่ายภาพด้วยกล้อง sCMOS 36

ฟังก์ชันคำสั่ง Python สำหรับถ่ายภาพ Background ด้วยกล้อง sCMOS 40

ฟังก์ชันคำสั่ง Python สำหรับอัพเดทสถานบน Status bar และ เปิด/ปิด Combo box ต่างๆ 42

ฟังก์ชันคำสั่ง Python สำหรับบันทึกการตั้งค่าการประมวลผล FFT ก่อนเริ่มการถ่ายภาพด้วยกล้องsCMOS 44

ฟังก์ชันคำสั่ง Python สำหรับถ่ายภาพ Snap (No Filters) ในโหมด Count ด้วยกล้อง sCMOS 45

ฟังก์ชันคำสั่ง Python สำหรับถ่ายภาพ Snap (No Filters) ในโหมด Bg Corrected ด้วยกล้อง sCMOS 47

ฟังก์ชันคำสั่ง Python สำหรับถ่ายภาพ Snap (Filters) ในโหมด Count ด้วยกล้อง sCMOS 49

ฟังก์ชันคำสั่ง Python สำหรับถ่ายภาพ Snap (Filters) ในโหมด Bg Corrected ด้วยกล้อง sCMOS 53

ฟังก์ชันคำสั่ง Python สำหรับถ่ายภาพ Accumulate Single Background (No Filter) ด้วยกล้อง sCMOS 57

ฟังก์ชันคำสั่ง Python สำหรับถ่ายภาพ Accumulate Single Background (Filters) ด้วยกล้อง sCMOS 60

ฟังก์ชันคำสั่ง Python สำหรับถ่ายภาพ Live (No Filters) ในโหมด Count ด้วยกล้อง sCMOS 65

ฟังก์ชันคำสั่ง Python สำหรับถ่ายภาพ Live (No Filters) ในโหมด Bg Corrected ด้วยกล้อง sCMOS 68

ฟังก์ชันคำสั่ง Python สำหรับถ่ายภาพ Live (Filters) ในโหมด Count ด้วยกล้อง sCMOS 70

ฟังก์ชันคำสั่ง Python สำหรับถ่ายภาพ Live (Filters) ในโหมด Bg Corrected ด้วยกล้อง sCMOS 74

ฟังก์ชันคำสั่ง Python สำหรับบันทึกภาพ Background ที่ถ่ายโดยกล้อง sCMOS 78

ฟังก์ชันคำสั่ง Python สำหรับบันทึกภาพ Image (+BG) ที่ถ่ายโดยกล้อง sCMOS 80

ฟังก์ชันคำสั่ง Python สำหรับบันทึกภาพ Image (-BG) ที่ถ่ายโดยกล้อง sCMOS 83

ฟังก์ชันคำสั่ง Python สำหรับบันทึกข้อมูลและภาพที่ถ่ายโดยกล้อง sCMOS หลังผ่านการประมวลผลด้วย FFT 86

คำสั่ง Python ทั้งหมดของ Main GUI ในซอฟท์แวร์ AndorSDK3 Camera Control 90

คำสั่ง Python ทั้งหมดของ Main GUI 90

อ้างอิง 97

สารบัญภาพ

รูป 1: หน้าต่างของซอฟท์แวร์ AndorSDK3 Camera Control 16

รูป 2: การเชื่อมต่อซอฟท์แวร์ AndorSDK3 Camera Control กับกล้อง sCMOS 18

รูป 3: วิธี import ไฟล์ dll เข้ามาในซอฟท์แวร์ 19

รูป 4: วิธีเชื่อมต่อกล้อง sCMOS 19

รูป 5: ปุ่ม Cooler ON/OFF 20

รูป 6: วิธีการตั้งค่าอุณหภูมิของกล้อง sCMOS 20

รูป 7: วิธีการแสดงสถานะและอุณหภูมิของกล้อง sCMOS 21

รูป 8: แผนก Camera Settings 21

รูป 9: การตั้งค่า Acquisition 22

รูป 10: การตั้งค่า Trigger Mode 23

รูป 11: การตั้งค่า Dynamic Range 24

รูป 12: การตั้งค่าพารามิเตอร์ต่างๆของกล้อง sCMOS 24

รูป 13: แผนก Image Processing 25

รูป 14: แผนก Toolbar ของซอฟท์แวร์ 25

รูป 15: แผนก Image Saving และ Image Saving with filter 26

รูป 16: ข้อมูลการทำงานของปุ่มใน Image Saving 26

รูป 17: แผนก Image Saving with filter 26

รูป 18: ตัวอย่างภาพ Background 27

รูป 19: ตัวอย่างภาพ Background ที่ accumulate 200 เฟรม 27

รูป 20: ตัวอย่างภาพ accumulate (Count) จำนวน 200 frames 28

รูป 21: (ก) ตัวอย่างภาพ Background และ (ข) accumulate single BG จำนวน 200 frames 28

รูป 22: ตัวอย่างภาพ snap แบบ FFT และใช้ Gaussian low pass filter 29

รูป 23: ตัวอย่างภาพ snap แบบ FFT และใช้ Gaussian high pass filter 29

Andor Sona CMOS Information

Sona quick start guide - USB3 and CXP

คู่มือแนะนำขั้นตอนการติดตั้งกล้อง Andor Sona อย่างครบถ้วน ตั้งแต่ข้อควรระวังเบื้องต้นก่อนใช้งาน วิธีการเปิดกล้องอย่างถูกต้อง การตรวจสอบการเชื่อมต่อระหว่างกล้องและคอมพิวเตอร์ ไปจนถึงการติดตั้งไดรเวอร์ของกล้องและการ์ด PCIe ทั้งในรูปแบบ USB 3.0 และ CoaXPress โดยเนื้อหาเหมาะสำหรับผู้ที่ต้องการติดตั้งกล้องและการ์ดเชื่อมต่ออย่างรวดเร็วและถูกต้องตามขั้นตอน

Sona and Marana-6 (Manual)

คู่มือสำหรับ Hardwareของกล้อง Andor Sona sCMOS ในคู่มือนี้จะบอกถึงรายละเอียดต่างๆ รวมไปถึงการทำงานของกล้อง sCMOS

Andor Sona sCMOS Driver Pack

Andor Driver Pack sCMOS ver. 3.15.30092.0 สำหรับกล้อง Andor Sona sCMOS

การใช้ Python สำหรับควบคุมกล้อง Andor Sona sCMOS

การเชื่อมต่อให้ Python กับกล้อง SDK3 (Andor Sona sCMOS)

Andor SDK 3 Software requirements

Library นี้ต้องการไฟล์ DLL ทุกไฟล์ไปวางไว้ใน folder “เดียวกันกับไฟล์ Python” โดย DLL ที่ซอฟต์แวร์ต้องการทั้งหมดมีดังนี้

- atcore.dll
- atblkbx.dll
- atcl_bitflow.dll
- atdevapogee.dll
- atdevregcam.dll
- atusb_libusb.dll
- atusb_libusb10.dll

ตัวอย่าง

atblkbx.dll	05/21/2025 3:02 PM	Application extens...	28 KB
atcl_bitflow.dll	05/21/2025 3:02 PM	Application extens...	89 KB
atcore.dll	05/21/2025 3:02 PM	Application extens...	419 KB
atdevapogee.dll	05/21/2025 3:02 PM	Application extens...	1,535 KB
atdevchamcam.dll	05/21/2025 3:02 PM	Application extens...	1,208 KB
atusb_libusb.dll	05/21/2025 3:02 PM	Application extens...	24 KB
atusb_libusb10.dll	05/21/2025 3:02 PM	Application extens...	85 KB
parameters.dat	05/21/2025 3:02 PM	DAT File	1 KB

ไฟล์ DLL ตัวอื่นนอกเหนือจากในภาพตัวอย่างสามารถหาได้จาก Andor Driver Pack 3 folder ซึ่งเป็นตำแหน่งที่ติดตั้ง driver

คำสั่ง Python ควบคุมการทำงานพื้นฐานของกล้อง sCMOS

สามารถดูคำสั่ง python ในการควบคุมกล้อง Andor Sona sCMOS (AndorSDK3Camera) ทั้งหมดได้ที่

1. คำสั่งในการควบคุมกล้องและคำอธิบายรายละเอียดการใช้งานต่างๆ >> pylablib.devices.Andor package — pylablib 1.4.1 documentation (คำสั่ง Python ทั้งหมดและคำสั่งของกล้อง SDK3 จะอยู่ในช่วงครึ่งหลังของเว็บไซต์ ในส่วนของครึ่งแรกจะเป็นคำสั่งสำหรับกล้อง SDK2)

2. ตัวอย่างการนำคำสั่งไปใช้, ข้อมูลเกี่ยวกับ SDK3 และคำสั่งต่างๆ >> pylablib Documentation (readthedocs.org)

ตัวอย่างการใช้ Python สั่งการควบคุมกล้อง Andor Sona sCMOS อ้างอิงจาก >> pylablib Documentation(readthedocs.org)

การเชื่อมต่อกล้อง Andor Sona sCMOS กับ Python

```
1. import pylablib as pll
2. pll.par["devices/dlls/andor_sdk3"] = "path/to/CMOS/dlls"
3. from pylablib.devices import Andor
4. cam = Andor.AndorSDK3Camera()
```

> บรรทัดที่ 1 และ 2 เป็นตัวอย่างการเชื่อมต่อกับตำแหน่งไฟล์ของ DLL ที่กล้องต้องการ

> บรรทัด 3 และ 4 เป็นการเชื่อมต่อกล้อง sCMOS กับ Python

ตัวอย่างการถ่ายภาพ

```
1. from pylablib import Andor
2. cam = AndorSDK3Camera() #connect to the camera
3. cam.set_exposure(20E-3) #set exposure time at 20 ms
4. cam.set_roi(0,512,0,512) #set ROI 512x512 px
5. image = cam.grab(10) #grab 10 frames
6. cam.close()
```

> บรรทัดที่ 3 เป็นการตั้งค่า exposure time ที่ 20 มิลลิวินาที

> บรรทัดที่ 4 เป็นการตั้งค่าขนาดของ ROI ที่ต้องการถ่ายภาพ

> บรรทัดที่ 5 คำสั่ง grab(n) เป็นคำสั่งถ่ายภาพโดยต้องระบุจำนวนของภาพที่จะถ่าย และบันทึกเป็นชุดข้อมูล

> บรรทัดที่ 6 เป็นคำสั่ง ปิดหรือตัดการเชื่อมต่อกับกล้อง sCMOS

คำสั่ง Python ถ่ายภาพแบบ snap ด้วยกล้อง sCMOS

Import library and Connect to the Andor SDK3 Camera

```
1. ## Create a parameter dictionary (e.g., for some processing script)
2. import pylablib as pll
3. pll.par["devices/dlls/andor_sdk3"] = "path/to/CMOS/dlls"
4.
5. import matplotlib.pyplot as plt
6. import matplotlib.colors
7. import numpy as np
8. import time
9. import cv2
10. import os
11.
12. # Import Andor devices module
13. from pylablib.devices import Andor
14. print("The number of cameras that can be controlled is: "
,Andor.get_cameras_number_SDK3())
15.
16. # Connect to Andor SDK3 camera
17. cam = Andor.AndorSDK3Camera()
```

> บรรทัดที่ 13 - 14 เป็นการบอกจำนวนกล้อง AndorSDK3 ที่เชื่อมต่อแล้ว

> บรรทัดที่ 17 ทำการเชื่อมต่อกล้อง หลังจากนั้นสามาถเรียกใช้ตัวแปร cam เพื่อใช้งานคำสั่งในการควบคุมกล้อง sCMOS

การตั้งค่าอุณหภูมิ

```
1. cam.set_temperature(-25, enable_cooler = True) # set temperature in celsius
2. cam.get_temperature()
3. cam.set_cooler(False) # Turn the cooler off (optional)
```

> บรรทัดที่ 1 เป็นการเซ็ตอุณหภูมิของกล้องที่ -25 องศาเซลเซียส ซึ่งกล้อง Andor Sona sCMOS สามารถตั้งค่าอุณหภูมิได้ถึง -25 องศาเซลเซียส โดยที่ไม่ได้ใช้ระบบทำความเย็นเพิ่มเติม อ้างอิงจาก [Sona_Quick_Start_Guide.pdf (oxinst.com)]

> บรรทัดที่ 2 เช็คอุณหภูมิของกล้อง sCMOS

> บรรทัดที่ 3 คำสั่งปิดระบบทำความเย็นของกล้อง sCMOS

การตั้งค่า parameter ต่างๆของกล้อง sCMOS

```
 1. # Cam setting
 2. cam.set_trigger_mode("int") # set trigger mode
 3. cam.set_exposure(20E-3) # set exposure
 4. # Set Binning paremeter
 5. hbin , vbin = 1 , 1 #set binning 1x1
 6.
 7. # Set ROI  paremeter (2048x2048)
 8. hstart , hend = 0 , 2048
 9. vstart , vend = 0 , 2048
10. # Set ROI
11. cam.set_roi(hstart = hstart, hend = hend, vstart = vstart, vend = vend, hbin = hbin,
vbin = vbin)
12.
13. # Display size
14. display_size = (512,512) #(width, height)
```

> บรรทัดที่ 2 เป็นการเซ็ต trigger ซึ่งประเภทของ trigger มี “int”, “ext”, “ext_exp”

> บรรทัดที่ 3 เป็นการตั้งค่า exposure time ที่ 20 มิลลิวินาที

> บรรทัดที่ 4 การตั้งค่าขนาด Binning

> บรรทัดที่ 8 - 9 กำหนดขนาดของ ROI เพื่อกำหนดขอบเขตของการรับข้อมูลโดย hstart, vstart คือจุดเริ่มต้นของพิกเซลและ hend, vend จุดสุดท้ายของพิกเซล จากนั้นพารามิเตอร์ที่เซ็ตไว้จะรวมอยู่ในคำสั่ง set_roi

> บรรทัดที่ 14 เป็นการตั้งค่าขนาดของหน้าต่างที่แสดงผลภาพถ่าย

คำสั่ง python ถ่ายภาพ Background สำหรับการถ่ายภาพในโหมด Bg corrected

```
 1. #Start Take Background Acquisition
 2. cam.setup_acquisition(mode = 'sequence', nframes = 100)
 3. cam.start_acquisition()
 4. print('Take background acquisition started')
 5.
 6. # Take initial background
 7. cam.wait_for_frame(since = 'lastread', nframes = 1 , timeout = 20 , error_on_stopped =
False)
 8. background = np.float32(cam.read_newest_image())
 9. print('Take initial background')
10.
11. # Normalize for display
12. bg_vis = background.copy()
13. bg_vis = cv2.normalize(bg_vis, None, 0, 65535, cv2.NORM_MINMAX).astype(np.uint16)
14.
15.
16. # Resize for better viewing
17. bg_vis_resized = cv2.resize(bg_vis, display_size , interpolation = cv2.INTER_NEAREST)
18.
19. #Stop acquisition
20. cam.stop_acquisition()
21. print('Take backgound acquisition stopped')
22.
23. # Show using OpenCV
24. cv2.imshow("Background Captured", bg_vis_resized)
25. cv2.waitKey(0)
26. cv2.destroyAllWindows()
```

> บรรทัดที่ 2 ตั้งค่า acquisition โหมดสามารถเป็นได้ทั้ง snap หรือ sequence (การเก็บข้อมูลอย่างต่อเนื่อง) nframes ใช้กำหนดจำนวนเฟรมที่จะเก็บใน single mode หรือขนาดของ buffer ในโหมด sequence (default คือ 100)

> บรรทัดที่ 3 - 4 คำสั่งให้เริ่ม acquisition โดยกล้องจะเก็บข้อมูลภาพตาม setup_acquisition ที่ตั้งค่าไว้

> บรรทัดที่ 7 – 9 เป็นคำสั่งให้กล้อง คงสถานะการเก็บภาพ wait_for_frame โดยตั้งค่าไว้ที่ 20 วินาทีเพื่อไม่ให้เกิด Time out ในระหว่างการถ่ายภาพ และในบรรทัดที่ 8 เป็นคำสั่งที่กล้องจะอ่านภาพล่าสุดจาก buffer

> บรรทัดที่ 12 - 13 เป็นคำสั่ง Python การทำ normalize ให้ค่าข้อมูลของพิกเซลต่ำสุดเป็น 0 และค่าสูงสุดเป็น 1 เพื่อที่จะนำไปแปลงเป็นข้อมูล Gray scale ชนิด 16-bit ในบรรทัดที่ 13 สำหรับแสดงผล

> บรรทัดที่ 17 เป็นคำสั่งปรับขนาดของน้ำต่างแสดงผลภาพ โดยสามารถปรับขนาดได้จากตัวแปร display_size

> บรรทัดที่ 20 เป็นคำสั่งหยุดการทำงานของ acquisition

> บรรทัดที่ 24 – 26 คำสั่งสำหรับการแสดงผลภาพ background ด้วย open cv

คำสั่ง python สร้าง folder สำหรับบันทึกภาพที่ถ่ายด้วยกล้อง sCMOS

```
1. #Create folder for image saving (optional)
2. os.makedirs("Data/Acquisition", exist_ok = True)
```

> บรรทัดที่ 2 คำสั่งสำหรับสร้าง folder ที่จะบันทึกภาพที่ถ่ายได้จากกล้อง sCMOS โดยสามารถเปลี่ยนชื่อของ folder ได้ในพารามิเตอร์ “Data /Acquisition”

คำสั่ง python การถ่ายภาพแบบ single frame ด้วยกล้อง sCMOS

```
 1. #Start acquisition
 2. cam.setup_acquisition(mode = 'sequence', nframes = 100) #Acquisition setup
 3. cam.start_acquisition()
 4. print('Live acquisition started')
 5.
 6. cam.wait_for_frame(since = 'lastread', nframes = 1, timeout = 20, error_on_stopped =
False)
 7. frame = np.float32(cam.read_newest_image())
 8.
 9. # Background corrected
10. signal = frame - background
11.
12. #Normalize for output
13. vis2 = signal.copy()
14. vis2 = cv2.normalize(vis2, None, 0, 65535, cv2.NORM_MINMAX).astype(np.uint16)
15.
16. #Re-size image
17. resized_img = cv2.resize(vis2, display_size , interpolation = cv2.INTER_NEAREST)
18.
19. #Save image (optional)
20. cv2.imwrite("Data/Acquisition/Snap_Data.tiff", resized_img)  #Save image as .tff
```

```
21. #np.savetxt("Data/Acquisition/Snap_Data.txt", resized_img) #Save image as  .txt
22.
23. #Stop acquisition
24. cam.stop_acquisition()
25. print('Acquisition stopped')
26.
27. #Display
28. cv2.imshow("Acquisition(Bg corrected)",resized_img)
29. cv2.waitKey(0)
30. cv2.destroyAllWindows()
```

> บรรทัดที่ 2 ตั้งค่า acquisition โหมดสามารถเป็นได้ทั้ง snap หรือ sequence (การเก็บข้อมูลอย่างต่อเนื่อง) nframes ใช้กำหนดจำนวนเฟรมที่จะเก็บใน single mode หรือขนาดของ buffer ในโหมด sequence (default คือ 100)

> บรรทัดที่ 3 - 4 คำสั่งให้เริ่ม acquisition โดยกล้องจะเก็บข้อมูลภาพตาม set_up acquisition ที่ตั้งค่าไว้

> บรรทัดที่ 6 เป็นคำสั่งให้กล้อง คงสถานะการเก็บภาพ wait_for_frame โดยตั้งค่าไว้ที่ 20 วินาทีเพื่อไม่ให้เกิด Time out ในระหว่างการถ่ายภาพ และในบรรทัดที่ 7 เป็นคำสั่งที่กล้องจะอ่านภาพล่าสุดจาก buffer

>บรรทัดที่ 10 เป็นคำสั่งการทำ Background corrected โดยจะนำข้อมูล array ของภาพที่ที่ถ่ายได้มาลบกับข้อมูล array ของภาพ background ที่ถ่ายไว้ในเซลล์ก่อนหน้า

> บรรทัดที่ 13 - 15 เป็นคำสั่ง Python การทำ normalize ให้ค่าข้อมูลของพิกเซลต่ำสุดเป็น 0 และค่าสูงสุดเป็น 1 เพื่อที่จะนำไปแปลงเป็นข้อมูล Gray scale ชนิด 16-bit ในบรรทัดที่ 15

> บรรทัดที่ 17 เป็นคำสั่งปรับขนาดของภาพ โดยสามารถปรับขนาดได้จากตัวแปร display_size

> บรรทัดที่ 20 เป็นคำสั่งสำหรับบันทึกภาพ .tiff ไว้ใน folder ที่สร้างไว้

> บรรทัดที่ 23 เป็นต้นไป คำสั่งสำหรับการแสดงผลภาพด้วย open cv และ stop_acquisition

คำสั่ง python ตัดการเชื่อมต่อกล้อง sCMOS

```
1. cam.close()
2. print('Disconnected Camera')
```

> บรรทัดที่ 1 เป็นคำสั่ง ปิดหรือตัดการเชื่อมต่อกับกล้อง sCMOS

คำสั่ง Python ถ่ายภาพแบบ Live ด้วยกล้อง sCMOS

Import library and Connect to the Andor SDK3 Camera

```
 1. import pylablib as pll
 2. pll.par["devices/dlls/andor_sdk3"] = "path/to/CMOS/dlls"
 3.
 4. import matplotlib.pyplot as plt
 5. import numpy as np
 6. import time
 7. import cv2
 8. import os
 9.
10. #Create folder for image saving
11. os.makedirs("output_Take_image_(Live)/Live Acquisition", exist_ok = True)
12.
13. # Import Andor devices module
14. from pylablib.devices import Andor
15. print("The number of cameras that can be controlled is :
",Andor.get_cameras_number_SDK3())
16.
17. # Connect to Andor SDK3 camera
18. cam = Andor.AndorSDK3Camera()
```

> บรรทัดที่ 14, 15 เป็นการบอกจำนวนกล้อง Andor SDK3 ที่เชื่อมต่อแล้ว

> บรรทัดที่ 18 ทำการเชื่อมต่อกับกล้อง sCMOS

การตั้งค่าอุณหภูมิ

```
1. cam.set_temperature(-25, enable_cooler = True) # set temperature in celsius
2. cam.get_temperature()
```

> บรรทัดที่ 1 เป็นการเซ็ตอุณหภูมิของกล้องที่ -25 องศาเซลเซียส ซึ่งกล้อง Andor Sona sCMOS สามารถตั้งค่าอุณหภูมิได้ถึง -25 องศาเซลเซียส โดยที่ไม่ได้ใช้ระบบทำความเย็นเพิ่มเติม อ้างอิงจาก [Sona_Quick_Start_Guide.pdf (oxinst.com)]

> บรรทัดที่ 2 เช็คอุณหภูมิของกล้อง sCMOS

การตั้งค่า Dynamic range ของกล้อง sCMOS

```
1. #Set attribute values(12-bit low noise, etc.)
2. cam.set_attribute_value('SimplePreAmpGainControl', '12-bit (low noise)')  #Set Dynamic
raage 16-bit (high dynamic range), 12-bit (low noise)
3.
4. #Show setting
5. print("Dynamic range :",cam.get_attribute_value('SimplePreAmpGainControl'))
```

> บรรทัดที่ 2 เป็นการตั้งค่า attribute ของกล้องโดยสำหรับเลือก dynamic range

การตั้งค่า parameter ต่างๆของกล้อง sCMOS

```
 1. #cam setting
 2. cam.set_trigger_mode("ext") # set trigger mode (ext, int, ext_exp)
 3. cam.set_exposure(20E-3) # set exposure
 4.
 5. #Set Binning  paremeter
 6. hbin , vbin = 8 , 8
 7.
 8. #Set ROI  paremeter
 9. hstart , hend = 256 , 1792
10. vstart , vend = 256 , 1792
11.
12. #set ROI
13. cam.set_roi(hstart = hstart, hend = hend, vstart = vstart, vend = vend, hbin = hbin,
vbin = vbin)
14.
15. #Display size
16. display_size = (512,512) #(width, height)
```

> บรรทัดที่ 2 เป็นการเซ็ต trigger ซึ่งประเภทของ trigger มี “int”, “ext”, “ext_exp”

> บรรทัดที่ 3 เป็นการตั้งค่า exposure time ที่ 20 มิลลิวินาที

> บรรทัดที่ 6 การตั้งค่าขนาด Binning

> บรรทัดที่ 9 - 10 กำหนดขนาดของ ROI เพื่อกำหนดขอบเขตของการรับข้อมูลโดย hstart, vstart

คือ จุดเริ่มต้นของพิกเซลและ hend, vend จุดสุดท้ายของพิกเซล จากนั้นพารามิเตอร์ที่เซ็ตไว้จะรวมอยู่ในคำสั่ง set_roi

> บรรทัดที่ 16 เป็นการตั้งค่าขนาดของหน้าต่างที่แสดงผลภาพถ่าย

คำสั่ง python เก็บภาพ Background สำหรับการถ่ายภาพในโหมด Bg corrected

```
 1. #Start Take Background Acquisition
 2. cam.setup_acquisition(mode = 'sequence', nframes = 100)
 3. cam.start_acquisition()
 4. print('Take background acquisition started')
 5.
 6. # Take initial background
 7. cam.wait_for_frame(since = 'lastread', nframes = 1 , timeout = 20 , error_on_stopped =
False)
 8. background = np.float32(cam.read_newest_image())
 9. print('Take initial background')
10. print(background)
11.
```

```
12. # Normalize for display
13. bg_vis = background.copy()
14. bg_vis = cv2.normalize(background, None, 0, 65535, cv2.NORM_MINMAX).astype(np.uint16)
15.
16. # Resize for better viewing
17. bg_vis_resized = cv2.resize(bg_vis, display_size , interpolation = cv2.INTER_NEAREST)
18. cam.stop_acquisition()
19. print('Take backgound acquisition stopped')
20.
21. # Show using OpenCV
22. cv2.imshow("Background Captured", bg_vis_resized)
23. cv2.waitKey(0)
24. cv2.destroyAllWindows()
```

> บรรทัดที่ 2 ตั้งค่า acquisition โหมดสามารถเป็นได้ทั้ง snap หรือ sequence (การเก็บข้อมูลอย่างต่อเนื่อง) nframes ใช้กำหนดจำนวนเฟรมที่จะเก็บใน single mode หรือขนาดของ buffer ในโหมด sequence (default คือ 100)

> บรรทัดที่ 3 - 4 คำสั่งให้เริ่ม acquisition โดยกล้องจะเก็บข้อมูลภาพตาม setup_acquisition ที่ตั้งค่าไว้

> บรรทัดที่ 7 – 8 เป็นคำสั่งให้กล้อง คงสถานะการเก็บภาพ wait_for_frame โดยตั้งค่าไว้ที่ 20 วินาทีเพื่อไม่ให้เกิด Time out ในระหว่างการถ่ายภาพ และในบรรทัดที่ 8 เป็นคำสั่งที่กล้องจะอ่านภาพล่าสุดจาก buffer

> บรรทัดที่ 12 - 14 เป็นคำสั่ง Python การทำ normalize ให้ค่าข้อมูลของพิกเซลต่ำสุดเป็น 0 และค่าสูงสุดเป็น 1 เพื่อที่จะนำไปแปลงเป็นข้อมูล Gray scale ชนิด 16-bit ในบรรทัดที่ 13 สำหรับแสดงผล

> บรรทัดที่ 17 เป็นคำสั่งปรับขนาดของน้ำต่างแสดงผลภาพ โดยสามารถปรับขนาดได้จากตัวแปร display_size

> บรรทัดที่ 18 เป็นคำสั่งหยุดการทำงานของ acquisition

> บรรทัดที่ 22 – 24 คำสั่งสำหรับการแสดงผลภาพ background ด้วย open cv

คำสั่ง python เก็บภาพสำหรับการถ่ายภาพแบบ Live ในโหมด Bg corrected

```
 1. #Start acquisition
 2. cam.setup_acquisition(mode = 'sequence', nframes = 100)
 3. cam.start_acquisition()
 4. print('Live acquisition started')
 5.
 6. frame_count = 0
 7.
 8. while True:
 9.     cam.wait_for_frame(since = 'lastread', nframes =1 , timeout = 20 , error_on_stopped
= False)
10.     frame = np.float32(cam.read_newest_image())
11.
12.     #Background corrected
13.     signal = frame - background
14.
15.     #Normalize for output
16.     image = signal.copy()
17.     vis2 = signal.copy()
18.     vis2 = cv2.normalize(vis2, None, 0, 65535, cv2.NORM_MINMAX).astype(np.uint16)
19.
20.     #Re-size image
21.     resized_img = cv2.resize(vis2, display_size , interpolation = cv2.INTER_NEAREST)
22.
23.     #Display
24.     frame_count += 1
25.     cv2.imshow("Live Acquisition(Bg corrected)",resized_img )
26.     if cv2.waitKey(1) & 0xFF == ord("w"):
27.         break
28.
29. cam.stop_acquisition()
30. print('Live acquisition stopped')
31. cv2.destroyAllWindows()
```

> บรรทัดที่ 2 ตั้งค่า acquisition โหมดสามารถเป็นได้ทั้ง snap หรือ sequence (การเก็บข้อมูลอย่างต่อเนื่อง) nframes ใช้กำหนดจำนวนเฟรมที่จะเก็บใน single mode หรือขนาดของ buffer ในโหมด sequence (default คือ 100)

> บรรทัดที่ 8 เป็นต้นไปจะเป็นคำสั่งถ่ายภาพแบบ Live คำสั่งให้เริ่ม acquisiton โดยกล้องจะเก็บข้อมูลภาพตาม setup_acquisition ที่ตั้งค่าไว้ จากนั้นเป็นคำสั่งให้กล้อง คงสถานะการเก็บภาพ wait_for_frame โดยตั้งค่าไว้ที่ 20 วินาทีเพื่อไม่ให้เกิด Time out ในระหว่างการถ่ายภาพ และในบรรทัดที่ 9-10 เป็นคำสั่งที่กล้องจะอ่านภาพล่าสุดจาก buffer และนำ array ข้อมูลของ background มาลบกับข้อมูล array ของภาพปัจจุบันที่ถ่าย จากนั้นจะเป็นการ Normalize และแสดงผลภาพแบบ Live เมื่อกด w จะเป็นการออกจากลูปการทำงาน

>บรรทัดที่ 29 เป็นคำสั่งหยุดการทำงานของ acquisition

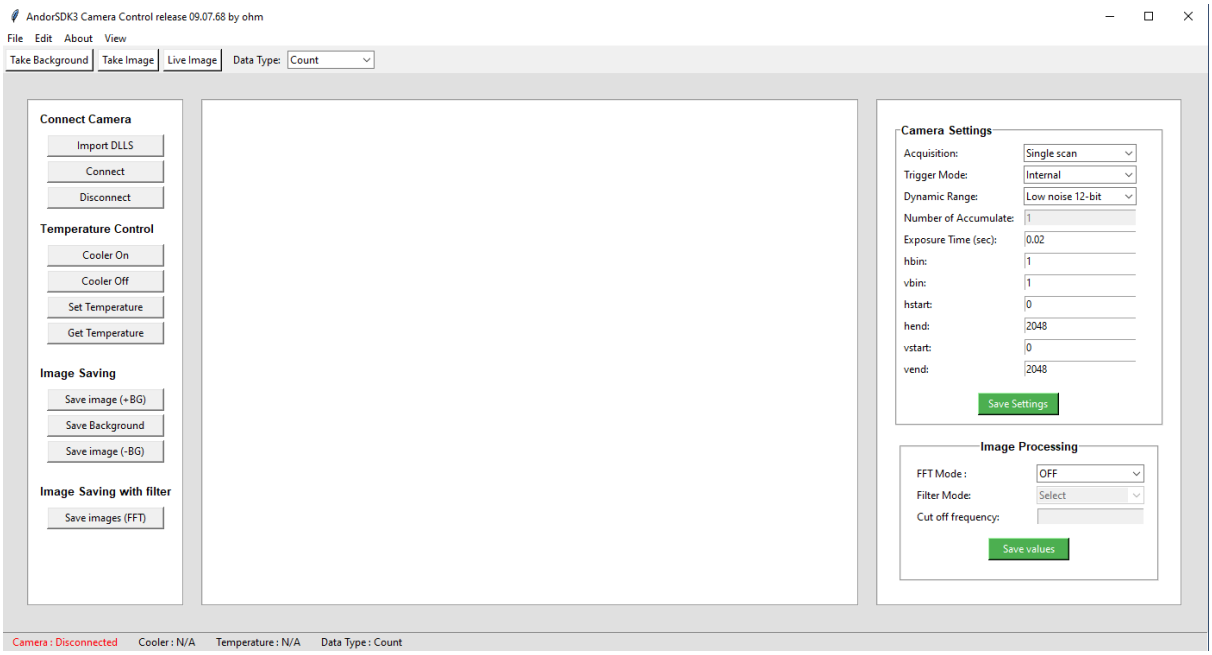
คำสั่ง Python ตัดการเชื่อมต่อกับกล้อง sCMOS

```
1. cam.close()
2. print('Disconnected Camera')
```

> บรรทัดที่ 1 เป็นคำสั่ง ปิดหรือตัดการเชื่อมต่อกับกล้อง sCMOS

วิธีใช้งานซอฟท์แวร์ AndorSDK3 camera control สำหรับควบคุมกล้อง sCMOS

หน้าต่างซอฟท์แวร์ AndorSDK3 Camera Control สำหรับควบคุมกล้อง Andor Sona sCMOS



รูป 1: หน้าต่างของซอฟท์แวร์ AndorSDK3 Camera Control

ก่อนใช้งานซอฟท์แวร์คอมพิวเตอร์ที่ใช้งานซอฟท์แวร์นี้ ต้องมี library เหล่านี้ติดตั้งอยู่ หากยังไม่มีให้ติดตั้งก่อนเพื่อป้องการปัญหาจากการใช้งานซอฟท์แวร์*

Library ที่จำเป็นต่อซอฟท์แวร์ AndorSDK3 Camera Control
tkinter – ไลบรารี GUI พื้นฐานของ Python สำหรับสร้างหน้าต่างและองค์ประกอบอินเทอร์เฟซ (widgets)
tkinter.ttk – โมดูลย่อยของ tkinter ที่มี widgets แบบปรับปรุง (themed widgets)
tkinter.messagebox และ tkinter.filedialog – โมดูลสำหรับแสดงกล่องข้อความ (เช่น เตือน, แจ้งเตือน) และหน้าต่างเลือกไฟล์
PIL.Image, PIL.ImageTk – ไลบรารีสำหรับจัดการและแสดงผลรูปภาพ (Pillow คือเวอร์ชันอัปเดตของ PIL)
threading – ไลบรารีสำหรับรันโค้ดหลายเธรดพร้อมกัน (multi-threading)
pylablib.devices.Andor – โมดูลสำหรับควบคุมกล้อง Andor ผ่านไลบรารี pylablib

pylablib – ไลบรารีสำหรับควบคุมอุปกรณ์ทดลอง เช่น กล้อง sCMOS, motion controllers ฯลฯ

cv2 – ไลบรารี OpenCV สำหรับประมวลผลภาพและวิดีโอ

numpy – ไลบรารีสำหรับการคำนวณเชิงตัวเลขและเมทริกซ์

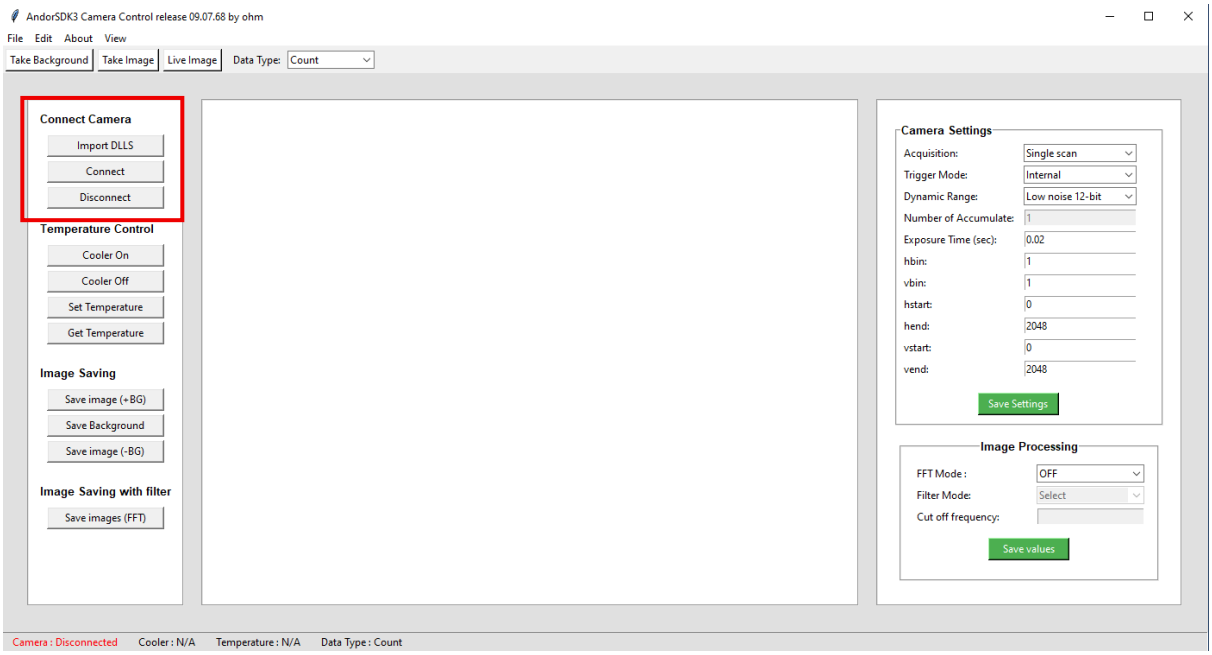
os – ไลบรารีสำหรับจัดการระบบไฟล์, path, และฟังก์ชันของระบบปฏิบัติการ

datetime – ไลบรารีสำหรับจัดการวัน เวลา และ timestamp

1. การเชื่อมต่อกับกล้อง sCMOS

การเชื่อมต่อซอฟท์แวร์ AndorSDK3 Camera Control กับกล้อง sCMOS มีขั้นตอนดังนี้

1.1 หลังจากเปิดโปรแกรม AndorSDK3 Camera Control จะพบกับหน้าต่างของ ซอฟท์แวร์สำหรับการเชื่อมต่อกับกล้อง sCMOS จะต้องไปที่แผนก Connect Camera ตามในกรอบสีแดงในภาพ

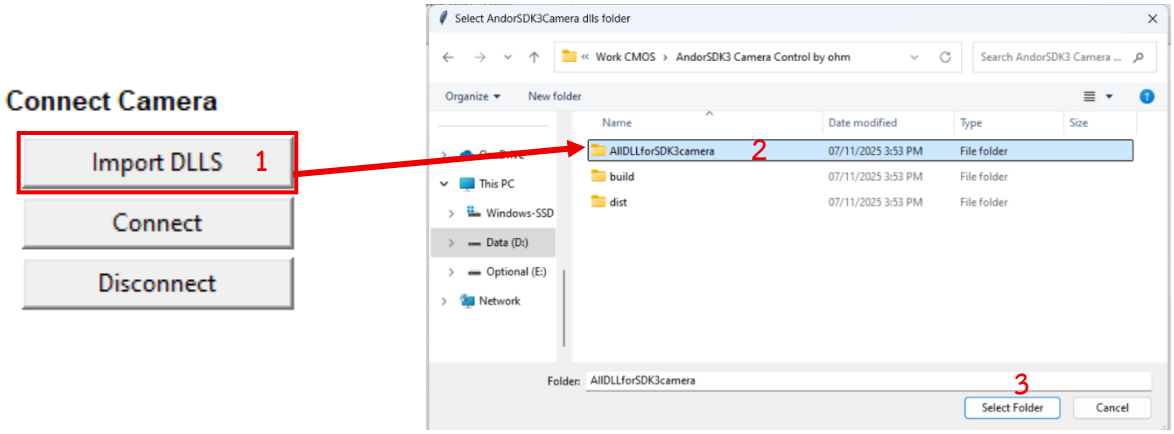


รูป 2: การเชื่อมต่อซอฟท์แวร์ AndorSDK3 Camera Control กับกล้อง sCMOS

1.1 เลือกปุ่ม “Import DLLS” จะเป็นการเปิด file dialog เพื่อหาตำแหน่งของ folder ที่มีไฟล์ .dll อยู่ข้างใน และ import โฟลเดอร์นั้นเข้ามา โดยไฟล์ dll ที่จำเป็นสำหรับกล้อง sCMOS สามารถดูได้ที่หัวข้อ

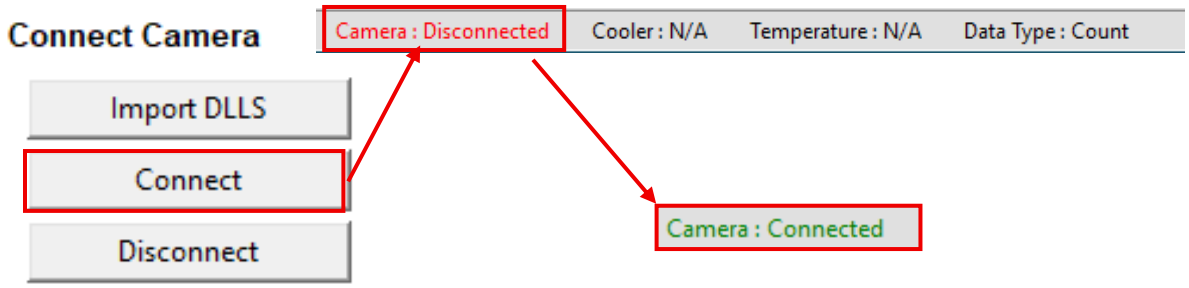
[การใช้ Python สำหรับควบคุมกล้อง Andor sona sCMOS](#) เมื่อเลือก Folder ที่มี dll อยู่แล้วจะแจ้งเตือนชื่อไฟล์ dll ทั้งหมดที่ทำการ import เข้ามา

Import DLLS > dll folder > Select Folder



รูป 3: วิธี import ไฟล์ dll เข้ามาในซอฟท์แวร์

1.2 เชื่อมซอฟท์แวร์กับกล้องเลือกปุ่ม “Connect” และรอสถานะการเชื่อมกับกล้องบริเวณ status bar (บริเวณแถบด้านล่างของซอฟท์แวร์) เปลี่ยนจาก “Camera : Disconnected” เป็น “Camera : Connected” ถือว่าเชื่อมต่อเสร็จสิ้น ดังภาพที่แสดงด้านล่าง

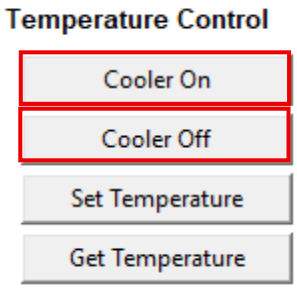


รูป 4: วิธีเชื่อมต่อกล้อง sCMOS

2. การตั้งค่าอุณหภูมิของกล้อง sCMOS

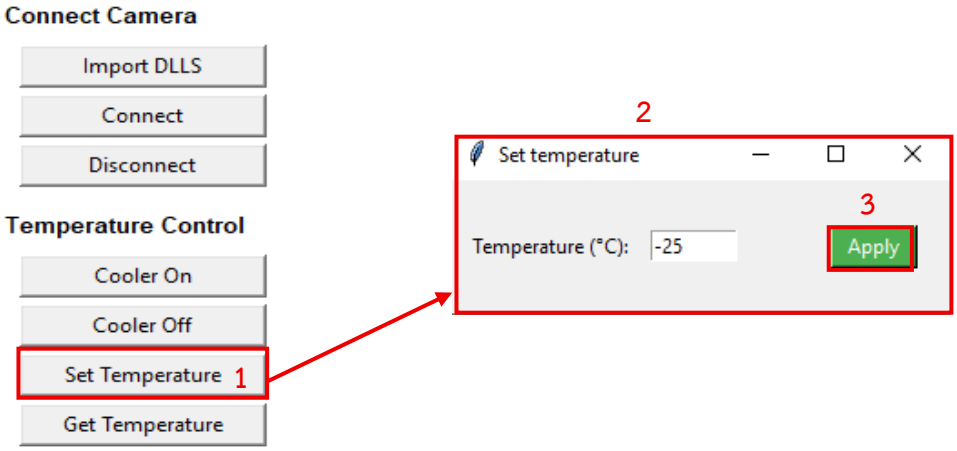
การตั้งค่าอุณหภูมิของกล้อง sCMOS ในซอฟท์แวร์ AndorSDK3 Camera Control มีขั้นตอนดังนี้

2.1 เมื่อเชื่อมซอฟท์แวร์กับกล้องsCMOS แล้ว ปุ่ม “Cooler On” เป็นปุ่มสำหรับเปิด cooler และปุ่ม “Cooler off ” เป็นปุ่มสำหรับปิด cooler โดยสถานะของการเปิด/ปิด cooler จะแจ้งเตือนบน status bar



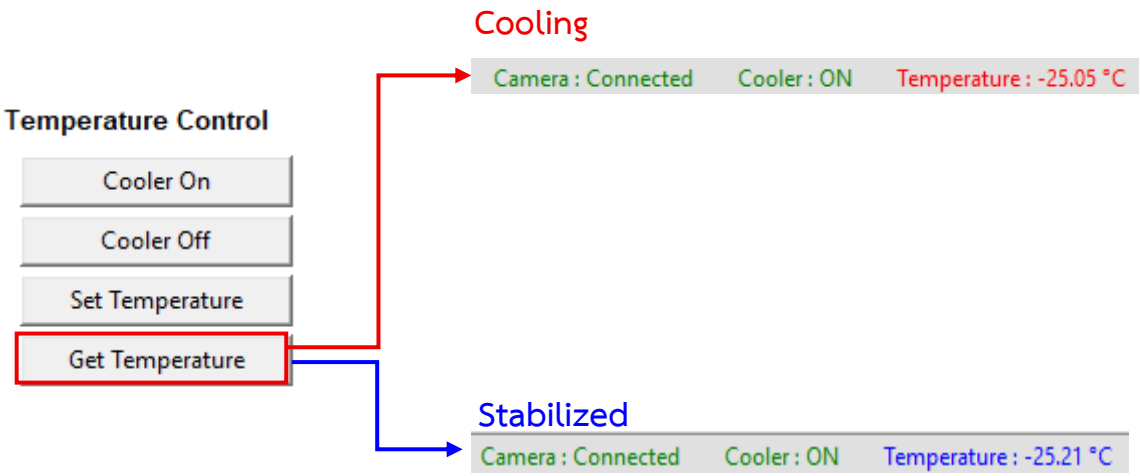
รูป 5: ปุ่ม Cooler ON/OFF

2.2 เมื่อเชื่อมซอฟท์แวร์กับกล้องsCMOS แล้วสิ่งที่ต้องทำเป็นลำดับถัดไปคือการตั้งค่าอุณหภูมิของกล้อง เมื่อกล้องอยู่ในสถานะ“Camera : Connected” ให้ไปที่แผนกของ Temperature Control และเลือกปุ่ม “Set Temperature” แสดง GUI ย่อยขึ้นมาให้กรอกค่าอุณหภูมิ เมื่อกรอกค่าอุณหภูมิเสร็จให้กดปุ่ม “Apply” เพื่อบันทึกค่าอุณหภูมิและเปิด cooler <สามารถใช้คีย์ลัด โดยกดปุ่ม “Enter” บนคีย์บอร์ดแทนได้>



รูป 6: วิธีการตั้งค่าอุณหภูมิของกล้อง sCMOS

2.3 เมื่อตั้งค่าอุณหูมิแล้ว การเลือกปุ่ม “Get Temperature” จะเป็นการบอกค่าอุณหภูมิของกล้องและแสดงผลบริเวณ status bar ถ้าอุณหภูมิของกล้องยังไม่เสถียรจะแสดงค่าอุณหภูมิเป็นสีแดงจนกว่าอุณหภูมิจะเสถียร หากอุณหภูมิของกล้องนั้นเสถียรแล้วจะแสดงเป็นสีน้ำเงิน

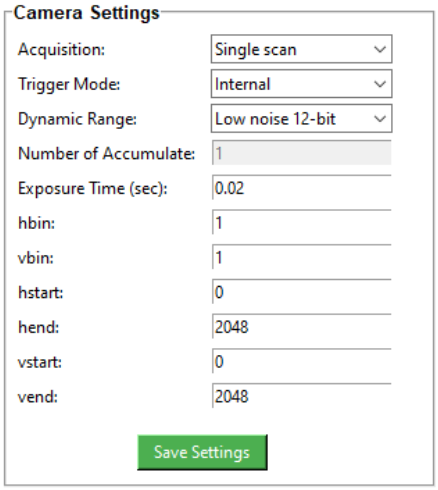


รูป 7: วิธีการแสดงสถานะและอุณหภูมิของกล้อง sCMOS

3. การตั้งค่าพารามิเตอร์ต่างๆก่อนเริ่มการถ่ายภาพด้วยกล้อง sCMOS

แผนก Camera Settings ของกล้อง sCMOS ในซอฟท์แวร์ AndorSDK3 Camera Control

(เมื่อตั้งค่าเสร็จต้องกดปุ่ม “Save Settings” ทุกครั้ง)



รูป 8: แผนก Camera Settings

3.1 Camera Settings ในส่วนที่เป็น Combo Box จะประกอบไปด้วย Acquisition, Trigger Mode, Dynamic Range

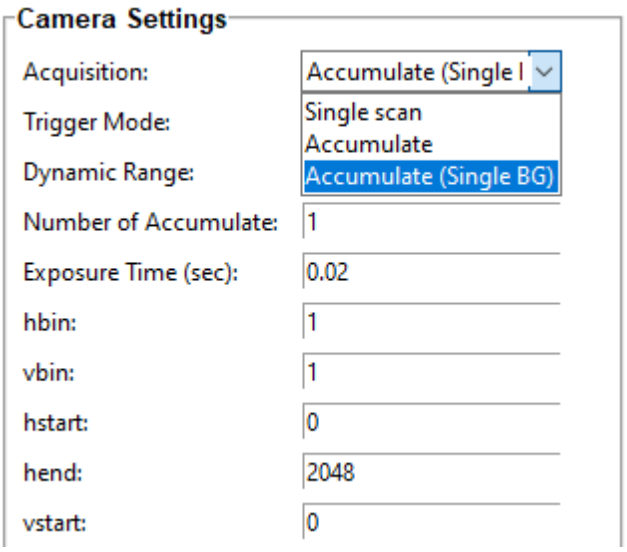
3.1.1 Acquisition สามารถเลือกได้ 3 โหมดได้แก่

Single Scan คือการถ่ายภาพถ่ายภาพแบบ 1 frame

Accumulate คือการถ่ายภาพแบบ stack หลายภาพไว้ใน 1 ภาพเมื่อเลือกโหมดนี้

ช่อง Number of Accumulate จะเปิดให้กรอกจำนวนครั้งของการ accumulate

Accumulate (Single BG) คือการถ่ายภาพแบบ stack หลายภาพไว้ใน 1 ภาพเมื่อเลือกโหมดนี้ช่อง Number of Accumulate จะเปิดให้กรอกจำนวนครั้งของการ accumulate แต่ใน mode นี้จะเป็นการถ่ายภาพ background เพียวครั้งเดียวและนำข้อมูลภาพมาลบกับข้อมูลภาพปัจจุบันที่ถ่าย



รูป 9: การตั้งค่า Acquisition

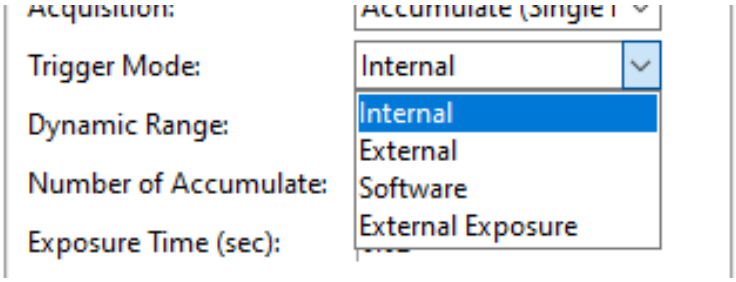
3.1.2 Trigger Mode สามารถเลือกได้ 4 โหมดได้แก่

Internal: กำหนดสัญญาณการเริ่มถ่ายภาพจะถูกส่งมาจากภายในตัวกล้อง

External: กำหนดสัญญาณการเริ่มถ่ายภาพจะถูกส่งมาจากภายนอกตัวกล้อง

Software: กำหนดสัญญาณทริกเกอร์จากซอฟท์แวร์ที่ เป็นโหมดที่สั่งให้กล้องถ่ายภาพหรือเริ่มการรับภาพผ่านคำสั่งจากซอฟท์แวร์

External Exposure: สัญญาณการเริ่มถ่ายภาพและช่วงเวลาการเก็บภาพ (exposure) จะถูกส่งมาจากภายนอกตัวกล้อง

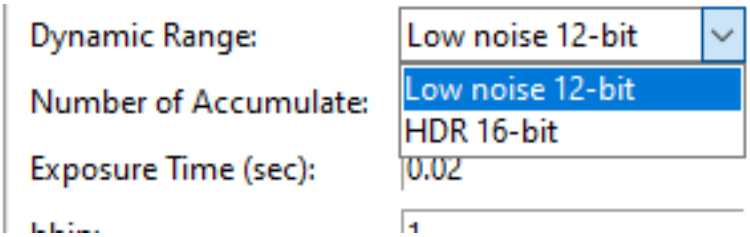


รูป 10: การตั้งค่า Trigger Mode

3.1.2 Dynamic Range สามารถเลือกได้ 2 โหมดได้แก่

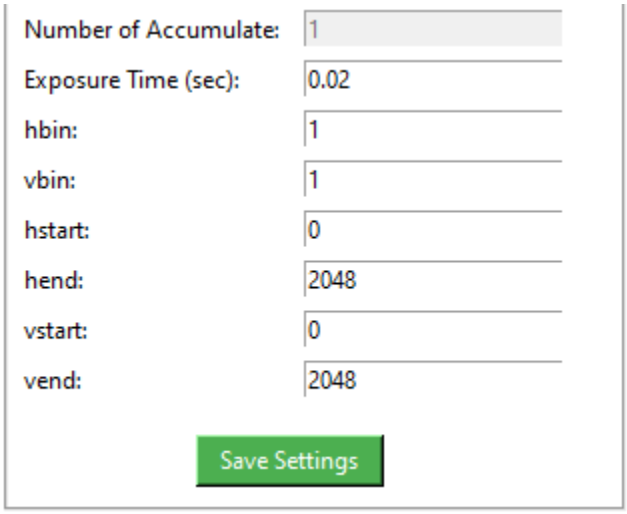
Low noise 12-bit: เป็นโหมดการทำงานของกล้อง sCMOS ที่ 1 พิกเซลสามารถเก็บค่าได้ตั้งแต่ 0 ถึง 4095

HDR 16-bit: เป็นโหมดการทำงานของกล้อง sCMOS ที่ 1 พิกเซลสามารถเก็บค่าได้ตั้งแต่ 0 ถึง 65,535



รูป 11: การตั้งค่า Dynamic Range

3.2 Camera Settings ในส่วนที่เป็น Entry box สำหรับกรอกค่าพารามิเตอร์ต่างๆของกล้อง sCMOS



รูป 12: การตั้งค่าพารามิเตอร์ต่างๆของกล้อง sCMOS

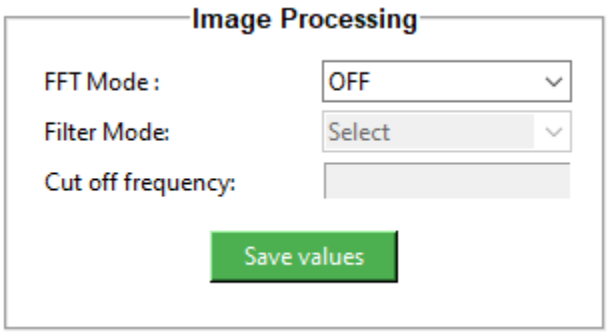
4. การตั้งค่าเปิดโหมด FFT และ Filter

แผนก Image Processing ในซอฟท์แวร์ AndorSDK3 Camera Control ในแผนกนี้ประกอบไปด้วยการเปิดใช้งาน FFT Mode และเลือก Filter โดยสามารถเลือกได้ 2 โหมดคือ “Low pass (GLPF)” และ “High pass (GHPF)” และช่องสำหรับกรอกค่า Cut off frequency โดยค่า cut off frequency ที่กรอกต้องเป็น float

Cut-off frequency น้อย (เช่น 0.1) ⟶ กรองมาก (GLPF), จะพบ Noise หรือขอบมาก (GHPF)

Cut-off frequency มาก (เช่น 1.0) ⟶ กรองน้อย (GLPF), จะพบ Noise หรือขอบน้อย (GHPF)

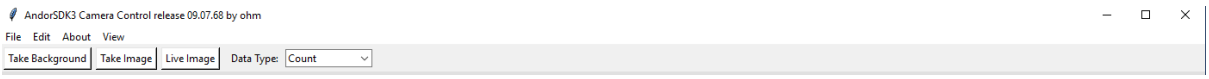
*** เมื่อตั้งค่าเสร็จต้องกดปุ่ม “Save values” ทุกครั้ง ***



รูป 13: แผนก Image Processing

5. แถบ Toolbar สำหรับสั่งใช้กล้อง sCMOS ให้ถ่ายภาพ

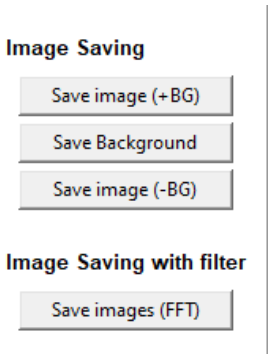
Toolbar ในซอฟท์แวร์ AndorSDK3 Camera Control ประกอบไปด้วย ปุ่ม “Take Background” สำหรับถ่ายภาพ background, ปุ่ม “Take image” สำหรับถ่ายภาพ, ปุ่ม “Live image” สำหรับถ่ายภาพแบบ live และ combo box สำหรับเลือก Data type สามารถเลือกได้ 2 โหมดได้แก่ “Count” และ “Bg Corrected”



รูป 14: แผนก Toolbar ของซอฟท์แวร์

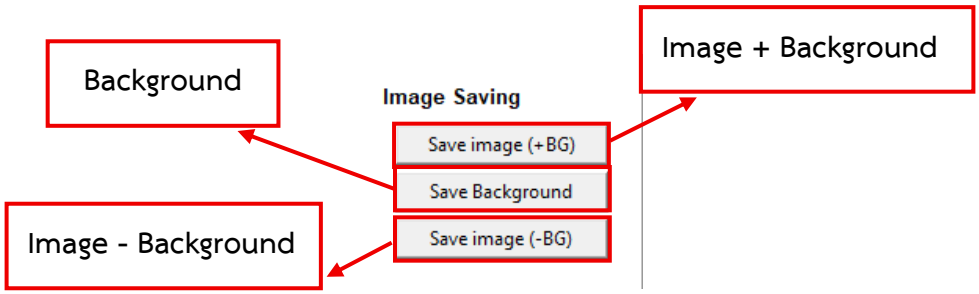
6. การบันทึกภาพถ่าย

แผนกการบันทึกภาพจะประกอบไปด้วย 2 ส่วนได้แก่ Image Saving และ Image Saving with filter



รูป 15: แผนก Image Saving และ Image Saving with filter

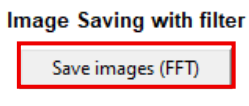
Image Saving



รูป 16: ข้อมูลการทำงานของปุ่มใน Image Saving

Image Saving with filter

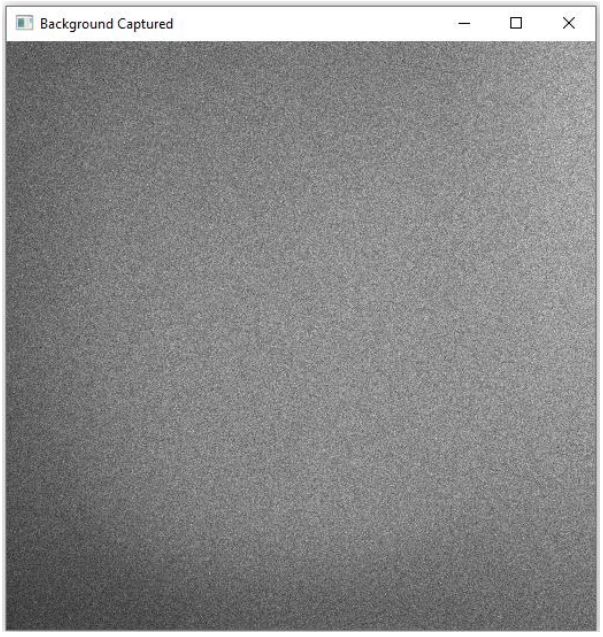
ในแผนกนี้เมื่อกดปุ่ม “Save images (FFT)” จะเป็นการบันทึกภาพที่ต้นฉบับและภาพที่ประมวลผลแล้ว (original image, magnitude spectrum, Filtered image)



รูป 17: แผนก Image Saving with filter

7. ตัวอย่างการถ่ายภาพในโหมดต่างๆ

7.1 การถ่ายภาพ Background



รูป 18: ตัวอย่างภาพ Background

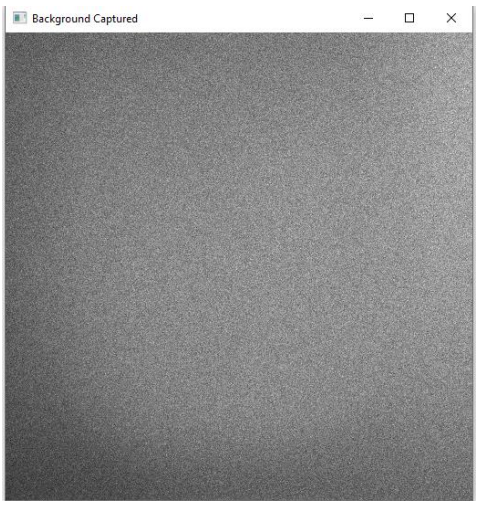
7.2 การถ่ายภาพ Background ที่ accumulate จำนวน 200 frames

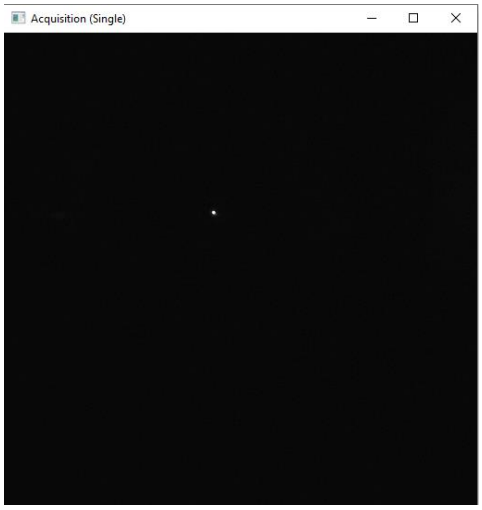
รูป 19: ตัวอย่างภาพ Background ที่ accumulate 200 เฟรม

7.3 การถ่ายภาพ accumulate (Count) จำนวน 200 frames

รูป 20: ตัวอย่างภาพ accumulate (Count) จำนวน 200 frames

7.4 การถ่ายภาพ Accumulate single BG จำนวน 200 frames และภาพ Background

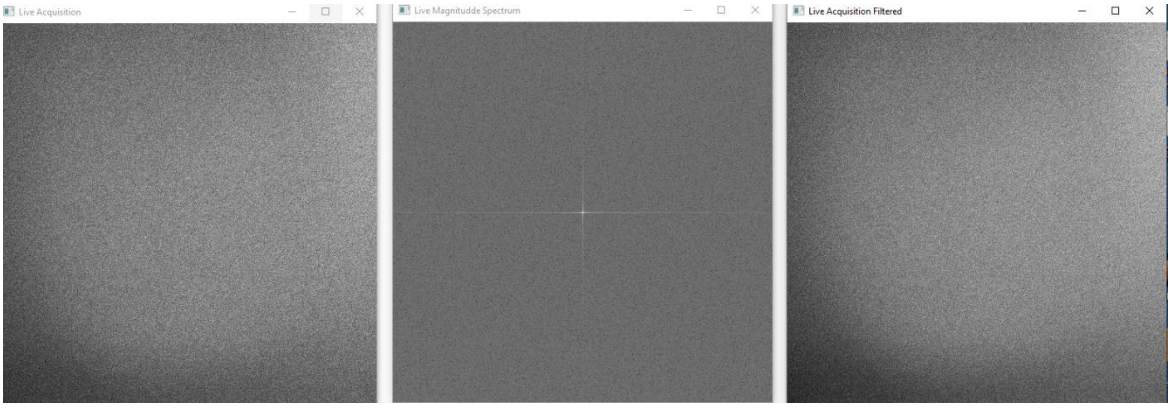




(ก) **(ข)**

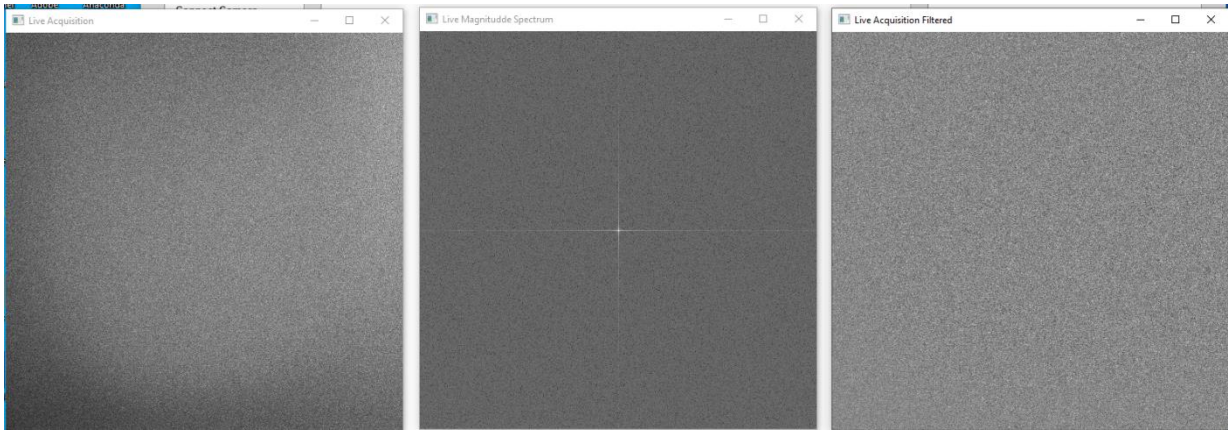
รูป 21: (ก) ตัวอย่างภาพ Background และ (ข) accumulate single BG จำนวน 200 frames

7.5 การถ่ายภาพ snap แบบ FFT และใช้ Gaussian low pass filter โดยมีภาพ original, magnitude spectrum, filtered image



รูป 22: ตัวอย่างภาพ snap แบบ FFT และใช้ Gaussian low pass filter

7.6 การถ่ายภาพ snap แบบ FFT และใช้ Gaussian high pass filter โดยมีภาพ original, magnitude spectrum, filtered image



รูป 23: ตัวอย่างภาพ snap แบบ FFT และใช้ Gaussian high pass filter

คำสั่ง Python ทั้งหมดของซอฟท์แวร์ AndorSDK3 Camera Control

Import library ทั้งหมดสำหรับซอฟท์แวร์ AndorSDK3 Camera Control

```
 1. import tkinter as tk
 2. from tkinter import ttk
 3. from tkinter import messagebox , filedialog
 4. from PIL import Image , ImageTk
 5. import threading
 6. from pylablib.devices import Andor
 7. import pylablib as pll
 8. import cv2
 9. import numpy as np
10. import os
11. from datetime import datetime
```

คำสั่ง Python กำหนดตัวแปรต่างๆแบบ global เพื่อรองรับพารามิเตอร์ในการควบคุมกล้อง sCMOS

```
  1. cam = None
  2. camera_settings = {
  3.                     "acquisition": None,
  4.                     "trigger_mode": None,
  5.                     "dynamic_range": None,
  6.                     "number_of_accumulate": None,
  7.                     "exposure_time": None,
  8.                     "hbin": None,
  9.                     "vbin": None,
 10.                     "hstart": None,
 11.                     "hend": None,
 12.                     "vstart": None,
 13.                     "vend": None,
 14.                     "data_type": None,
 15.                     "fft_mode": None,
 16.                     "filter_mode": None,
 17.                     "cut_off_frequency": None
 18.                    }
 19.
 20. background_image = None
 21. signal_bg_image = None
 22. signal_image = None
 23. original_image = None
 24. magnitude_image = None
 25. filtered_image = None
```

บรรทัดที่ 1 คือตัวแปรที่จะรองรับคำสั้งสำหรับควบคุมกล้อง sCMOS

บรรทัดที่ 2-18 ตัวแปร Camera_settings สำหรรับรองรับค่าพารามิเตอร์สำรหรับการถ่ายภาพ

บรรทัดที่ 20 - 25 คือตัวแปรที่กำหนดไว้สำหรับรองรับภาพที่ถ่ายจากโหมดต่างๆ

ฟังก์ชันคำสั่ง Python สำหรับ import ไฟล์ DLL ที่จะเป็นของกล้อง sCMOS

```
27. #Import dll and parameter
28. def set_dlls():
29.     #Open file dialog for find dlls
30.     dll_path = filedialog.askdirectory(
31.         title =  "Select AndorSDK3Camera dlls folder"
32.     )
33.     if not dll_path:
34.         messagebox.showwarning("Warning", "No DLL file selected.")
35.         return
36.
37.     #Check dlls files in folder
38.     dll_files = [f for f in os.listdir(dll_path) if f.lower().endswith(".dll")]
39.     if not dll_files:
40.         messagebox.showerror("Error", "No DLL files found in the selected folder.")
41.         return
42.
43.     pll.par["devices/dlls/andor_sdk3"] = dll_path
44.     #messagebox.showinfo("Success", f"Set DLL path:\n{dll_path}")
45.     dll_list_text = "\n".join(f"{fname} (DLL file)" for fname in dll_files)
46.     messagebox.showinfo("Success", f"Set DLL folder:\n{dll_path}\n\nFound
{len(dll_files)} DLL file(s):\n\n{dll_list_text}")
```

1. **บรรทัดที่ 30 - 31**

เป็นคำสั่ง File dialog สำหรักบเปิดหาโฟลเดอร์ที่มี dll สำหรับควบคุมกล้อง sCMOS

2. **บรรทัดที่ 33 - 35**

เป็นคำสั่งตรวจสอบ Error หากไม่ได้เลือกตำแหน่งของ folder จะมีการแจ้งเตือน “No DLL file selected.”

3. บรรทัดที่ 38 - 41

คำสั้งสำหรับตรวจสอบว่า folder ที่เลือกว่ามี dll file อยู่ข้างใน folder หากไม่มีจะแจ้งเตือน Error “No DLL file found in the selected folder.”

4. **บรรทัดที่ 43**

จะบันทึกตำแหน่งของ folder ของไฟล์ dll เพื่อนำตัวแปรนี้ไปใช้งานสำหรับเชื่อมต่อกับกล้อง sCMOS

5. บรรทัดที่ 45

เป็นต้นไป คือคำสั่งแจ้งเตือนชื่อไฟล์ dll ทั้งหมดที่ import เข้ามาจาก folder ที่เก็บไฟล์ dll ไว้

ฟังก์ชันคำสั่ง Python สำหรับจัดเรียงตำแหน่งและแถวภายใน Right Panel

```
49. #Right Panel row
50. def create_input_row(parent, label_text, input_widget, row):
51.     label = tk.Label(parent, text=label_text, bg = "white", anchor = "w", width = 18)
52.     label.grid(row = row, column = 0, sticky = "w", padx = 5, pady = 2)
53.     input_widget.grid(row = row, column = 1, sticky = "e", padx = 5, pady = 2)
```

1. บรรทัดที่ 51

คำสั่งสร้าง Label ใน parent โดยพารามิเตอร์ text = “ข้อความที่จะแสดง”,

bg = “สีของพื้นหลัง”, anchor = “จัดชิดซ้าย (West)”, width = “ความกว้าง 18 ตัวอักษร”

2. บรรทัดที่ 52

คำสั่งวาง Label ไว้ตามพารามิเตอร์ row ที่กำหนด, column 0 (ซ้ายสุด), sticky = “w” (ชิดซ้าย),

padx, pady = “ระยะห่างรอบข้างของแกน x และแกน y (padding)”

3. บรรทัดที่ 53

คำสั่งวาง input widget (เช่น ช่องกรอกข้อความ) ที่ column 1 (ขวาของ Label), sticky = “e” (ชิดขวา),

padx, pady = “ระยะห่างรอบข้างของแกน x และแกน y (padding)”

ฟังก์ชันคำสั่ง Python สำหรับเชื่อมต่อและตัดการเชื่อมต่อกับกล้อง sCMOS

```
55. # Connect the camera
56. def connect_camera():
57.     global cam
58.     #check the connected cconnect_camera
59.     cam_count = Andor.get_cameras_number_SDK3()
60.     if cam_count > 0:
61.         cam = Andor.AndorSDK3Camera()
62.         camera_status_label.config(text = "Camera : Connected", fg = "green")
63.     else:
64.         status_label.config(text = "Not Found Camera", fg = "red")
65.
66. #Disconeted
67. def disconnect_camera():
68.     global cam
69.     cam.close()
70.     cam = None
71.     camera_status_label.config(text = "Camera : Disconnected",fg = "red")
```

1. บรรทัดที่ 56

ฟังก์ชันคำสั่งสำหรับเชื่อมต่อกล้อง sCMOS กับ python

2. บรรทัดที่ 57

ดึงตัวแปร global cam

3. บรรทัดที่ 58 - 59

เป็นคำสั่งเช็คจำนวนกล้องที่เชื่อมต่ออยู่กับ python

4. บรรทัดที่ 60 – 64

เป็นคำสั่งเชื่อมตอ่กล้อง โดยจะเช็คเงื่อนไขหาก cam_count > 0 จะทำทำการเชื่อมต่อกล้องกับกล้อง sCMOS และ แสดงสถานะการเชื่อมต่อ “Connected” บริเวณแถบ status bar และหากไม่พบกล้อง จะแสดง “Not Found Camera”

5. บรรทัดที่ 67 - 71

ฟังก์ชันสำหรับตัดการเชื่อมต่อกล้องกับ python และแสดงสถานะ “Disconnected” บริเวณ status bar

ฟังก์ชันคำสั่ง Python สำหรับ เปิด/ปิด ระบบทำความเย็นของกล้อง sCMOS

```
74. # def fn cooler on
75. def cooler_on():
76.     global cam
77.     if cam is not None:
78.         cam.set_cooler(True)
79.         cooler_status_label.config(text = "Cooler : ON" , fg = "green")
80.     else:
81.         messagebox.showwarning("Warning", "Camera not connected")
82.
83. #def fn cooler off
84. def cooler_off():
85.     global cam
86.     if cam is not None:
87.         cam.set_cooler(False)
88.         cooler_status_label.config(text = "Cooler : OFF", fg = "red")
89.     else:
90.         messagebox.showwarning("Warning", "Camera not connected")
```

1. บรรทัดที่ 75 - 81

ฟังก์ชันเปิดระบบทำความเย็นของกล้อง sCMOS ซึ่งในฟังก์ชันนี้จะไม่สามารถตั้งค่าอุณหภูมิของกล้องได้ โดยการทำงานของฟังก์ชันนี้จะเช็คเงื่อนไขหาก เชื่อมต่อกล้องแล้วจะเปิดระบบทำความเย็นและ แสดงสถานะ Cooler = “ON” บริเวณ status bar และหากยังไม่ได้เชื่อมต่อกล้องจะแจ้งเตือน “Camera not connected”

2. บรรทัดที่ 84 - 90

ฟังก์ชันปิดระบบทำความเย็นของกล้อง sCMOS โดยการทำงานของฟังก์ชันนี้จะเช็คเงื่อนไขหาก เชื่อมต่อกล้องแล้วจะปิดระบบทำความเย็นและ แสดงสถานะ Cooler = “OFF” บริเวณ status bar และหากยังไม่ได้เชื่อมต่อกล้องจะแจ้งเตือน “Camera not connected”

ฟังก์ชันคำสั่ง Python สำหรับตั้งค่าอุณหภูมิและเปิดระบบทำความเย็นของกล้อง sCMOS

```
 92. #Set temperature
 93. def set_temp():
 94.     def apply_temp():
 95.         try:
 96.             temp_val = float(et1.get())
 97.             temp.destroy()
 98.             if cam is not None:
 99.                 cam.set_temperature(temp_val, enable_cooler = True)
100.                 cooler_status_label.config(text = "Cooler : ON", fg = "green")
101.             else:
102.                 messagebox.showwarning("Warning", "Camera not connected")
103.         except ValueError:
104.             messagebox.showerror("Error !!", "invalid temperature input")
105.
106.     temp = tk.Toplevel(root)
107.     temp.title("Set temperature")
108.     temp.geometry("300x80")
109.
110.     temp_set = tk.Label(temp, text = "Temperature (°C):")
111.     temp_set.pack(side = tk.LEFT, padx = 10)
112.
113.     #Entry box
114.     et1 = tk.Entry(temp, width = 8)
115.     et1.pack(side = tk.LEFT, padx = 0)
116.
117.     apply_btn = tk.Button(temp, text = "Apply", bg = "#4CAF50", fg = "white", width =
6, command = apply_temp)
118.     apply_btn.pack(side = tk.RIGHT, padx = 20)
119.
120.     #Bind Enter key
121.     temp.bind('<Return>', lambda event: apply_temp())
122.     et1.focus_set()
123.     temp.mainloop()
```

กล้อง Andor Sona sCMOS สามารถตั้งค่าอุณหภูมิได้ถึง -25 องศาเซลเซียส โดยที่ไม่ได้ใช้ระบบทำความเย็นเพิ่มเติม อ้างอิงจาก [Sona_Quick_Start_Guide.pdf (oxinst.com)]

1. บรรทัดที่ 93 – 104

ฟังก์ชัน apply_temp จะทำงานโดยรับค่าอุณหภูมิที่เป็นค่าทศนิยม (float) กรอกเข้ามายังกล่องข้อความ (Entry box) และเช็คเงื่อนไขดังนี้ ถ้าหากเชื่อมต่อกล้อง sCMOS แล้วจะเปิดการใช้งาน cooler และเซ็ตค่าอุณภูมิพร้อมกับแสดงสถานะ “Cooler : ON” หากยังไม่ได้เชื่อมต่อกล้องจะแจ้งเตือน “Camera not connected” ในฟังก์ชันนี้จะมีการตรวจจับข้อผิดพลาดการกรอกค่าใน Entry box หากกรอกค่าที่ไม่ใช่ float จะแจ้งเตือน ValueError

2. บรรทัดที่ 106 – 111

เป็นการสร้างหน้าต่าง GUI ย่อยขึ้นมาสำหรับกรอกค่าอุณภูมิในหน่วย องศาเซลเซียส

3. บรรทัดที่ 113 – 118

เป็น Entry box สำหรับกรอกค่าอุณภูมิบน GUI ย่อย และปุ่ม Apply เพื่อบันทึกค่าอุณหภูมิ

4. บรรทัดที่ 121 – 123

เป็นการเชื่อมต่อปุ่ม Apply กับคีย์ลัด <Return> หรือปุ่ม Enter บน keyboard ให้สามารถบันทึกค่าอุณหภูมิได้เลย

ฟังก์ชันคำสั่ง Python สำหรับแสดงอุณหภูมิของกล้อง sCMOS บน Status bar

```
125. #Show temperature
126. def get_temp():
127.     global cam
128.     if cam is None:
129.         messagebox.showwarning("Warning", "Camera not connected")
130.
131.     def update_temp():
132.         if cam is not None:
133.             try:
134.                 temp_val = "{:.2f}".format(cam.get_temperature())
135.                 temp_status = cam.get_attribute_value('TemperatureStatus')
136.
137.                 if temp_status == "Stabilised":
138.                     temp_status_label.config(text = f"Temperature : {temp_val} °C", fg
= "blue")
139.                 else:
140.                     temp_status_label.config(text = f"Temperature : {temp_val} °C", fg
= "red")
141.                     #Recall function every n (ms)
142.                     root.after(5000, update_temp)
143.             except Exception as e:
144.                 messagebox.showerror("Error", f"Temperature read error: {e}")
145.     update_temp()
```

1. บรรทัดที่ 126 – 129

เป็นการเช็คว่าเชื่อมต่อกับกล้อง sCMOS หากยังไม่ได้เชื่อมต่อจะแจ้งเตือน Warning

2. บรรทัดที่ 131-145

ฟังก์ชัน update_temp จะทำงานโดยแสดงค่าอุณหภูมิของกล้องบน status bar และมีการเช็คเงื่อนไขความเสถียรของอุณหภูมิ โดยบรรทัดที่ 137 จะเช็ค attribute_value ของกล้องถ้ารับค่า “Stabilised” จะแสดงค่าอุณหภูมิบน status bar เป็นสีน้ำเงิน และหากอุณหภูมิของกล้องยังไม่เสถียรจะแสดงค่าอุณหภูมิเป็นสีแดง และแสดงค่าทุกๆ 5 วินาทีจนกว่าอุณหภูมิจะเสถียร

ฟังก์ชันคำสั่ง Python สำหรับบันทึกการตั้งค่าและพารามิเตอร์ที่ใช้ถ่ายภาพด้วยกล้อง sCMOS

```
147. #Save Data (Right panel)
148. def save_data():
149.     global cam
150.     if cam is not None:
151.         # Combo box
152.         camera_settings["acquisition"] = acquisition_combo.get()
153.         camera_settings["trigger_mode"] = trigger_combo.get()
154.         camera_settings["dynamic_range"] = dynamic_combo.get()
155.
156.         if acquisition_combo.get() == "Accumulate" or "Accumulate (Single BG)":
157.             try:
158.                 num_acc = int(num_acc_entry.get())
159.                 if num_acc <= 0:
160.                     messagebox.showerror("Error", "Number of accumulate must be a
positive integer")
161.                     return
162.                 camera_settings["number_of_accumulate"] = num_acc
163.             except ValueError:
164.                 messagebox.showerror("Error", "Number of accumulate must be an
integer")
165.                 return
166.         else:
167.             camera_settings["number_of_accumulate"] = 1 #default (Single scan)
168.
169.         #Check value (float check)
170.         try:
171.             exposure_val = float(exposure_entry.get())
172.             camera_settings["exposure_time"] = exposure_val
173.         except ValueError:
174.             messagebox.showerror("Error", "Exposure time must be a number")
175.             return
```

1. บรรทัดที่ 148 – 154

ฟังก์ชัน save_data เป็นการเช็คว่าเชื่อมต่อกับกล้อง sCMOS หากยังไม่ได้เชื่อมต่อจะแจ้งเตือน Warning (คำสั่ง else อยู่ในบรรทัดที่ 261)

2. บรรทัดที่ 151 – 154

เป็นคำสั่งรับค่าจาก Combo box ที่อยู่ใน Right panel ในส่วนของ Camera Settings ได้แก่ acquisition, trigger mode, dynamic range

3. บรรทัดที่ 156 – 165

เป็นคำสั่งเปิดใช้งานให้กรอก Entry box ของ Number of Accumulate จะทำงานโดย เช็คเงื่อนไขถ้าเลือก acquisition เป็น Accumulate หรือ Accumulate (Single BG) จะเปิด Entry box ให้กรอกค่า จากนั้นจะเช็คเงื่อนไขรองโดยจะต้องกรอกค่าเป็นจำนวนเต็มบวกเท่านั้น หากไม่ตรงเงื่อนไขจะแจ้งเตือน Error และต้องกรอกค่าใหม่

4. บรรทัดที่ 166 -167

เป็นคำสั่งสำหรับเซ็ตค่า default ของ Number of Accumulate เป็น 1

(Single Scan)

5. บรรทัดที่ 170 – 175

เป็นคำสั่งเช็คเงื่อนไข float ของ Entry box ในส่วนของ Camera Settings ซึ่งมีเพียง Exposure time ที่สามารถรับค่าเป็น float ได้ หากไม่ตรงเงื่อนไขจะแจ้งเตือน “Exposure time must be a number”

```
177.         #Entry box
178.         camera_settings["exposure_time"] = float(exposure_entry.get())
179.         camera_settings["number_of_accumulate"] = int(num_acc_entry.get())
180.         camera_settings["hbin"] = int(hbin_entry.get())
181.         camera_settings["vbin"] = int(vbin_entry.get())
182.         camera_settings["hstart"] = int(hstart_entry.get())
183.         camera_settings["hend"] = int(hend_entry.get())
184.         camera_settings["vstart"] = int(vstart_entry.get())
185.         camera_settings["vend"] = int(vend_entry.get())
186.
187.         #Entry box int check
188.         int_check = {
189.             "number_of_accumulate" : num_acc_entry.get(),
190.             "hbin": hbin_entry.get(),
191.             "vbin": vbin_entry.get(),
192.             "hstart": hstart_entry.get(),
193.             "hend": hend_entry.get(),
194.             "vstart": vstart_entry.get(),
195.             "vend": vend_entry.get()
196.         }
197.
198.         for key, value in int_check.items():
199.             try:
200.                 int_check = int(value)
201.                 camera_settings[key] = int_check
202.             except ValueError:
203.                 messagebox.showerror("Error", f"{key} must be an integer")
204.
```

6. บรรทัดที่ 177 – 185

เป็นคำสั่งรับค่าจาก Entry box ใน Camera Settings ได้แก่ exposure time, hbin, vbin, hstart, hend, vstart, vend และเก็บค่าของแต่ละตัวแปรลงใน ตัวแปร global ของ camera_settings

7.บรรทัดที่ 187 – 203

เป็นคำสั่งตรวจสอบเงื่อนไขค่า int ของตัวแปร hbin, vbin, hstart, hend, vstart, vend

ในบรรทัดที่ 198 – 203 ถ้ากรอกค่าที่ไม่ใช่ int เข้ามาใน Entry box จะมีการแจ้งเตือน “(ชื่อตัวแปร) must be an integer”

```
205.         #Checking condition (Optional)
206.         if camera_settings["hstart"] >= camera_settings["hend"]:
207.             messagebox.showerror("Error", "hstart must be less than hend")
208.             return
209.
210.         if camera_settings["vstart"] >= camera_settings["vend"]:
211.             messagebox.showerror("Error", "vstart must be less than vend")
212.             return
213.
214.         if camera_settings["exposure_time"] <= 0:
215.             messagebox.showerror("Error", "Exposure time must be positive")
216.             return
217.
218.         if camera_settings["hbin"] <= 0 or camera_settings["vbin"] <= 0:
219.             messagebox.showerror("Error", "Bin value must be positive")
220.             return
```

8. บรรทัดที่ 205 – 220

เป็นคำสั่งตรวจเช็คค่าของตัวแปรที่กรอกเข้ามาใน Entry box เพื่อป้องกันข้อผิดพลาดเมื่อบันทึกค่า

```
222.         #Acquisition setup
223.         if camera_settings["acquisition"] == "Single scan":
224.             cam.set_attribute_value('AccumulateCount',
camera_settings["number_of_accumulate"])
225.         elif camera_settings["acquisition"] == "Accumulate":
226.             cam.set_attribute_value('AccumulateCount',
camera_settings["number_of_accumulate"])
227.         elif camera_settings["acquisition"] == "Accumulate (Single BG)":
228.             cam.set_attribute_value('AccumulateCount', 1)
229.
230.         #Trigger mode setup
231.         if camera_settings["trigger_mode"] == "Internal":
232.             cam.set_trigger_mode("int")
233.         elif camera_settings["trigger_mode"] == "External":
234.             cam.set_trigger_mode("ext")
235.         elif camera_settings["trigger_mode"] == "Software":
236.             cam.set_trigger_mode("software")
237.         elif camera_settings["trigger_mode"] == "External Exposure":
238.             cam.set_trigger_mode("ext_exp")
239.
240.         print(cam.get_trigger_mode())
241.
242.         #Dynamic range setup
243.         if camera_settings["dynamic_range"] == "Low noise 12-bit":
244.             cam.set_attribute_value('SimplePreAmpGainControl', '12-bit (low noise)')
245.         elif camera_settings["dynamic_range"] == "HDR 16-bit":
246.             cam.set_attribute_value('SimplePreAmpGainControl', '16-bit (high dynamic
range)')
247.         print(cam.get_attribute_value('SimplePreAmpGainControl'))
248.
```

```
249.         #Entry box setup (e.g. Eopusre time, ROI, Binning)
250.         cam.set_exposure(camera_settings["exposure_time"]) # set exposure
251.         hbin , vbin = camera_settings["hbin"] , camera_settings["vbin"]  #Set Binning
252.         #Set ROI  paremeter
253.         hstart , hend = camera_settings["hstart"], camera_settings["hend"]
254.         vstart , vend = camera_settings["vstart"] , camera_settings["vend"]
255.         #set ROI
256.         cam.set_roi(hstart = hstart, hend = hend, vstart = vstart, vend = vend, hbin =
hbin, vbin = vbin)
257.
258.         print(cam.get_settings())
259.         messagebox.showinfo("Camera Settings", "The camera settings are saved")
260.
261.     else:
262.         messagebox.showwarning("Warning", "Camera not connected")
```

9. บรรทัดที่ 222 – 228

Acquisition setup เป็นคำสั่งนำค่าที่เก็บในตัวแปร camera_settings [“acquisition”] มาบันทึกลงใน attribute_value('AccumulateCount') ซึ่งเป็นคำสั่งสำหรับควบคุมโหมดการถ่ายภาพของกล้อง sCMOS

10. บรรทัดที่ 230 – 238

Trigger mode setup เป็นคำสั่งนำค่าที่เก็บในตัวแปร camera_settings [“trigger_mode”] มาบันทึกลงในคำสั่ง set_trigger_mode ซึ่งเป็นคำสั่งสำหรับเลือกโหมดของ trigger เพื่อใช้ในถ่ายภาพของกล้อง sCMOS

11. บรรทัดที่ 242 – 246

Dynamic range setup เป็นคำสั่งนำค่าที่เก็บในตัวแปร camera_settings [“dynamic_range”] มาบันทึกลงใน attribute_value('SimplePreAmpGainControl') ซึ่งเป็นคำสั่งสำหรับเลือกโหมด dynamic range มีอยู่ 2 โหมดได้แก่ 12-bit (low noise) และ 16 bit (high dynamic range)

12. บรรทัดที่ 249 – 262

เป็นคำสั่งนำเอาค่าที่เก็บไว้ใน camera_settings ที่รับเข้ามาโดย Entry box มาบันทึกลงในพารามิเตอร์ของคำสั่งต่างๆ และเมื่อกด ปุ่ม Save settings จะเป็นการยืนยันการบันทึกค่าทั้งหมดในส่วนของ Camera Settings

ฟังก์ชันคำสั่ง Python สำหรับถ่ายภาพ Background ด้วยกล้อง sCMOS

```
264. ##Toolbar
265. #Take Background
266. def take_background():
267.     global cam , background_image
268.     if cam is not None:
269.         #Setup acquisition
270.         cam.setup_acquisition(mode = 'sequence', nframes = 100)
271.         cam.start_acquisition()
272.         print("Take background acquisition started")
273.
274.         #Take initial background
275.         cam.wait_for_frame(since = 'lastread', nframes = 1, timeout = 20,
error_on_stopped = False)
276.         background = np.float32(cam.read_newest_image())
277.         print(background)
278.         print("Take initial background")
279.
280.         #Normalize for output display
281.         bg_vis = cv2.normalize(background, None, 0, 65535,
cv2.NORM_MINMAX).astype(np.uint16)
282.
283.         #Resize for better viewing
284.         display_size = (512,512) #(width, height)
285.         bg_vis_resiezed = cv2.resize(bg_vis, display_size, interpolation =
cv2.INTER_NEAREST)
286.         cam.stop_acquisition()
287.         print("Take background acquisition stopped")
288.
289.         #Save for later use
290.         background_image = background.copy()
291.         background_vis = bg_vis.copy()
292.
293.         #Display
294.         cv2.imshow("Background Captured", bg_vis_resiezed)
295.         cv2.waitKey(0)
296.         cv2.destroyAllWindows()
297.         return
298.
299.     else:
300.         messagebox.showerror("AcquisitionError", "Camera not connected")
```

ฟังก์ชัน take_background คำสั่ง Python จะทำงานดังนี้

1. เงื่อนไขตรวจสอบการเชื่อมต่อกับกล้อง sCMOS

ในบรรทัดที่ 266 เป็นการเช็คว่าเชื่อมต่อกับกล้อง sCMOS หากยังไม่ได้เชื่อมต่อจะแจ้งเตือน AcquisitionError (คำสั่ง else อยู่ในบรรทัดที่ 300)

2. Set up Acquisition ของการถ่ายภาพ background

เมื่อเชื่อมต่อกล้องแล้วจะทำงานดังนี้ ตั้งค่า acquisition โหมดสามารถเป็นได้ทั้ง snap หรือ sequence (การเก็บข้อมูลอย่างต่อเนื่อง) nframes ใช้กำหนดจำนวนเฟรมที่จะเก็บใน single mode หรือขนาดของ buffer ในโหมด sequence (default คือ 100)

3. การถ่ายภาพ background ด้วยกล้อง sCMOS

เมื่อ Set up acquisition แล้วจะเป็นคำสั่งให้เริ่ม acquisiton โดยกล้องจะเก็บข้อมูลภาพตาม setup_acquisition ที่ตั้งค่าไว้ ในบรรทัดที่ 275 เป็นคำสั่งให้กล้อง คงสถานะการเก็บภาพ wait_for_frame โดยตั้งค่าไว้ที่ 20 วินาทีเพื่อไม่ให้เกิด Time out ในระหว่างการถ่ายภาพ และในบรรทัดที่ 276 เป็นคำสั่งที่กล้องจะอ่านภาพล่าสุดจาก buffer

4. คำสั่ง Python การทำ Normalize และปรับขนาดหน้าต่างสำหรับแสดงผล

ต่อไป เป็นคำสั่ง Python การทำ normalize ให้ค่าข้อมูลของพิกเซลต่ำสุดเป็น 0 และค่าสูงสุดเป็น 1 เพื่อที่จะนำไปแปลงเป็นข้อมูล Gray scale ชนิด 16-bit ในบรรทัดที่ 281 สำหรับการแสดงผลภาพที่ถ่าย

ในบรรทัดที่ 283 เป็นคำสั่งปรับขนาดของหน้าต่างแสดงผลภาพ โดยสามารถปรับขนาดได้จากตัวแปร display_size

5. คำสั่ง Python สำหรับ stop acquisition ของการถ่ายภาพ background

บรรทัดที่ 286 เป็นคำสั่งหยุดการทำงานของ acquisition และในบรรทัด 290 จะเป็นการเก็บ array ของภาพที่ยังไม่ได้ normalize ไว้ในตัวแปร background_image ของตัวแปร global เพื่อนำไปใช้ต่อในฟังก์ชั่นอื่นๆ

6. คำสั่ง Python สำหรับแสดงผลด้วย open cv

บรรทัดที่ 294 – 297 เป็นคำสั่งการแสดงผลที่ทำการ normalize แล้วด้วย open cv

ฟังก์ชันคำสั่ง Python สำหรับอัพเดทสถานบน Status bar และ เปิด/ปิด Combo box ต่างๆ

```
302. #Data type updating (Tool bar)
303. def on_data_type_change(event):
304.     camera_settings["data_type"] = data_type_combo.get()
305.     data_type_status_label.config(text = f"Data Type : {camera_settings['data_type']}")
306.
307. #Data type updating (Tool bar)
308. def on_acquisition_change(event = None):
309.     camera_settings["acquisition"] = acquisition_combo.get()
310.     if camera_settings["acquisition"] == "Accumulate" or camera_settings["acquisition"]  == "Accumulate (Single BG)":
311.         num_acc_entry.config(state = "normal")
312.     else:
313.         num_acc_entry.delete(0, tk.END)
314.         num_acc_entry.insert(0, "1")
315.         num_acc_entry.config(state = "disable")
316.
317.     # Check acquisition mode
318.     if acquisition_combo.get() == "Accumulate (Single BG)":
319.         data_type_combo.set("Bg Corrected")
320.         data_type_combo.config(state = "disable")
321.         camera_settings["data_type"] = "Bg Corrected"
322.
323.         #Update status label
324.         data_type_status_label.config(text = f"Data Type: {camera_settings['data_type']}")
325.     else:
326.         data_type_combo.config(state = "normal")
327.         data_type_status_label.config(text = f"Data Type: {camera_settings['data_type']}")
328.
329.
330.     if acquisition_combo.get() == "Accumulate" or acquisition_combo.get() == "Accumulate (Single BG)":
331.         num_acc_entry.config(state = "normal")
332.     else:
333.         num_acc_entry.delete(0, tk.END)
334.         num_acc_entry.insert(0, "1")
335.         num_acc_entry.config(state = "disable")
```

1. บรรทัดที่ 303 – 305 คำสั่ง Python สำหรับอัปเดต Data Type จาก Tool Bar

บรรทัดที่ 303 คือคำสั่งประกาศฟังก์ชัน on_data_type_change สำหรับจัดการเมื่อมีการเปลี่ยนแปลง Data Type จาก Combo Box

บรรทัดที่ 304 คือคำสั่งอัปเดตค่าการตั้งค่า camera_settings["data_type"] ให้ตรงกับค่าที่เลือกจาก data_type_combo

บรรทัดที่ 305 คือคำสั่งอัปเดตข้อความใน label แสดงสถานะ เพื่อแสดง Data Type ที่เลือกปัจจุบัน

2. บรรทัดที่ 308 – 335 คำสั่ง Python สำหรับอัปเดต Acquisition Mode และตั้งค่าอื่นๆ อัตโนมัติ

บรรทัดที่ 308 – 335 คือฟังก์ชัน on_acquisition_change ใช้สำหรับอัปเดต Acquisition Mode ที่เลือกจาก Combo Box และจะทำงานดังต่อไปนี้

คำสั่งตั้งค่าการเปิด/ปิดช่องกรอก Number of Accumulate อัตโนมัติ ถ้าเลือก "Accumulate" หรือ "Accumulate (Single BG)" จะเปิดให้กรอก ถ้าเลือก Mode อื่น จะตั้งค่าเป็น 1 และปิดช่องกรอก และกรณีเลือก "Accumulate (Single BG)" บังคับ Data Type เป็น "Bg Corrected" และปิดไม่ให้เปลี่ยนอัปเดต Label สถานะ Data Type ให้ตรงกับค่าที่บังคับ

ฟังก์ชันคำสั่ง Python สำหรับบันทึกการตั้งค่าการประมวลผล FFT ก่อนเริ่มการถ่ายภาพด้วยกล้อง sCMOS

```
337. #Image Processing Section
338. def on_fft_mode_change(event=None):
339.     camera_settings["fft_mode"] = fft_combo.get()
340.     camera_settings["filter_mode"] = filter_mode_combo.get()
341.     camera_settings["cut_off_frequency"] = cutoff_entry.get()
342.
343.     if camera_settings["fft_mode"] == "ON":
344.         # Enable filter mode selection
345.         filter_mode_combo.config(state="normal")
346.         cutoff_entry.config(state = "normal")
347.
348.     else:
349.         filter_mode_combo.config(state="disable")
350.         cutoff_entry.config(state="disable")
351.
352. #Save image process setting
353. def save_process_settings():
354.     camera_settings["fft_mode"] = fft_combo.get()
355.     camera_settings["filter_mode"] = filter_mode_combo.get()
356.
357.     if camera_settings["fft_mode"] == "ON":
358.         try:
359.             cutoff = float(cutoff_entry.get())
360.             camera_settings["cut_off_frequency"] = cutoff
361.         except ValueError:
362.             messagebox.showerror("ValueError", "Cut-off must be a valid float
number.")
363.             return
364.         messagebox.showinfo("Image Processing Settings", "saved successfully.")
365.     else:
366.         messagebox.showerror("Error", "FFT mode is OFF.")
```

1. บรรทัดที่ 337 – 350 ฟังก์ชัน on_fft_mode_change

ฟังก์ชัน on_fft_mode_change จะนำรับค่าจาก Combo box ในส่วนของ Image processing ได้แก่ FFT Mode , Filter Mode และรับค่าจาก Entry box ของ cut off frequency จากนั้นจะตรวจสอบเงื่อนไขถ้า fft_mode = ON จะเปิดการใช้งาน FFT Mode combo box และ cut off frequency entry box หากไม่ตรงเงื่อนไขจะปิดการใช้งานเพื่อป้องการข้อผิดพลาดในการเลือกโหมดถ่ายภาพของซอฟท์แวร์

2. บรรทัดที่ 353 – 366 ฟังก์ชัน save_pocess_settings

ฟังก์ชัน save_process_settings เป็นฟังก์ชันสำหรับบันทึกค่าในส่วนของ Image Processing โดยจะตรวจสอบเงื่อนไขถ้า fft_mode = ON จะมีการตรวจสอบค่า float ของ cut off frequency ในตัวแปร camera_settings[“cut_off_frequency”] หากไม่เป็น float จะแจ้งเตือน ValueError และต้องกรอกข้อมูลใหม่ หากข้อมูลถูกต้องจะแจ้งเตือน “save- successfully” ในบรรทัดที่ 365 - 366 หาก fft_mode ไม่ตรงเงื่อนไขจะแจ้งเตือน Error

ฟังก์ชันคำสั่ง Python สำหรับถ่ายภาพ Snap (No Filters) ในโหมด Count ด้วยกล้อง sCMOS

```
 369. #Take Acquisition (Snap)
 370. def snap_image():
 371.     global cam, signal_bg_image, background_image , signal_image, original_image,
magnitude_image, filtered_image
 372.     camera_settings["data_type"] = data_type_combo.get()
 373.     camera_settings["acquisition"] = acquisition_combo.get()
 374.     camera_settings["fft_mode"] = fft_combo.get()
 375.     camera_settings["filter_mode"] = filter_mode_combo.get()
 376.     camera_settings["cut_off_frequency"] = cutoff_entry.get()
 377.
 378.     if cam is None:
 379.         messagebox.showerror("AcquisitionError", "Camera not connected")
 380.         return
 381.
 382.     if camera_settings["fft_mode"] == "OFF":
 383.         if camera_settings["data_type"] == "Count" and camera_settings["acquisition"]
in ["Single scan", "Accumulate"]:
 384.             #Start Acquisition
 385.             cam.setup_acquisition(mode = 'sequence', nframes = 100) #Default
 386.             cam.start_acquisition()
 387.             print("Aqcuisition started")
 388.             #Take signal image
 389.             cam.wait_for_frame(since = 'lastread', nframes = 1, timeout = 20,
error_on_stopped = False)
 390.             frame = np.float32(cam.read_newest_image())
 391.
 392.             #Normalize for output display
 393.             vis2 = cv2.normalize(frame, None, 0, 65535,
cv2.NORM_MINMAX).astype(np.uint16)
 394.
 395.             #Resize
 396.             display_size = (512, 512)d
 397.             resize_img = cv2.resize(vis2, display_size, interpolation =
cv2.INTER_NEAREST)
 398.             cam.stop_acquisition() #Stop Acquisition
 399.             print("Aqcuisition stopped")
 400.
 401.             #Save for later use
 402.             signal_bg_image = frame.copy()
 403.
 404.             #Display
 405.             cv2.imshow("Acquisition", resize_img)
 406.             cv2.waitKey(0)
 407.             cv2.destroyAllWindows()
 408.             return
```

ฟังก์ชัน snap_image โหมด Count และ No filter คำสั่ง Python จะทำงานดังนี้

1. บรรทัดที่ 371 – 380 เงื่อนไขตรวจสอบการเชื่อมต่อกับกล้อง sCMOS และเรียกตัวแปร global

เรียกตัวแปร global ในบรรทัดที่ 371 ที่เกี่ยวข้องและ กำหนดให้ตัวแปร camera_settings รับค่าจาก Combo box และ Entry box จากนั้นในบรรทัดที่ 378 – 380 คือคำสั่งตรวจสอบการเชื่อมต่อกับกล้อง sCMOS หากไม่ได้เชื่อมต่อจะแจ้งเตือน AcquisitionError

2. บรรทัดที่ 382 – 386 Set up Acquisition ของการถ่ายภาพ

ในบรรทัดที่ 382 จะเป็นเงื่อนไขตรวจสอบ Filter Mode โดยจะให้ตัวแปร camera_settings[“fft_mode”] เป็น OFF และเช็คเงื่อนไขของ Acquisition mode ในบรรทัดที่ 383 โดย data_type จะต้องเท่ากับ “Count” และ acquisition จะต้องเป็น “Single scan” หรือ “Accumulate” จากนั้นเป็นการตั้งค่า acquisition โหมด สามารถเป็นได้ทั้ง snap หรือ sequence (การเก็บข้อมูลอย่างต่อเนื่อง) nframes ใช้กำหนดจำนวนเฟรมที่จะเก็บใน single mode หรือขนาดของ buffer ในโหมด sequence (default คือ 100)

3. บรรทัดที่ 388 – 390 การถ่ายภาพด้วยกล้อง sCMOS

เมื่อ Set up acquisition แล้วจะเป็นคำสั่งให้เริ่ม acquisiton โดยกล้องจะเก็บข้อมูลภาพตาม setup_acquisition ที่ตั้งค่าไว้ ในบรรทัดที่ 389 เป็นคำสั่งให้กล้อง คงสถานะการเก็บภาพ wait_for_frame โดยตั้งค่าไว้ที่ 20 วินาทีเพื่อไม่ให้เกิด Time out ในระหว่างการถ่ายภาพ และในบรรทัดที่ 390 เป็นคำสั่งที่กล้องจะอ่านภาพล่าสุดจาก buffer

4. บรรทัดที่ 393 – 397 คำสั่ง Python การทำ Normalize และปรับขนาดหน้าต่างสำหรับแสดงผล

คำสั่ง Python การทำ normalize ให้ค่าข้อมูลของพิกเซลต่ำสุดเป็น 0 และค่าสูงสุดเป็น 1 เพื่อที่จะนำไปแปลงเป็นข้อมูล Gray scale ชนิด 16-bit ในบรรทัดที่ 293 สำหรับการแสดงผลภาพที่ถ่าย

ในบรรทัดที่ 397 เป็นคำสั่งปรับขนาดของหน้าต่างแสดงผลภาพ โดยสามารถปรับขนาดได้จากตัวแปร display_size

5. บรรทัดที่ 398 - 402 คำสั่ง Python สำหรับ stop acquisition ของการถ่ายภาพ

บรรทัดที่ 398 เป็นคำสั่งหยุดการทำงานของ acquisition และในบรรทัด 402 จะเป็นการเก็บ array ของภาพที่ยังไม่ได้ normalize ไว้ในตัวแปร signal_bg_image ของตัวแปร global เพื่อนำไปใช้งานต่อในฟังก์ชั้นอื่นๆ

6. บรรทัดที่ 405 – 408 คำสั่ง Python สำหรับแสดงผลด้วย open cv

บรรทัดที่ 405 – 408 เป็นคำสั่งการแสดงผลที่ทำการ normalize แล้วด้วย open cv

ฟังก์ชันคำสั่ง Python สำหรับถ่ายภาพ Snap (No Filters) ในโหมด Bg Corrected ด้วยกล้อง sCMOS

```
410.         #Take image without filters (Bg Corrected)
411.         elif camera_settings["data_type"] == "Bg Corrected" and
camera_settings["acquisition"] in ["Single scan", "Accumulate"]:
412.             if background_image is not None:
413.                 #Start Acquisition
414.                 cam.setup_acquisition(mode = 'sequence', nframes = 100) #Default
415.                 cam.start_acquisition()
416.                 print("Aqcuisition started")
417.                 #Take signal image
418.                 cam.wait_for_frame(since = 'lastread', nframes = 1, timeout = 20,
error_on_stopped = False)
419.                 frame = np.float32(cam.read_newest_image())
420.
421.                 #Background Corrected
422.                 signal = frame - background_image
423.                 print(signal)
424.
425.                 #Normalize for output display
426.                 vis2 = cv2.normalize(signal, None, 0, 65535,
cv2.NORM_MINMAX).astype(np.uint16)
427.
428.                 #Resize
429.                 display_size = (512, 512)
430.                 resize_img = cv2.resize(vis2, display_size, interpolation =
cv2.INTER_NEAREST)
431.                 cam.stop_acquisition() #Stop Acquisition
432.                 print("Aqcuisition stopped")
433.
434.                 #Save for later use
435.                 signal_bg_image = frame.copy()
436.                 signal_image = signal.copy()
437.
438.                 #Display
439.                 cv2.imshow("Acquisition", resize_img)
440.                 cv2.waitKey(0)
441.                 cv2.destroyAllWindows()
442.                 return
443.
444.             else:
445.                 messagebox.showerror("Acqusition Error", "Background image not found")
446.                 return
```

ฟังก์ชัน snap_image โหมด Bg Corrected และ No filter คำสั่ง Python จะทำงานดังนี้

1. บรรทัดที่ 411 – 412 เป็นการตรวจสอบเงื่อนไข elif จากตัวแปร global ที่เรียกมาใช้

ตรวจสอบเงื่อนไขของ Acquisition mode ในบรรทัดที่ 411 โดย data_type จะต้องเท่ากับ “Bg Corrected” และ acquisition จะต้องเป็น “Single scan” หรือ “Accumulate” ในบรรทัดที่ 412 คือการตรวจสอบถ่ายภาพ background หากยังไม่ได้ถ่ายจะแจ้งเตือน Acquisition Error

2. บรรทัดที่ 414 – 415 Set up Acquisition ของการถ่ายภาพ

ในบรรทัดที่ 414 โดย data_type จะต้องเท่ากับ “Bg Corrected” และ acquisition จะต้องเป็น “Single scan” หรือ “Accumulate” จากนั้นเป็นการตั้งค่า acquisition โหมดสามารถเป็นได้ทั้ง snap หรือ sequence (การเก็บข้อมูลอย่างต่อเนื่อง) nframes ใช้กำหนดจำนวนเฟรมที่จะเก็บใน single mode หรือขนาดของ buffer ในโหมด sequence (default คือ 100)

3. บรรทัดที่ 418 – 419 การถ่ายภาพด้วยกล้อง sCMOS

เมื่อ Set up acquisition แล้วจะเป็นคำสั่งให้เริ่ม acquisiton โดยกล้องจะเก็บข้อมูลภาพตาม setup_acquisition ที่ตั้งค่าไว้ ในบรรทัดที่ 418 เป็นคำสั่งให้กล้อง คงสถานะการเก็บภาพ wait_for_frame โดยตั้งค่าไว้ที่ 20 วินาทีเพื่อไม่ให้เกิด Time out ในระหว่างการถ่ายภาพ และในบรรทัดที่ 419 เป็นคำสั่งที่กล้องจะอ่านภาพล่าสุดจาก buffer

4. บรรทัดที่ 422 คำสั่ง Python การทำ Background Corrected

ในบรรทัดที่ 422 จะเป็นการนำเอาข้อมูล array ของ image มาลบกับข้อมูลภาพ background และเก็บค่าไว้ในตัวแปร signal

5. บรรทัดที่ 426 – 430 คำสั่ง Python การทำ Normalize และปรับขนาดหน้าต่างสำหรับแสดงผล

คำสั่ง Python การทำ normalize ให้ค่าข้อมูลของพิกเซลต่ำสุดเป็น 0 และค่าสูงสุดเป็น 1 เพื่อที่จะนำไปแปลงเป็นข้อมูล Gray scale ชนิด 16-bit ในบรรทัดที่ 293 สำหรับการแสดงผลภาพที่ถ่าย

ในบรรทัดที่ 430 เป็นคำสั่งปรับขนาดของหน้าต่างแสดงผลภาพ โดยสามารถปรับขนาดได้จากตัวแปร display_size

6. บรรทัดที่ 431 - 436 คำสั่ง Python สำหรับ stop acquisition ของการถ่ายภาพ

บรรทัดที่ 431 เป็นคำสั่งหยุดการทำงานของ acquisition ในบรรทัดที่ 435 และ 436 จะเป็นการเก็บ array ของภาพที่ยังไม่ได้ normalize ไว้ในตัวแปร signal_bg_image, signal_image ของตัวแปร global เพื่อนำไปใช้ต่อในฟังก์ชั้นอื่นๆ

7. บรรทัดที่ 439 – 442 คำสั่ง Python สำหรับแสดงผลด้วย open cv

บรรทัดที่ 439 - 442 เป็นคำสั่งการแสดงผลภาพที่ normalize แล้วด้วย open cv

ฟังก์ชันคำสั่ง Python สำหรับถ่ายภาพ Snap (Filters) ในโหมด Count ด้วยกล้อง sCMOS

```
    #Take image with FFT & Filters (Snap)
    if camera_settings["fft_mode"] == "ON":
        if camera_settings["data_type"] == "Count" and camera_settings["acquisition"] in ["Single scan", "Accumulate"]:
            #Check cut off input
            try:
                cutoff = float(cutoff_entry.get())
                camera_settings["cut_off_frequency"] = cutoff
            except ValueError:
                messagebox.showerror("ValueError", "Cut-off frequency must be a number (float).")
                return

            #Gaussian low-pass ffilter
            def gaussian_lowpass(size, cutoff):
                x = np.linspace(-1, 1, cols)
                y = np.linspace(-1, 1, rows)
                xv, yv = np.meshgrid(x, y)
                d = np.sqrt(xv**2 + yv**2)

                return np.exp(-(d**2) / (2 * (cutoff**2)))

            #Gaussian high-pass filter
            def gaussian_highpass(size, cutoff):
                return 1 - gaussian_lowpass(size, cutoff)

            #Start Acquisition
            cam.setup_acquisition(mode = 'sequence', nframes = 100) #Default
            cam.start_acquisition()
            print('Acquisition started')

            #Started Acquisition
            cam.wait_for_frame(since = 'lastread', nframes = 1 , timeout = 20 , error_on_stopped = False)
            frame = np.float32(cam.read_newest_image())
            print("original shape =", frame.shape)
            print("original = ", frame)

            #FFT 2D Processing
            f_transform = np.fft.fft2(frame)
            f_shift = np.fft.fftshift(f_transform)
            magnitude_spectrum = np.abs(f_shift)
            magnitude_spectrum = np.log1p(magnitude_spectrum)
            print("magnitude = ", magnitude_spectrum)

            #Normalize image & magnitude for output
            vis2 = cv2.normalize(frame, None, 0, 65535, cv2.NORM_MINMAX).astype(np.uint16)
            magnitude_vis = cv2.normalize(magnitude_spectrum, None, 0, 65535, cv2.NORM_MINMAX).astype(np.uint16)

            #Create a filter mask
            rows, cols = f_shift.shape
            crow ,ccol = rows // 2, cols // 2
            cutoff = camera_settings["cut_off_frequency"]
            gaussian_lp_filter = gaussian_lowpass((rows, cols), cutoff = cutoff )
```

```
            gaussian_hp_filter = gaussian_highpass((rows, cols), cutoff = cutoff)

            #Apply the filters (check condition of GLPF, GHPF)
            if camera_settings["filter_mode"] == "Low Pass (GLPF)":
                f_filtered_lp = f_shift * gaussian_lp_filter # Apply Gaussain low-pass filter
                #Inverse FFT
                f_ishift_lp = np.fft.ifftshift(f_filtered_lp)
                img_back_lp = np.fft.ifft2(f_ishift_lp)
                img_back_lp = np.real(img_back_lp)
                img_back_lp = img_back_lp.astype(int)

                image_vis = img_back_lp.copy()

            elif camera_settings["filter_mode"] == "High Pass (GHPF)":
                f_filtered_hp = f_shift * gaussian_hp_filter  #Apply Gaussian High-pass filter
                #Inverse FFT
                f_ishift_hp = np.fft.ifftshift(f_filtered_hp)
                img_back_hp = np.fft.ifft2(f_ishift_hp)
                img_back_hp = np.real(img_back_hp)
                img_back_hp = img_back_hp.astype(int)

                image_vis = img_back_hp.copy()
                print("filtered shape =", img_back_hp.shape)

            #Save for later use
            print("filtered = ",image_vis)
            signal_bg_image = frame.copy()
            original_image = frame.copy()
            magnitude_image = magnitude_spectrum.copy()
            filtered_image = image_vis.copy()

            #Normalize  filtered  for display
            display_size = (512,512)
            image_fil_vis = cv2.normalize(image_vis, None, 0, 65535, cv2.NORM_MINMAX).astype(np.uint16)

            #Resize
            resize = cv2.resize(vis2, display_size, interpolation = cv2.INTER_NEAREST)
            resize_mag = cv2.resize(magnitude_vis, display_size, interpolation = cv2.INTER_NEAREST)
            resize_vis = cv2.resize(image_fil_vis, display_size, interpolation = cv2.INTER_NEAREST)

            #Display
            cv2.imshow("Live Acquisition", resize)
            cv2.imshow("Live Magnitudde Spectrum", resize_mag)
            cv2.imshow("Live Acquisition Filtered", resize_vis)

            #Stop Acquisition
            cam.stop_acquisition()
            print('Acquisition stopped')
            cv2.waitKey(0)
            cv2.destroyAllWindows()
            return
```

ฟังก์ชัน snap_image โหมด Count และ Filters คำสั่ง Python จะทำงานดังนี้

1. บรรทัดที่ 449 – 457 เงื่อนไขตรวจสอบการเปิดใช้งาน FFT Mode จากตัวแปร global

เงื่อนไขตรวจสอบการเบิดใช้การ FFT Mode และตรวจสอบเงื่อนไขในตัวแปร camera_settings โดย data_type และ acquisition ต้องเป็น “Count” , “Single scan” หรือ “Accumulate” จากนั้น ในบรรทัดที่ 452 – 457 คือคำสั่งตรวจสอบการรับค่า float ของ cut off frequency

2. บรรทัดที่ 460 – 470 กำหนดฟังก์ชัน Gaussian low-pass filter และ Gaussian high-pass filter

ในบรรทัดที่ 460 – 466 คือฟังก์ชัน Gaussian low pass filter เพื่อเรียกไปใช้สำหรับการประมวลผลภาพ ในบรรทัดที่ 468 – 470 คือฟังก์ชัน Gaussian high pass filter เพื่อเรียกไปใช้สำหรับการประมวลผลภาพ

3. บรรทัดที่ 473 – 479 การ Set up acquisition และการถ่ายภาพด้วยกล้อง sCMOS

ในบรรทัดที่ 473 – 474 เป็นคำสั่งสำหรับ set up acquisition และเริ่ม acquisiton โดยกล้องจะเก็บข้อมูลภาพตาม setup_acquisition ที่ตั้งค่าไว้ ในบรรทัดที่ 478 เป็นคำสั่งให้กล้อง คงสถานะการเก็บภาพ wait_for_frame โดยตั้งค่าไว้ที่ 20 วินาทีเพื่อไม่ให้เกิด Time out ในระหว่างการถ่ายภาพ และในบรรทัดที่ 479 เป็นคำสั่งที่กล้องจะอ่านภาพล่าสุดจาก buffer

4. บรรทัดที่ 484 – 487 คำสั่ง Python การประมวลผลภาพด้วย FFT

ในบรรทัดที่ 484 เป็นการนำตัวแปร frame เป็นข้อมูล array ของภาพถ่ายมาทำ FFT โดยจะแปลงค่าจาก spatial domain ให้ไปอยู่ใน frequency domain เพื่อประมวลผลภาพที่อยู่ใน frequency domain ต่อ ในบรรทัดที่ 485 คือการ shift ความค่าความถี่ใน frequency domain ให้มาอยู่บริเวณตรงกลางของภาพ ในบรรทัดที่ 486 – 487 การคำนวน Magnitude spectrum ของการทำ FFT หรือเป็นภาพที่อยู่ใน frequency domain

4. บรรทัดที่ 491 – 492 คำสั่ง Python สำหรับ Normalize ภาพจากตัวแปร frame และ magnitude spectrum

คำสั่ง Python การทำ normalize ให้ค่าข้อมูลของพิกเซลต่ำสุดเป็น 0 และค่าสูงสุดเป็น 1 เพื่อที่จะนำไปแปลงเป็นข้อมูล Gray scale ชนิด 16-bit ของตัวแปร frame, magnitude spectrum สำหรับการแสดงผลภาพ original และ magnitude spectrum

5. บรรทัดที่ 495 – 499 คำสั่ง Python สำหรับสร้างขอบเขตของการใช้ filters เพื่อประมวลผลภาพ

ในคำสั่งนี้จะเป็นการสร้างขอบเขตของ filters สำหรับนำมาประมวลผลในภาพที่อยู่ใน frequency domain และดึงค่า camera_settings[“cut_off_frequency”] จากตัวแปร global

5. บรรทัดที่ 501 - 521 คำสั่ง Python เลือกใช้ filters (GLPF or GHPF)

>> คำสั่งเลือกใช้งาน Gaussian low pass filter (GLPF) จะทำงานโดยเช็คเงื่อนไขของตัวแปร camera_settings[“filter_mode”] ถ้ารับค่าเป็น “Low Pass (GLPF)” จะเป็นการสั่งใช้งาน GLPF โดยในคำสั่งจะนำตัวแปร f_shift ที่เป็นภาพที่อยู่ใน frequency domain มาคำนวณกับฟังก์ชัน GLPF ที่กำหนดไว้ จากนั้นเป็นการทำ inverse FFT เพื่อแปลงภาพกลับไปยัง spatial domain และเก็บค่าไว้ในตัวแปร image_vis สำหรับนำไปแสดงผลต่อ

>> คำสั่งเลือกใช้งาน Gaussian high pass filter (GHPF) จะทำงานโดยเช็คเงื่อนไขของตัวแปร camera_settings[“filter_mode”] ถ้ารับค่าเป็น “High Pass (GHPF)” จะเป็นการสั่งใช้งาน GHPF โดยในคำสั่งจะนำตัวแปร f_shift ที่เป็นภาพที่อยู่ใน frequency domain มาคำนวณกับฟังก์ชัน GHPF ที่กำหนดไว้ จากนั้นเป็นการทำ inverse FFT เพื่อแปลงภาพกลับไปยัง spatial domain และเก็บค่าไว้ในตัวแปร image_vis สำหรับนำไปแสดงผลต่อ

6. บรรทัดที่ 523 – 528 คำสั่ง Python สำหรับเก็บขอ้มูล array ของภาพไว้ในตัวแปร global

คำสั่งนี้จะนำข้อมูล array ของ original , magnitude spectrum, filtered image ที่ยังไม่ผ่านการ normalize ไปเก็บไว้ในตัวแปร global เพื่อสามารถนำไปใช้ต่อในฟังก์ชันอื่นต่อได้

7. บรรทัดที่ 530 – 537 คำสั่ง Python สำหรับ normalize ภาพที่ฟิลเตอร์แล้ว และปรับขนาดหน้าต่างแสดงผล

ในบรรทัดที่ 532 คือคำสั่ง normalize ภาพที่แปลงกลับมาอยู่ใน spatial domain ซึ่งเก็บอยู่ในตัวแปร image_vis และปรับขนาดของหน้าต่างแสดงผลในบรรทัดที่ 535 – 536

8. บรรทัดที่ 539 – 549 คำสั่ง Python สำหรับแสดงผลภาพ และ stop acquisition

คำสั่งนี้จะแสดงผลภาพ original, magnitude spectrum, filtered image ที่ normalize ด้วย open cv และ stop acquisition หลังจากปิดหน้าต่างแสดงผล

ฟังก์ชันคำสั่ง Python สำหรับถ่ายภาพ Snap (Filters) ในโหมด Bg Corrected ด้วยกล้อง sCMOS

```
        #Take image with FFT & Filters (Bg Corrected)
        elif camera_settings["data_type"] == "Bg Corrected" and camera_settings["acquisition"] in ["Single scan", "Accumulate"]:
            #Check cut off input
            try:
                cutoff = float(cutoff_entry.get())
                camera_settings["cut_off_frequency"] = cutoff
            except ValueError:
                messagebox.showerror("ValueError", "Cut-off frequency must be a number (float).")
                return
            if background_image is not None:
                #Gaussian low-pass ffilter
                def gaussian_lowpass(size, cutoff):
                    x = np.linspace(-1, 1, cols)
                    y = np.linspace(-1, 1, rows)
                    xv, yv = np.meshgrid(x, y)
                    d = np.sqrt(xv**2 + yv**2)

                    return np.exp(-(d**2) / (2 * (cutoff**2)))

                #Gaussian high-pass filter
                def gaussian_highpass(size, cutoff):
                    return 1 - gaussian_lowpass(size, cutoff)

                #Start Acquisition
                cam.setup_acquisition(mode = 'sequence', nframes = 100) #Default
                cam.start_acquisition()
                print('Acquisition started')

                #Started Acquisition
                cam.wait_for_frame(since = 'lastread', nframes = 1 , timeout = 20 , error_on_stopped = False)
                frame = np.float32(cam.read_newest_image())

                ##Background Corrected
                signal = frame - background_image

                #FFT 2D Processing
                f_transform = np.fft.fft2(signal)
                f_shift = np.fft.fftshift(f_transform)
                magnitude_spectrum = np.abs(f_shift)
                magnitude_spectrum = np.log1p(magnitude_spectrum)

                #Normalize for output
                vis2 = cv2.normalize(signal, None, 0, 65535, cv2.NORM_MINMAX).astype(np.uint16)
                magnitude_vis = cv2.normalize(magnitude_spectrum, None, 0, 65535, cv2.NORM_MINMAX).astype(np.uint16)

                #Create a filter mask
                rows, cols = f_shift.shape
                crow ,ccol = rows // 2, cols // 2
                cutoff = camera_settings["cut_off_frequency"]
                gaussian_lp_filter = gaussian_lowpass((rows, cols), cutoff = cutoff )
                gaussian_hp_filter = gaussian_highpass((rows, cols), cutoff = cutoff)
```

```
602.
603.                    #Apply the filters (check condition of GLPF, GHPF)
604.                    if camera_settings["filter_mode"] == "Low Pass (GLPF)":
605.                        f_filtered_lp = f_shift * gaussian_lp_filter # Apply Gaussain low-
pass filter
606.                        #Inverse FFT
607.                        f_ishift_lp = np.fft.ifftshift(f_filtered_lp)
608.                        img_back_lp = np.fft.ifft2(f_ishift_lp)
609.                        img_back_lp = np.real(img_back_lp)
610.                        img_back_lp = img_back_lp.astype(int)
611.
612.                        image_vis = img_back_lp.copy()
613.
614.                    elif camera_settings["filter_mode"] == "High Pass (GHPF)":
615.                        f_filtered_hp = f_shift * gaussian_hp_filter  #Apply Gaussian
High-pass filter
616.                        #Inverse FFT
617.                        f_ishift_hp = np.fft.ifftshift(f_filtered_hp)
618.                        img_back_hp = np.fft.ifft2(f_ishift_hp)
619.                        img_back_hp = np.real(img_back_hp)
620.                        img_back_hp = img_back_hp.astype(int)
621.
622.                        image_vis = img_back_hp.copy()
623.
624.                    #Save for later use
625.                    signal_bg_image = frame.copy()
626.                    signal_image = signal.copy()
627.                    original_image = signal.copy()
628.                    magnitude_image = magnitude_spectrum.copy()
629.                    filtered_image = image_vis.copy()
630.
631.                    #Normalize  filtered  for display
632.                    display_size = (512,512)
633.                    image_fil_vis = cv2.normalize(image_vis, None, 0, 65535,
cv2.NORM_MINMAX).astype(np.uint16)
634.
635.                    #Resize
636.                    resize = cv2.resize(vis2, display_size, interpolation =
cv2.INTER_NEAREST)
637.                    resize_mag = cv2.resize(magnitude_vis, display_size, interpolation =
cv2.INTER_NEAREST)
638.                    resize_vis = cv2.resize(image_fil_vis, display_size, interpolation =
cv2.INTER_NEAREST)
639.
640.                    #Display
641.                    cv2.imshow("Live Acquisition", resize)
642.                    cv2.imshow("Live Magnitudde Spectrum", resize_mag)
643.                    cv2.imshow("Live Acquisition Filtered", resize_vis)
644.
645.                    #Stop Acquisition
646.                    cam.stop_acquisition()
647.                    print('Acquisition stopped')
648.                    cv2.waitKey(0)
649.                    cv2.destroyAllWindows()
650.                    return
651.                else:
652.                    messagebox.showerror("Acqusition Error", "Background image not found")
653.                    return
```

ฟังก์ชัน snap_image โหมด Bg Corrected และ Filters คำสั่ง Python จะทำงานดังนี้

1. บรรทัดที่ 552 – 560 เงื่อนไขตรวจสอบการเปิดใช้งาน FFT Mode จากตัวแปร global

เงื่อนไขตรวจสอบการเบิดใช้การ FFT Mode และตรวจสอบเงื่อนไขในตัวแปร camera_settings โดย data_type และ acquisition ต้องเป็น “Bg Corrected” , “Single scan” หรือ “Accumulate” จากนั้น ในบรรทัดที่ 452 – 457 คือคำสั่งตรวจสอบการรับค่า float ของ cut off frequency

ในบรรทัดที่ 560 จะมีการตรวจสอบเงื่อนไขการถ่ายภาพ background หากยังไมได้ถ่ายจะแจ้งเตือน Acquisition Error (อยู่ในบรรทัดที่ 652)

2. บรรทัดที่ 562 – 572 กำหนดฟังก์ชัน Gaussian low-pass filter และ Gaussian high-pass filter

ในบรรทัดที่ 562 – 568 คือฟังก์ชัน Gaussian low pass filter เพื่อเรียกไปใช้สำหรับการประมวลผลภาพ

ในบรรทัดที่ 571 – 572 คือฟังก์ชัน Gaussian high pass filter เพื่อเรียกไปใช้สำหรับการประมวลผลภาพ

3. บรรทัดที่ 580 - 581 การ Set up acquisition และการถ่ายภาพด้วยกล้อง sCMOS

ในบรรทัดที่ 473 – 474 เป็นคำสั่งสำหรับ set up acquisition และเริ่ม acquisiton โดยกล้องจะเก็บข้อมูลภาพตาม setup_acquisition ที่ตั้งค่าไว้ ในบรรทัดที่ 478 เป็นคำสั่งให้กล้อง คงสถานะการเก็บภาพ wait_for_frame โดยตั้งค่าไว้ที่ 20 วินาทีเพื่อไม่ให้เกิด Time out ในระหว่างการถ่ายภาพ และในบรรทัดที่ 479 เป็นคำสั่งที่กล้องจะอ่านภาพล่าสุดจาก buffer

4. บรรทัดที่ 584 คำสั่ง Python สำหรับการทำ background corrected

ในบรรทัดที่ 584 จะเป็นการนำเอาข้อมูล array ของ image มาลบกับข้อมูลภาพ background และเก็บค่าไว้ในตัวแปร signal

5. บรรทัดที่ 587 – 590 คำสั่ง Python การประมวลผลภาพด้วย FFT

ในบรรทัดที่ 587 เป็นการนำตัวแปร signal เป็นข้อมูล array ของภาพถ่ายมาทำ FFT โดยจะแปลงค่าจาก spatial domain ให้ไปอยู่ใน frequency domain เพื่อประมวลผลภาพที่อยู่ใน frequency domain ต่อไปในตัวแปรอื่นๆ

ในบรรทัดที่ 588 คือการ shift ความค่าความถี่ใน frequency domain ให้มาอยู่บริเวณตรงกลางของภาพ

ในบรรทัดที่ 589 - 590 การคำนวน Magnitude spectrum ของการทำ FFT หรือเป็นภาพที่อยู่ใน frequency domain

6. บรรทัดที่ 593 – 594 คำสั่ง Python สำหรับ Normalize ภาพจากตัวแปร signal และ magnitude spectrum

คำสั่ง Python การทำ normalize ให้ค่าข้อมูลของพิกเซลต่ำสุดเป็น 0 และค่าสูงสุดเป็น 1 เพื่อที่จะนำไปแปลงเป็นข้อมูล Gray scale ชนิด 16-bit ของตัวแปร signal, magnitude spectrum สำหรับการแสดงผลภาพ original และ magnitude spectrum

7. บรรทัดที่ 597 – 601 คำสั่ง Python สำหรับสร้างขอบเขตของการใช้ filters เพื่อประมวลผลภาพ

ในคำสั่งนี้จะเป็นการสร้างขอบเขตของ filters สำหรับนำมาประมวลผลในภาพที่อยู่ใน frequency domain และดึงค่า camera_settings[“cut_off_frequency”] จากตัวแปร global

8. บรรทัดที่ 604 - 620 คำสั่ง Python เลือกใช้ filters (GLPF or GHPF)

>> คำสั่งเลือกใช้งาน Gaussian low pass filter (GLPF) จะทำงานโดยเช็คเงื่อนไขของตัวแปร camera_settings[“filter_mode”] ถ้ารับค่าเป็น “Low Pass (GLPF)” จะเป็นการสั่งใช้งาน GLPF โดยในคำสั่งจะนำตัวแปร f_shift ที่เป็นภาพที่อยู่ใน frequency domain มาคำนวณกับฟังก์ชัน GLPF ที่กำหนดไว้ จากนั้นเป็นการทำ inverse FFT เพื่อแปลงภาพกลับไปยัง spatial domain และเก็บค่าไว้ในตัวแปร image_vis สำหรับนำไปแสดงผลต่อ

>> คำสั่งเลือกใช้งาน Gaussian high pass filter (GHPF) จะทำงานโดยเช็คเงื่อนไขของตัวแปร camera_settings[“filter_mode”] ถ้ารับค่าเป็น “High Pass (GHPF)” จะเป็นการสั่งใช้งาน GHPF โดยในคำสั่งจะนำตัวแปร f_shift ที่เป็นภาพที่อยู่ใน frequency domain มาคำนวณกับฟังก์ชัน GHPF ที่กำหนดไว้ จากนั้นเป็นการทำ inverse FFT เพื่อแปลงภาพกลับไปยัง spatial domain และเก็บค่าไว้ในตัวแปร image_vis สำหรับนำไปแสดงผลต่อ

9. บรรทัดที่ 523 – 528 คำสั่ง Python สำหรับเก็บขอ้มูล array ของภาพไว้ในตัวแปร global

คำสั่งนี้จะนำข้อมูล array ของ original , magnitude spectrum, filtered image ที่ยังไม่ผ่านการ normalize ไปเก็บไว้ในตัวแปร global เพื่อสามารถนำไปใช้ต่อในฟังก์ชันอื่นต่อได้

10. บรรทัดที่ 530 – 537 คำสั่ง Python สำหรับ normalize ภาพที่ฟิลเตอร์แล้ว และปรับขนาดหน้าต่างแสดงผล

ในบรรทัดที่ 532 คือคำสั่ง normalize ภาพที่แปลงกลับมาอยู่ใน spatial domain ซึ่งเก็บอยู่ในตัวแปร image_vis และปรับขนาดของหน้าต่างแสดงผลในบรรทัดที่ 535 – 536

11. บรรทัดที่ 539 – 549 คำสั่ง Python สำหรับแสดงผลภาพ และ stop acquisition

คำสั่งนี้จะแสดงผลภาพ original, magnitude spectrum, filtered image ที่ normalize ด้วย open cv และ stop acquisition หลังจากปิดหน้าต่างแสดงผล

ฟังก์ชันคำสั่ง Python สำหรับถ่ายภาพ Accumulate Single Background (No Filter) ด้วยกล้อง sCMOS

```
 655.     #Accumulate Single Background (No Filter)
 656.     if camera_settings["fft_mode"] == "OFF":
 657.         if camera_settings["data_type"] == "Bg Corrected" and
camera_settings["acquisition"] == "Accumulate (Single BG)":
 658.             if background_image is not None:
 659.                 n_frame = camera_settings["number_of_accumulate"]
 660.
 661.                 #Start Acquisition
 662.                 cam.setup_acquisition(mode = 'sequence', nframes = 100) #Default
 663.                 cam.start_acquisition()
 664.                 print("Aqcuisition started")
 665.
 666.                 accumulate_image = None
 667.                 accumulate_image_bg = None
 668.
 669.                 for i in range(n_frame):
 670.                     cam.wait_for_frame(since = 'lastread', nframes = 1, timeout = 100,
error_on_stopped = False)
 671.                     frame = np.float32(cam.read_newest_image())
 672.
 673.                     ##Background Corrected
 674.                     signal = frame - background_image
 675.
 676.                     #Initial accumulate arrays
 677.                     if accumulate_image is None:
 678.                         accumulate_image_bg = np.zeros_like(frame)
 679.                         accumulate_image = np.zeros_like(signal)
 680.
 681.                     accumulate_image_bg += frame
 682.                     accumulate_image += signal
 683.
 684.                 #Image After Accumulated
 685.                 acc_image_bg = accumulate_image_bg.copy()
 686.                 acc_image = accumulate_image.copy()
 687.
```

```
688.                 cam.stop_acquisition()
689.                 print("Acquisition stopped")
690.
691.                 #Normalize for output
692.                 display_size = (512, 512)
693.                 accumulated_vis = cv2.normalize(acc_image, None, 0, 65535,
cv2.NORM_MINMAX).astype(np.uint16)
694.
695.                 #Re-size image
696.                 resize = cv2.resize(accumulated_vis, display_size, interpolation =
cv2.INTER_NEAREST) #LINEAR, NEAREST
697.
698.                 #Save for later use
699.                 signal_bg_image = acc_image_bg.copy()
700.                 signal_image = acc_image.copy()
701.
702.                 #Display
703.                 cv2.imshow("Acquisition (Single)",resize)
704.                 cv2.waitKey(0)
705.                 cv2.destroyAllWindows()
706.                 return
707.
708.             else:
709.                 messagebox.showerror("Acquisition Error", "Background image not
found")
```

ฟังก์ชัน snap_image โหมด Accumulate single background และ No Filter คำสั่ง Python จะทำงานดังนี้

1. บรรทัดที่ 656 – 659 เงื่อนไขตรวจสอบการถ่ายภาพโหมด Accumulate single background

บรรทัดที่ 656 – 658 เป็นคำสั่งตรวจสอบเงื่อนไขของตัวแปร FFT Mode และ data type โดยในโหมดนี้ data type และ acquisition ต้องถูกเลือกเป็น “Bg Corrected” และ “Accumulate (Single BG)” เพื่อทำงานต่อในฟังก์ชันนี้ บรรทัดที่ 658 เป็นคำสั่งตรวจสอบภาพ background หากยังไม่ยังไม่ถ่ายจะแจ้งเตือน Acquisition Error

2. บรรทัดที่ 659 – 667 คำสั่ง Python สำหรับ Set up acquisition ก่อนเริ่มการถ่ายภาพ

บรรทัดที่ 659 เป็นคำสั่งรับค่าจำนวนครั้งของการ accumulate โดยจะรับค่าจากตัวแปร camera_settings จากนั้นเป็นการ set up acquisition และเริ่ม acquisition โดยกล้องจะเก็บข้อมูลภาพตาม setup_acquisition ในบรรทัดที่ 666 - 667 คำสั่งกำหนดตัวแปรสำหรับเก็บข้อมูล array ของการถ่ายภาพแบบ accumulate

3. บรรทัดที่ 669 – 689 คำสั่ง Python การถ่ายภาพแบบ Accumulate single background

คำสั่งนี้ทำงานโดยใช้ for loop เก็บข้อมูลภาพตามจำนวนครั้งของการ accumulate ในตัวแปร n_frame ในบรรทัดที่ 670 -671 เป็นคำสั่งให้กล้อง คงสถานะการเก็บภาพ wait_for_frame และเก็บอ่านภาพล่าสุดจาก buffer จากนั้นจะเป็นการทำ Background Corrected โดยจะนำตัวแปร frame ที่มีข้อมูลภาพที่ถ่ายมาลบกับ array ของภาพ background

ในบรรทัดที่ 667 – 682 เป็นคำสั่งการ stack ภาพโดยจะสร้าง array ที่มีค่าเป็น 0 และนาดเท่ากับตัวแปร frame, signal เพื่อทำหน้าที่เป็นแปรที่จะนำภาพมา stack กันไว้ ซึ่งบรรทัดที่ 681 – 682 จะเป็นการนำข้อมูล array มาบวกกันไว้ใน array ที่ต้องการ stack ภาพไว้ เมื่อครบ loop แล้วจะใช้คำสั่ง stop acquisition หยุดการถ่ายภาพ

4. บรรทัดที่ 691 – 706 คำสั่ง Python สำหรับ normalize และแสดงผลภาพ

คำสั่งตั้งแต่บรรทัดที่ 691 ถึง 706 จะทำงานดังนี้ บรรทัดที่ 693 คือคำสั่ง Python การทำ normalize ให้ค่าข้อมูลของพิกเซลต่ำสุดเป็น 0 และค่าสูงสุดเป็น 1 เพื่อที่จะนำไปแปลงเป็นข้อมูล Gray scale ชนิด 16-bit สำหรับการแสดงผลภาพที่ถ่าย ในบรรทัดที่ 696 เป็นคำสั่งปรับขนาดของหน้าต่างแสดงผลภาพ โดยสามารถปรับขนาดได้จากตัวแปร display_size จากนั้นบรรทัดที่ 699 – 700 คำสั่งให้เก็บข้อมูล array ของภาพที่ accumulate ไว้ในตัวแปร global เพื่อที่จะสามารถเรียกไปใช้ใน function อื่นๆ และบรรทัดที่ 703 – 705 คือคำสั่งแสดงผลภาพด้วย open cv

ฟังก์ชันคำสั่ง Python สำหรับถ่ายภาพ Accumulate Single Background (Filters) ด้วยกล้อง sCMOS

```
    #Accumulate Single Background (Filter)
    elif camera_settings["fft_mode"] == "ON":
        if camera_settings["data_type"] == "Bg Corrected" and camera_settings["acquisition"] == "Accumulate (Single BG)":
            #Check cut off input
            try:
                cutoff = float(cutoff_entry.get())
                camera_settings["cut_off_frequency"] = cutoff
            except ValueError:
                messagebox.showerror("ValueError", "Cut-off frequency must be a number (float).")
                return

            if background_image is not None:
                #Gaussian low-pass ffilter
                def gaussian_lowpass(size, cutoff):
                    x = np.linspace(-1, 1, cols)
                    y = np.linspace(-1, 1, rows)
                    xv, yv = np.meshgrid(x, y)
                    d = np.sqrt(xv**2 + yv**2)

                    return np.exp(-(d**2) / (2 * (cutoff**2)))

                #Gaussian high-pass filter
                def gaussian_highpass(size, cutoff):
                    return 1 - gaussian_lowpass(size, cutoff)
                n_frame = camera_settings["number_of_accumulate"]

                #Start Acquisition
                cam.setup_acquisition(mode = 'sequence', nframes = 100) #Default
                cam.start_acquisition()
                print("Aqcuisition started")

                accumulate_image = None
                accumulate_image_bg = None

                for i in range(n_frame):
                    cam.wait_for_frame(since = 'lastread', nframes = 1, timeout = 200, error_on_stopped = False)
                    frame = np.float32(cam.read_newest_image())

                    ##Background Corrected
                    signal = frame - background_image

                    #Initial accumulate arrays
                    if accumulate_image is None:
                        accumulate_image_bg = np.zeros_like(frame)
                        accumulate_image = np.zeros_like(signal)

                    accumulate_image_bg += frame
                    accumulate_image += signal

                #Image After Accumulated
                acc_image_bg = accumulate_image_bg.copy()
```

```
762.                    acc_image = accumulate_image.copy()
763.
764.                    #FFT 2D Processing
765.                    f_transform = np.fft.fft2(acc_image)
766.                    f_shift = np.fft.fftshift(f_transform)
767.                    magnitude_spectrum = np.abs(f_shift)
768.                    magnitude_spectrum = np.log1p(magnitude_spectrum)
769.
770.                    #Normalize for output
771.                    vis2 = cv2.normalize(acc_image, None, 0, 65535,
cv2.NORM_MINMAX).astype(np.uint16)
772.                    magnitude_vis = cv2.normalize(magnitude_spectrum, None, 0, 65535,
cv2.NORM_MINMAX).astype(np.uint16)
773.
774.                    #Create a filter mask
775.                    rows, cols = f_shift.shape
776.                    crow ,ccol = rows // 2, cols // 2
777.                    cutoff = camera_settings["cut_off_frequency"]
778.                    gaussian_lp_filter = gaussian_lowpass((rows, cols), cutoff = cutoff )
779.                    gaussian_hp_filter = gaussian_highpass((rows, cols), cutoff = cutoff)
780.
781.                    #Apply the filters (check condition of GLPF, GHPF)
782.                    if camera_settings["filter_mode"] == "Low Pass (GLPF)":
783.                        f_filtered_lp = f_shift * gaussian_lp_filter # Apply Gaussain low-
pass filter
784.                        #Inverse FFT
785.                        f_ishift_lp = np.fft.ifftshift(f_filtered_lp)
786.                        img_back_lp = np.fft.ifft2(f_ishift_lp)
787.                        img_back_lp = np.real(img_back_lp)
788.                        img_back_lp = img_back_lp.astype(int)
789.
790.                        image_vis = img_back_lp.copy()
791.
792.                    elif camera_settings["filter_mode"] == "High Pass (GHPF)":
793.                        f_filtered_hp = f_shift * gaussian_hp_filter  #Apply Gaussian
High-pass filter
794.                        #Inverse FFT
795.                        f_ishift_hp = np.fft.ifftshift(f_filtered_hp)
796.                        img_back_hp = np.fft.ifft2(f_ishift_hp)
797.                        img_back_hp = np.real(img_back_hp)
798.                        img_back_hp = img_back_hp.astype(int)
799.
800.                        image_vis = img_back_hp.copy()
801.
802.                    #Save for later use
803.                    signal_bg_image = acc_image_bg.copy()
804.                    original_image = acc_image.copy()
805.                    magnitude_image = magnitude_spectrum.copy()
806.                    filtered_image = image_vis.copy()
807.
808.                    #Normalize  filtered  for display
809.                    display_size = (512,512)
810.                    image_fil_vis = cv2.normalize(image_vis, None, 0, 65535,
cv2.NORM_MINMAX).astype(np.uint16)
811.
812.                    #Resize
813.                    resize = cv2.resize(vis2, display_size, interpolation =
cv2.INTER_NEAREST)
814.                    resize_mag = cv2.resize(magnitude_vis, display_size, interpolation =
cv2.INTER_NEAREST)
```

```
815.                 resize_vis = cv2.resize(image_fil_vis, display_size, interpolation =
cv2.INTER_NEAREST)
816.
817.                 #Display
818.                 cv2.imshow("Live Acquisition", resize)
819.                 cv2.imshow("Live Magnitudde Spectrum", resize_mag)
820.                 cv2.imshow("Live Acquisition Filtered", resize_vis)
821.
822.                 #Stop Acquisition
823.                 cam.stop_acquisition()
824.                 print('Acquisition stopped')
825.                 cv2.waitKey(0)
826.                 cv2.destroyAllWindows()
827.                 return
828.         else:
829.             messagebox.showerror("Acquisition Error", "Background image not found")
```

ฟังก์ชัน snap_image โหมด Accumulate single background และ Filters คำสั่ง Python จะทำงานดังนี้

1. บรรทัดที่ 712 – 720 เงื่อนไขตรวจสอบการถ่ายภาพโหมด Accumulate single background

คำสั่งตรวจสอบเงื่อนไขของตัวแปร FFT Mode และ data type โดยในโหมดนี้ data type และ acquisition ต้องถูกเลือกเป็น “Bg Corrected” และ “Accumulate (Single BG)” เพื่อทำงานต่อในฟังก์ชันนี้ ในบรรทัดที่ 715 – 720 คือคำสั่งตรวจสอบชนิดค่า float ของ cut off frequency ที่รับค่าเข้ามา

2. บรรทัดที่ 722 – 762 คำสั่ง Python สำหรับการถ่ายภาพ Accumulate single background

บรรทัดที่ 724 – 730 คือฟังก์ชัน Gaussian low pass filter เพื่อเรียกไปใช้สำหรับการประมวลผลภาพ

บรรทัดที่ 733 – 734 คือฟังก์ชัน Gaussian high pass filter เพื่อเรียกไปใช้สำหรับการประมวลผลภาพ

การถ่ายภาพแบบ accumulate single BG จะทำงานเหมือนกับคำสั่งการถ่ายแบบไม่ใช้ FFT Mode แต่จะต่างกันที่การ stop acquisition จะไปหยุดเมื่อประมวลผลภาพเสร็จแล้ว

3. บรรทัดที่ 765 – 768 คำสั่ง Python การประมวลผลภาพด้วย FFT

ในบรรทัดที่ 765 เป็นการนำตัวแปร acc_image เป็นข้อมูล array ของภาพถ่ายมาทำ FFT โดยจะแปลงค่าจาก spatial domain ให้ไปอยู่ใน frequency domain เพื่อประมวลผลภาพที่อยู่ใน frequency domain ต่อไปในตัวแปรอื่นๆ

ในบรรทัดที่ 766 คือการ shift ความค่าความถี่ใน frequency domain ให้มาอยู่บริเวณตรงกลางของภาพ

ในบรรทัดที่ 768 - 769 การคำนวน Magnitude spectrum ของการทำ FFT หรือเป็นภาพที่อยู่ใน frequency domain

4. บรรทัดที่ 771 – 772 คำสั่ง Python สำหรับ Normalize ภาพจากตัวแปร signal และ magnitude spectrum

คำสั่ง Python การทำ normalize ให้ค่าข้อมูลของพิกเซลต่ำสุดเป็น 0 และค่าสูงสุดเป็น 1 เพื่อที่จะนำไปแปลงเป็นข้อมูล Gray scale ชนิด 16-bit ของตัวแปร acc_image, magnitude spectrum สำหรับการแสดงผลภาพ original และ magnitude spectrum

5. บรรทัดที่ 774 – 779 คำสั่ง Python สำหรับสร้างขอบเขตของการใช้ filters เพื่อประมวลผลภาพ

ในคำสั่งนี้จะเป็นการสร้างขอบเขตของ filters สำหรับนำมาประมวลผลในภาพที่อยู่ใน frequency domain และดึงค่า camera_settings[“cut_off_frequency”] จากตัวแปร global

8. บรรทัดที่ 781 - 800 คำสั่ง Python เลือกใช้ filters (GLPF or GHPF)

>> คำสั่งเลือกใช้งาน Gaussian low pass filter (GLPF) จะทำงานโดยเช็คเงื่อนไขของตัวแปร camera_settings[“filter_mode”] ถ้ารับค่าเป็น “Low Pass (GLPF)” จะเป็นการสั่งใช้งาน GLPF โดยในคำสั่งจะนำตัวแปร f_shift ที่เป็นภาพที่อยู่ใน frequency domain มาคำนวณกับฟังก์ชัน GLPF ที่กำหนดไว้ จากนั้นเป็นการทำ inverse FFT เพื่อแปลงภาพกลับไปยัง spatial domain และเก็บค่าไว้ในตัวแปร image_vis สำหรับนำไปแสดงผลต่อ

>> คำสั่งเลือกใช้งาน Gaussian high pass filter (GHPF) จะทำงานโดยเช็คเงื่อนไขของตัวแปร camera_settings[“filter_mode”] ถ้ารับค่าเป็น “High Pass (GHPF)” จะเป็นการสั่งใช้งาน GHPF โดยในคำสั่งจะนำตัวแปร f_shift ที่เป็นภาพที่อยู่ใน frequency domain มาคำนวณกับฟังก์ชัน GHPF ที่กำหนดไว้ จากนั้นเป็นการทำ inverse FFT เพื่อแปลงภาพกลับไปยัง spatial domain และเก็บค่าไว้ในตัวแปร image_vis สำหรับนำไปแสดงผลต่อ

9. บรรทัดที่ 803 – 806 คำสั่ง Python สำหรับเก็บขอ้มูล array ของภาพไว้ในตัวแปร global

คำสั่งนี้จะนำข้อมูล array ของ original , magnitude spectrum, filtered image ที่ยังไม่ผ่านการ normalize ไปเก็บไว้ในตัวแปร global เพื่อสามารถนำไปใช้ต่อในฟังก์ชันอื่นต่อได้

10. บรรทัดที่ 808 – 815 คำสั่ง Python สำหรับ normalize ภาพที่ฟิลเตอร์แล้ว และปรับขนาดหน้าต่างแสดงผล

ในบรรทัดที่ 810 คือคำสั่ง normalize ภาพที่แปลงกลับมาอยู่ใน spatial domain ซึ่งเก็บอยู่ในตัวแปร image_vis และปรับขนาดของหน้าต่างแสดงผลในบรรทัดที่ 813 - 815

11. บรรทัดที่ 818 – 827 คำสั่ง Python สำหรับแสดงผลภาพ และ stop acquisition

คำสั่งนี้จะแสดงผลภาพ original, magnitude spectrum, filtered image ที่ normalize ด้วย open cv และ stop acquisition หลังจากปิดหน้าต่างแสดงผล

ฟังก์ชันคำสั่ง Python สำหรับถ่ายภาพ Live (No Filters) ในโหมด Count ด้วยกล้อง sCMOS

```
##Take Acquisition (Live)
def live_acquisition():
    global cam, signal_bg_image, background_image , signal_image, original_image, magnitude_image, filtered_image
    camera_settings["data_type"] = data_type_combo.get()
    camera_settings["acquisition"] =acquisition_combo.get()
    camera_settings["fft_mode"] = fft_combo.get()
    camera_settings["filter_mode"] = filter_mode_combo.get()

    if cam is None:
        messagebox.showerror("AcquisitionError", "Camera not connected")
        return

    if camera_settings["fft_mode"] == "OFF":
        if camera_settings["data_type"] == "Count" and camera_settings["acquisition"] == "Single scan":
            #Start acquisition
            cam.setup_acquisition(mode = 'sequence', nframes = 100)
            cam.start_acquisition()
            print('Live acquisition started')

            frame_count = 0
            while True:
                cam.wait_for_frame(since = 'lastread', nframes = 1 , timeout = 200 , error_on_stopped = False)
                frame = np.float32(cam.read_newest_image())

                #Normalize for output
                vis2 = cv2.normalize(frame, None, 0, 65535, cv2.NORM_MINMAX).astype(np.uint16)

                #Re-size image
                display_size = (512, 512)
                resized_img = cv2.resize(vis2, display_size , interpolation = cv2.INTER_NEAREST) #LINEAR, NEAREST

                #Save for later use
                signal_bg_image = frame.copy()

                #Display
                frame_count += 1
                cv2.imshow("Live Acquisition",resized_img )
                if cv2.getWindowProperty("Live Acquisition", cv2.WND_PROP_VISIBLE) < 1 or cv2.waitKey(1) & 0xFF == ord("q"):
                    break

            cam.stop_acquisition()
            print('Live acquisition stopped')
            cv2.destroyAllWindows()
            return

        elif camera_settings["data_type"] == "Bg Corrected" and camera_settings["acquisition"] == "Single scan":
            if background_image is not None:
                #Start acquisition
                cam.setup_acquisition(mode = 'sequence', nframes = 100)
```

```
880.                 cam.start_acquisition()
881.                 print('Live acquisition started')
882.
883.                 frame_count = 0
884.                 while True:
885.                     cam.wait_for_frame(since = 'lastread', nframes = 1 , timeout = 200
, error_on_stopped = False)
886.                     frame = np.float32(cam.read_newest_image())
887.
888.                     ##Background Corrected
889.                     signal = frame - background_image
890.
891.                     #Normalize for output
892.                     vis2 = cv2.normalize(frame, None, 0, 65535,
cv2.NORM_MINMAX).astype(np.uint16)
893.                     #Re-size image
894.                     display_size = (512, 512)
895.                     resized_img = cv2.resize(vis2, display_size , interpolation =
cv2.INTER_NEAREST) #LINEAR, NEAREST
896.
897.                     #Save for later use(optional)
898.                     signal_image = signal.copy()
899.
900.                     #Display
901.                     frame_count += 1
902.                     cv2.imshow("Live Acquisition", resized_img )
903.                     if cv2.getWindowProperty("Live Acquisition", cv2.WND_PROP_VISIBLE)
< 1 or cv2.waitKey(1) & 0xFF == ord("q"):
904.                         break
905.
906.                 cam.stop_acquisition()
907.                 print('Live acquisition stopped')
908.                 cv2.destroyAllWindows()
909.                 return
910.
911.             elif background_image is None:
912.                 messagebox.showerror("Acquisition Error", "Background image not
found")
```

ฟังก์ชัน live_acquisition โหมด Count และ No filter คำสั่ง Python จะทำงานดังนี้

1. บรรทัดที่ 832 – 841 เงื่อนไขตรวจสอบการเชื่อมต่อกับกล้อง sCMOS และเรียกตัวแปร global

เรียกตัวแปร global ในบรรทัดที่ 833 ที่เกี่ยวข้อง และกำหนดให้ตัวแปร camera_settings รับค่าจาก Combo box และ Entry box จากนั้นในบรรทัดที่ 839 – 841 คือคำสั่งตรวจสอบการเชื่อมต่อกับกล้อง sCMOS หากไม่ได้เชื่อมต่อจะแจ้งเตือน AcquisitionError

2. บรรทัดที่ 843 – 847 Set up Acquisition ของการถ่ายภาพ

บรรทัดที่ 843 จะเป็นเงื่อนไขตรวจสอบ Filter Mode โดยจะให้ตัวแปร camera_settings[“fft_mode”] เป็น OFF และเช็คเงื่อนไขของ Acquisition mode โดย data_type จะต้องเท่ากับ “Count” และ acquisition จะต้องเป็น “Single scan” หรือ “Accumulate” จากนั้นเป็นการตั้งค่า acquisition โหมดสามารถเป็นได้ทั้ง

snap หรือ sequence (การเก็บข้อมูลอย่างต่อเนื่อง) nframes ใช้กำหนดจำนวนเฟรมที่จะเก็บใน single mode หรือขนาดของ buffer ในโหมด sequence (default คือ 100)

3. บรรทัดที่ 851 – 874 การถ่ายภาพ Live ด้วยกล้อง sCMOS

เมื่อ Set up acquisition แล้วจะเป็นคำสั่งให้เริ่ม acquisition โดยกล้องจะเก็บข้อมูลภาพตาม setup_acquisition ที่ตั้งค่าไว้ ในบรรทัดที่ 852 เป็นคำสั่งให้กล้อง คงสถานะการเก็บภาพ wait_for_frame โดยตั้งค่าไว้ที่ 20 วินาทีเพื่อไม่ให้เกิด Time out ในระหว่างการถ่ายภาพ และในบรรทัดที่ 853 เป็นคำสั่งที่กล้องจะอ่านภาพล่าสุดจาก buffer ซึ่งการถ่ายภาพแบบ Live ต้องทำงานใน while loop เพื่อนำภาพมาแสดงผลอย่างต่อเนื่อง

4. บรรทัดที่ 855 – 860 คำสั่ง Python การทำ Normalize และปรับขนาดหน้าต่างสำหรับแสดงผล

คำสั่ง Python การทำ normalize ให้ค่าข้อมูลของพิกเซลต่ำสุดเป็น 0 และค่าสูงสุดเป็น 1 เพื่อที่จะนำไปแปลงเป็นข้อมูล Gray scale ชนิด 16-bit ในบรรทัดที่ 856 สำหรับการแสดงผลภาพที่ถ่าย

ในบรรทัดที่ 860 เป็นคำสั่งปรับขนาดของหน้าต่างแสดงผลภาพ โดยสามารถปรับขนาดได้จากตัวแปร display_size

5. บรรทัดที่ 866 - 874 คำสั่ง Python สำหรับ stop acquisition ของการถ่ายภาพ

บรรทัดที่ 866 คือคำสั่งที่เก็บจำนวนภาพที่ถ่ายทั้งหมด และแสดงผลภาพด้วย open cv ในบรรทัดที่ 867 บรรทัดที่ 868 – 869 คือคำสั่งการหยุดการทำงานของ while loop ถ้าต้องการหยุดการแสดงผลและหยุดการทำงานของ loop ให้กดปุ่ม “q” เมื่อหยุดการทำงานของ loop แล้วจะเป็นการ stop acquisition

ฟังก์ชันคำสั่ง Python สำหรับถ่ายภาพ Live (No Filters) ในโหมด Bg Corrected ด้วยกล้อง sCMOS

```
875.         #Take Acquisition Bg Corrected(Live)
876.         elif camera_settings["data_type"] == "Bg Corrected" and
camera_settings["acquisition"] == "Single scan":
877.             if background_image is not None:
878.                 #Start acquisition
879.                 cam.setup_acquisition(mode = 'sequence', nframes = 100)
880.                 cam.start_acquisition()
881.                 print('Live acquisition started')
882.
883.                 frame_count = 0
884.                 while True:
885.                     cam.wait_for_frame(since = 'lastread', nframes = 1 , timeout = 200
, error_on_stopped = False)
886.                     frame = np.float32(cam.read_newest_image())
887.
888.                     ##Background Corrected
889.                     signal = frame - background_image
890.
891.                     #Normalize for output
892.                     vis2 = cv2.normalize(frame, None, 0, 65535,
cv2.NORM_MINMAX).astype(np.uint16)
893.                     #Re-size image
894.                     display_size = (512, 512)
895.                     resized_img = cv2.resize(vis2, display_size , interpolation =
cv2.INTER_NEAREST) #LINEAR, NEAREST
896.
897.                     #Save for later use (optional)
898.                     signal_image = signal.copy()
899.
900.                     #Display
901.                     frame_count += 1
902.                     cv2.imshow("Live Acquisition", resized_img )
903.                     if cv2.getWindowProperty("Live Acquisition", cv2.WND_PROP_VISIBLE)
< 1 or cv2.waitKey(1) & 0xFF == ord("q"):
904.                         break
905.
906.                 cam.stop_acquisition()
907.                 print('Live acquisition stopped')
908.                 cv2.destroyAllWindows()
909.                 return
910.
911.             elif background_image is None:
912.                 messagebox.showerror("Acquisition Error", "Background image not
found")
```

ฟังก์ชัน live_acquisition โหมด Bg Corrected และ No filter คำสั่ง Python จะทำงานดังนี้

1. บรรทัดที่ 876 เงื่อนไขตรวจสอบและเรียกตัวแปร global

บรรทัดที่ 876 ตรวจสอบเงื่อนไขเลือก data type เป็น Bg Corrected และ acquisition เป็น single scan เพื่อถ่ายภาพ Live ในโหมด BG Corrected

2. บรรทัดที่ 884 – 886 Set up Acquisition ของการถ่ายภาพ

คำสั่งตั้งค่า acquisition โหมดสามารถเป็นได้ทั้ง snap หรือ sequence (การเก็บข้อมูลอย่างต่อเนื่อง) nframes ใช้กำหนดจำนวนเฟรมที่จะเก็บใน single mode หรือขนาดของ buffer ในโหมด sequence (default คือ 100) เมื่อ Set up acquisition แล้วจะเป็นคำสั่งให้เริ่ม acquisition โดยกล้องจะเก็บข้อมูลภาพตาม setup_acquisition ที่ตั้งค่าไว้ ในบรรทัดที่ 885 เป็นคำสั่งให้กล้อง คงสถานะการเก็บภาพ wait_for_frame โดยตั้งค่าไว้ที่ 20 วินาทีเพื่อไม่ให้เกิด Time out ในระหว่างการถ่ายภาพ และในบรรทัดที่ 856 เป็นคำสั่งที่กล้องจะอ่านภาพล่าสุดจาก buffer ซึ่งการถ่ายภาพแบบ Live ต้องทำงานใน while loop เพื่อนำภาพมาแสดงผลอย่างต่อเนื่อง

3. บรรทัดที่ 889 คำสั่ง Python การทำ Background Corrected

คำสั่งการนำเอาข้อมูล array ของภาพที่ถ่ายมาลบกับข้อมูลภาพ background และเก็บค่าไว้ในตัวแปร signal

4. บรรทัดที่ 892 – 895 คำสั่ง Python การทำ Normalize และปรับขนาดหน้าต่างสำหรับแสดงผล

คำสั่ง Python การทำ normalize ให้ค่าข้อมูลของพิกเซลต่ำสุดเป็น 0 และค่าสูงสุดเป็น 1 เพื่อที่จะนำไปแปลงเป็นข้อมูล Gray scale ชนิด 16-bit ในบรรทัดที่ 892 สำหรับการแสดงผลภาพที่ถ่าย ในบรรทัดที่ 895 เป็นคำสั่งปรับขนาดของหน้าต่างแสดงผลภาพ โดยสามารถปรับขนาดได้จากตัวแปร display_size

5. บรรทัดที่ 900 - 909 คำสั่ง Python สำหรับ stop acquisition ของการถ่ายภาพ

บรรทัดที่ 901 คือคำสั่งที่เก็บจำนวนภาพที่ถ่ายทั้งหมด และแสดงผลภาพด้วย open cv ในบรรทัดที่ 902 บรรทัดที่ 902 – 904 คือคำสั่งการหยุดการทำงานของ while loop ถ้าต้องการหยุดการแสดงผลและหยุดการทำงานของ loop ให้กดปุ่ม “q” เมื่อหยุดการทำงานของ loop แล้วจะเป็นการ stop acquisition

ฟังก์ชันคำสั่ง Python สำหรับถ่ายภาพ Live (Filters) ในโหมด Count ด้วยกล้อง sCMOS

```
    #Take Acquisition Count FFT & Filters (Live)
    if camera_settings["fft_mode"] == "ON":
        if camera_settings["data_type"] == "Count" and camera_settings["acquisition"] == "Single scan":
            #Check cut off input
            try:
                cutoff = float(cutoff_entry.get())
                camera_settings["cut_off_frequency"] = cutoff
            except ValueError:
                messagebox.showerror("ValueError", "Cut-off frequency must be a number (float).")
                return

            #Gaussian low-pass ffilter
            def gaussian_lowpass(size, cutoff):
                x = np.linspace(-1, 1, cols)
                y = np.linspace(-1, 1, rows)
                xv, yv = np.meshgrid(x, y)
                d = np.sqrt(xv**2 + yv**2)

                return np.exp(-(d**2) / (2 * (cutoff**2)))

            #Gaussian high-pass filter
            def gaussian_highpass(size, cutoff):
                return 1 - gaussian_lowpass(size, cutoff)

            #Start acquisition
            cam.setup_acquisition(mode = 'sequence', nframes = 100)
            cam.start_acquisition()
            print('Live acquisition started')

            frame_count = 0
            while True:
                #Read newest image from the camera
                cam.wait_for_frame(since = 'lastread', nframes = 1 , timeout = 200 , error_on_stopped = False)
                frame = np.float32(cam.read_newest_image())

                #FFT 2D Processing
                f_transform = np.fft.fft2(frame)
                f_shift = np.fft.fftshift(f_transform)
                magnitude_spectrum = np.abs(f_shift)
                magnitude_spectrum = np.log1p(magnitude_spectrum)

                #Normalize for output
                vis2 = cv2.normalize(frame, None, 0, 65535, cv2.NORM_MINMAX).astype(np.uint16)
                magnitude_vis = cv2.normalize(magnitude_spectrum, None, 0, 65535, cv2.NORM_MINMAX).astype(np.uint16)

                #Create a filter mask
                rows, cols = f_shift.shape
                crow ,ccol = rows // 2, cols // 2
                cutoff = camera_settings["cut_off_frequency"]
                gaussian_lp_filter = gaussian_lowpass((rows, cols), cutoff = cutoff )
                gaussian_hp_filter = gaussian_highpass((rows, cols), cutoff = cutoff)
```

```
964.
965.                 #Apply the filters (check condition of GLPF, GHPF)
966.                 if camera_settings["filter_mode"] == "Low Pass (GLPF)":
967.                     f_filtered_lp = f_shift * gaussian_lp_filter # Apply Gaussain low-
pass filter
968.                     #Inverse FFT
969.                     f_ishift_lp = np.fft.ifftshift(f_filtered_lp)
970.                     img_back_lp = np.fft.ifft2(f_ishift_lp)
971.                     img_back_lp = np.real(img_back_lp)
972.                     img_back_lp = img_back_lp.astype(int)
973.
974.                     image_vis = img_back_lp.copy()
975.
976.                 elif camera_settings["filter_mode"] == "High Pass (GHPF)":
977.                     f_filtered_hp = f_shift * gaussian_hp_filter  #Apply Gaussian
High-pass filter
978.                     #Inverse FFT
979.                     f_ishift_hp = np.fft.ifftshift(f_filtered_hp)
980.                     img_back_hp = np.fft.ifft2(f_ishift_hp)
981.                     img_back_hp = np.real(img_back_hp)
982.                     #img_back_hp = img_back_hp.astype(int)
983.
984.                     image_vis = img_back_hp.copy()
985.
986.
987.                 #Normalize  filtered  for display
988.                 display_size = (512,512)
989.                 image_fil_vis = cv2.normalize(image_vis, None, 0, 65535,
cv2.NORM_MINMAX).astype(np.uint16)
990.
991.                 #Resize
992.                 resize = cv2.resize(vis2, display_size, interpolation =
cv2.INTER_NEAREST)
993.                 resize_mag = cv2.resize(magnitude_vis, display_size, interpolation =
cv2.INTER_NEAREST)
994.                 resize_vis = cv2.resize(image_fil_vis, display_size, interpolation =
cv2.INTER_NEAREST)
995.
996.                 # Display
997.                 cv2.imshow("Live Acquisition", resize)
998.                 cv2.imshow("Live Magnitudde Spectrum", resize_mag)
999.                 cv2.imshow("Live Acquisition Filtered", resize_vis)
1000.                frame_count += 1
1001.
1002.                # Check if any window is closed or 'q'key is pressed
1003.                if (
1004.                    cv2.getWindowProperty("Live Acquisition", cv2.WND_PROP_VISIBLE) <
1 or
1005.                    cv2.getWindowProperty("Live Magnitudde Spectrum",
cv2.WND_PROP_VISIBLE) < 1 or
1006.                    cv2.getWindowProperty("Live Acquisition Filtered",
cv2.WND_PROP_VISIBLE) < 1 or
1007.                    cv2.waitKey(1) & 0xFF == ord("q")
1008.                ):
1009.                    break
1010.
```

```
1011.            #Stop Acquisition
1012.            cam.stop_acquisition()
1013.            print('Acquisition stopped')
1014.            cv2.destroyAllWindows()
1015.            return
```

ฟังก์ชัน live_acquisition โหมด Count และ Filters คำสั่ง Python จะทำงานดังนี้

1. บรรทัดที่ 914 – 922 เงื่อนไขตรวจสอบการถ่ายภาพโหมด Accumulate single background

คำสั่งตรวจสอบเงื่อนไขของตัวแปร FFT Mode เป็น “ON” และ data type โดยในโหมดนี้ data type และ acquisition ต้องถูกเลือกเป็น “Count” และ “Single scan” เพื่อทำงานต่อในฟังก์ชันนี้ ในบรรทัดที่ 917 – 922 คือคำสั่งตรวจสอบชนิดค่า float ของ cut off frequency ที่รับค่าเข้ามา

2. บรรทัดที่ 924 – 946 คำสั่ง Python สำหรับการถ่ายภาพ Live (Filter)

บรรทัดที่ 925 – 931 คือฟังก์ชัน Gaussian low pass filter เพื่อเรียกไปใช้สำหรับการประมวลผลภาพ

บรรทัดที่ 934 – 935 คือฟังก์ชัน Gaussian high pass filter เพื่อเรียกไปใช้สำหรับการประมวลผลภาพ

การถ่ายภาพแบบ Live ในโหมด Count และ Filters จะทำงานเหมือนกับคำสั่งการถ่ายภาพ Live โหมด Count ไม่ใช้ FFT Mode แต่จะต่างกันที่การ stop acquisition จะไปหยุดเมื่อประมวลผลภาพเสร็จแล้ว

3. บรรทัดที่ 949 – 952 คำสั่ง Python การประมวลผลภาพด้วย FFT

ในบรรทัดที่ 949 เป็นการนำตัวแปร frame เป็นข้อมูล array ของภาพถ่ายมาทำ FFT โดยจะแปลงค่าจาก spatial domain ให้ไปอยู่ใน frequency domain เพื่อประมวลผลภาพที่อยู่ใน frequency domain ต่อไปในตัวแปรอื่นๆ

ในบรรทัดที่ 950 คือการ shift ความค่าความถี่ใน frequency domain ให้มาอยู่บริเวณตรงกลางของภาพ

ในบรรทัดที่ 951 - 952 การคำนวน Magnitude spectrum ของการทำ FFT หรือเป็นภาพที่อยู่ใน frequency domain

4. บรรทัดที่ 955 – 956 คำสั่ง Python สำหรับ Normalize ภาพจากตัวแปร signal และ magnitude spectrum

คำสั่ง Python การทำ normalize ให้ค่าข้อมูลของพิกเซลต่ำสุดเป็น 0 และค่าสูงสุดเป็น 1 เพื่อที่จะนำไปแปลงเป็นข้อมูล Gray scale ชนิด 16-bit ของตัวแปร frame, magnitude spectrum สำหรับการแสดงผลภาพ original และ magnitude spectrum

5. บรรทัดที่ 959 – 963 คำสั่ง Python สำหรับสร้างขอบเขตของการใช้ filters เพื่อประมวลผลภาพ

ในคำสั่งนี้จะเป็นการสร้างขอบเขตของ filters สำหรับนำมาประมวลผลในภาพที่อยู่ใน frequency domain และดึงค่า camera_settings[“cut_off_frequency”] จากตัวแปร global

6. บรรทัดที่ 966 - 984 คำสั่ง Python เลือกใช้ filters (GLPF or GHPF)

>> คำสั่งเลือกใช้งาน Gaussian low pass filter (GLPF) จะทำงานโดยเช็คเงื่อนไขของตัวแปร camera_settings[“filter_mode”] ถ้ารับค่าเป็น “Low Pass (GLPF)” จะเป็นการสั่งใช้งาน GLPF โดยในคำสั่งจะนำตัวแปร f_shift ที่เป็นภาพที่อยู่ใน frequency domain มาคำนวณกับฟังก์ชัน GLPF ที่กำหนดไว้ จากนั้นเป็นการทำ inverse FFT เพื่อแปลงภาพกลับไปยัง spatial domain และเก็บค่าไว้ในตัวแปร image_vis สำหรับนำไปแสดงผลต่อ

>> คำสั่งเลือกใช้งาน Gaussian high pass filter (GHPF) จะทำงานโดยเช็คเงื่อนไขของตัวแปร camera_settings[“filter_mode”] ถ้ารับค่าเป็น “High Pass (GHPF)” จะเป็นการสั่งใช้งาน GHPF โดยในคำสั่งจะนำตัวแปร f_shift ที่เป็นภาพที่อยู่ใน frequency domain มาคำนวณกับฟังก์ชัน GHPF ที่กำหนดไว้ จากนั้นเป็นการทำ inverse FFT เพื่อแปลงภาพกลับไปยัง spatial domain และเก็บค่าไว้ในตัวแปร image_vis สำหรับนำไปแสดงผลต่อ

7. บรรทัดที่ 988 – 994 คำสั่ง Python สำหรับ normalize ภาพที่ฟิลเตอร์แล้ว และปรับขนาดหน้าต่างแสดงผล

ในบรรทัดที่ 989 คือคำสั่ง normalize ภาพที่แปลงกลับมาอยู่ใน spatial domain ซึ่งเก็บอยู่ในตัวแปร image_vis และปรับขนาดของหน้าต่างแสดงผลในบรรทัดที่ 992 – 994

8. บรรทัดที่ 997 – 1015 คำสั่ง Python สำหรับแสดงผลภาพ และ stop acquisition

บรรทัดที่ 1000 คือคำสั่งที่เก็บจำนวนภาพที่ถ่ายทั้งหมด และแสดงผลภาพด้วย open cv

บรรทัดที่ 1003 – 1009 คือคำสั่งการหยุดการทำงานของ while loop ถ้าต้องการหยุดการแสดงผลและหยุดการทำงานของ loop ให้กดปุ่ม “q” เมื่อหยุดการทำงานของ loop แล้วจะเป็นการ stop acquisition

ฟังก์ชันคำสั่ง Python สำหรับถ่ายภาพ Live (Filters) ในโหมด Bg Corrected ด้วยกล้อง sCMOS

```
1016.         #Take Acquisition Bg Corrected FFT & Filters (Live)
1017.         elif camera_settings["data_type"] == "Bg Corrected" and
camera_settings["acquisition"] == "Single scan":
1018.             #Check cut off input
1019.             try:
1020.                 cutoff = float(cutoff_entry.get())
1021.                 camera_settings["cut_off_frequency"] = cutoff
1022.             except ValueError:
1023.                 messagebox.showerror("ValueError", "Cut-off frequency must be a number
(float).")
1024.                 return
1025.
1026.             if background_image is not None:
1027.                 #Gaussian low-pass ffilter
1028.                 def gaussian_lowpass(size, cutoff):
1029.                     x = np.linspace(-1, 1, cols)
1030.                     y = np.linspace(-1, 1, rows)
1031.                     xv, yv = np.meshgrid(x, y)
1032.                     d = np.sqrt(xv**2 + yv**2)
1033.
1034.                     return np.exp(-(d**2) / (2 * (cutoff**2)))
1035.
1036.                 #Gaussian high-pass filter
1037.                 def gaussian_highpass(size, cutoff):
1038.                     return 1 - gaussian_lowpass(size, cutoff)
1039.
1040.                 #Start acquisition
1041.                 cam.setup_acquisition(mode = 'sequence', nframes = 100)
1042.                 cam.start_acquisition()
1043.                 print('Live acquisition started')
1044.
1045.                 frame_count = 0
1046.                 while True:
1047.                     #Read newest image from the camera
1048.                     cam.wait_for_frame(since = 'lastread', nframes = 1 , timeout = 200
, error_on_stopped = False)
1049.                     frame = np.float32(cam.read_newest_image())
1050.
1051.                     ##Background Corrected
1052.                     signal = frame - background_image
1053.
1054.                     #FFT 2D Processing
1055.                     f_transform = np.fft.fft2(signal)
1056.                     f_shift = np.fft.fftshift(f_transform)
1057.                     magnitude_spectrum = np.abs(f_shift)
1058.                     magnitude_spectrum = np.log1p(magnitude_spectrum)
1059.
1060.                     #Normalize for output
1061.                     vis2 = cv2.normalize(signal, None, 0, 65535,
cv2.NORM_MINMAX).astype(np.uint16)
1062.                     magnitude_vis = cv2.normalize(magnitude_spectrum, None, 0, 65535,
cv2.NORM_MINMAX).astype(np.uint16)
1063.
```

```
1064.                    #Create a filter mask
1065.                    rows, cols = f_shift.shape
1066.                    crow ,ccol = rows // 2, cols // 2
1067.                    cutoff = camera_settings["cut_off_frequency"]
1068.                    gaussian_lp_filter = gaussian_lowpass((rows, cols), cutoff =
cutoff )
1069.                    gaussian_hp_filter = gaussian_highpass((rows, cols), cutoff =
cutoff)
1070.
1071.                    #Apply the filters (check condition of GLPF, GHPF)
1072.                    if camera_settings["filter_mode"] == "Low Pass (GLPF)":
1073.                        f_filtered_lp = f_shift * gaussian_lp_filter # Apply Gaussain
low-pass filter
1074.                        #Inverse FFT
1075.                        f_ishift_lp = np.fft.ifftshift(f_filtered_lp)
1076.                        img_back_lp = np.fft.ifft2(f_ishift_lp)
1077.                        img_back_lp = np.real(img_back_lp)
1078.                        img_back_lp = img_back_lp.astype(int)
1079.
1080.                        image_vis = img_back_lp.copy()
1081.
1082.                    elif camera_settings["filter_mode"] == "High Pass (GHPF)":
1083.                        f_filtered_hp = f_shift * gaussian_hp_filter  #Apply Gaussian
High-pass filter
1084.                        #Inverse FFT
1085.                        f_ishift_hp = np.fft.ifftshift(f_filtered_hp)
1086.                        img_back_hp = np.fft.ifft2(f_ishift_hp)
1087.                        img_back_hp = np.real(img_back_hp)
1088.                        img_back_hp = img_back_hp.astype(int)
1089.
1090.                        image_vis = img_back_hp.copy()
1091.
1092.                    #Normalize  filtered  for display
1093.                    display_size = (512,512)
1094.                    image_fil_vis = cv2.normalize(image_vis, None, 0, 65535,
cv2.NORM_MINMAX).astype(np.uint16)
1095.
1096.                    #Resize
1097.                    resize = cv2.resize(vis2, display_size, interpolation =
cv2.INTER_NEAREST)
1098.                    resize_mag = cv2.resize(magnitude_vis, display_size, interpolation
= cv2.INTER_NEAREST)
1099.                    resize_vis = cv2.resize(image_fil_vis, display_size, interpolation
= cv2.INTER_NEAREST)
1100.
1101.                    #Display
1102.                    cv2.imshow("Live Acquisition", resize)
1103.                    cv2.imshow("Live Magnitudde Spectrum", resize_mag)
1104.                    cv2.imshow("Live Acquisition Filtered", resize_vis)
1105.                    frame_count += 1
1106.
```

```
1107.                    # Check if any window is closed or 'q' key is pressed
1108.                    if (
1109.                        cv2.getWindowProperty("Live Acquisition",
cv2.WND_PROP_VISIBLE) < 1 or
1110.                        cv2.getWindowProperty("Live Magnitudde Spectrum",
cv2.WND_PROP_VISIBLE) < 1 or
1111.                        cv2.getWindowProperty("Live Acquisition Filtered",
cv2.WND_PROP_VISIBLE) < 1 or
1112.                        cv2.waitKey(1) & 0xFF == ord("q")
1113.                    ):
1114.                        break
1115.
1116.                #Stop Acquisition
1117.                cam.stop_acquisition()
1118.                print('Acquisition stopped')
1119.                cv2.destroyAllWindows()
1120.                return
1121.            else:
1122.                messagebox.showerror("Acquisition Error", "Background image not
found")
```

ฟังก์ชัน live_acquisition โหมด Bg Corrected และ Filters คำสั่ง Python จะทำงานดังนี้

1. บรรทัดที่ 876 เงื่อนไขตรวจสอบและเรียกตัวแปร global

คำสั่งตรวจสอบเงื่อนไขของตัวแปร FFT Mode เป็น “ON” และ data type โดยในโหมดนี้ data type และ acquisition ต้องถูกเลือกเป็น “Bg Corrected” และ “Single scan” เพื่อทำงานต่อในฟังก์ชันนี้ ในบรรทัดที่ 1019 – 1024 คือคำสั่งตรวจสอบชนิดค่า float ของ cut off frequency ที่รับค่าเข้ามา

2. บรรทัดที่ 1028 – 1049 คำสั่ง Python สำหรับการถ่ายภาพ Live (Filter)

บรรทัดที่ 1028 – 1034 คือฟังก์ชัน Gaussian low pass filter เพื่อเรียกไปใช้สำหรับการประมวลผลภาพ บรรทัดที่ 1037 – 1038 คือฟังก์ชัน Gaussian high pass filter เพื่อเรียกไปใช้สำหรับการประมวลผลภาพ การถ่ายภาพแบบ Live ในโหมด Count และ Filters จะทำงานเหมือนกับคำสั่งการถ่ายภาพ Live โหมด Count ไม่ใช้ FFT Mode แต่จะต่างกันที่การ stop acquisition จะไปหยุดเมื่อประมวลผลภาพเสร็จแล้ว

3. บรรทัดที่ 1052 คำสั่ง Python การทำ Background Corrected

คำสั่งการนำเอาข้อมูล array ของภาพที่ถ่ายมาลบกับข้อมูลภาพ background และเก็บค่าไว้ในตัวแปร signal

4. บรรทัดที่ 1055 – 1058 คำสั่ง Python การประมวลผลภาพด้วย FFT

ในบรรทัดที่ 1055 เป็นการนำตัวแปร signal เป็นข้อมูล array ของภาพถ่ายมาทำ FFT โดยจะแปลงค่าจาก spatial domain ให้ไปอยู่ใน frequency domain เพื่อประมวลผลภาพที่อยู่ใน frequency domain ต่อไปในตัวแปรอื่นๆ

ในบรรทัดที่ 1056 คือการ shift ความค่าความถี่ใน frequency domain ให้มาอยู่บริเวณตรงกลางของภาพ

ในบรรทัดที่ 1057 - 1058 การคำนวน Magnitude spectrum ของการทำ FFT หรือเป็นภาพที่อยู่ใน frequency domain

4. บรรทัดที่ 1061 – 1062 คำสั่ง Python สำหรับ Normalize ภาพจากตัวแปร signal และ magnitude spectrum

คำสั่ง Python การทำ normalize ให้ค่าข้อมูลของพิกเซลต่ำสุดเป็น 0 และค่าสูงสุดเป็น 1 เพื่อที่จะนำไปแปลงเป็นข้อมูล Gray scale ชนิด 16-bit ของตัวแปร signal, magnitude spectrum สำหรับการแสดงผลภาพ original และ magnitude spectrum

5. บรรทัดที่ 1065 – 1069 คำสั่ง Python สำหรับสร้างขอบเขตของการใช้ filters เพื่อประมวลผลภาพ

ในคำสั่งนี้จะเป็นการสร้างขอบเขตของ filter สำหรับนำมาประมวลผลในภาพที่อยู่ใน frequency domain และดึงค่า camera_settings[“cut_off_frequency”] จากตัวแปร global

6. บรรทัดที่ 1075 - 1090 คำสั่ง Python เลือกใช้ filters (GLPF or GHPF)

>> คำสั่งเลือกใช้งาน Gaussian low pass filter (GLPF) จะทำงานโดยเช็คเงื่อนไขของตัวแปร camera_settings[“filter_mode”] ถ้ารับค่าเป็น “Low Pass (GLPF)” จะเป็นการสั่งใช้งาน GLPF โดยในคำสั่งจะนำตัวแปร f_shift ที่เป็นภาพที่อยู่ใน frequency domain มาคำนวณกับฟังก์ชัน GLPF ที่กำหนดไว้ จากนั้นเป็นการทำ inverse FFT เพื่อแปลงภาพกลับไปยัง spatial domain และเก็บค่าไว้ในตัวแปร image_vis สำหรับนำไปแสดงผลต่อ

>> คำสั่งเลือกใช้งาน Gaussian high pass filter (GHPF) จะทำงานโดยเช็คเงื่อนไขของตัวแปร camera_settings[“filter_mode”] ถ้ารับค่าเป็น “High Pass (GHPF)” จะเป็นการสั่งใช้งาน GHPF โดยในคำสั่งจะนำตัวแปร f_shift ที่เป็นภาพที่อยู่ใน frequency domain มาคำนวณกับฟังก์ชัน GHPF ที่กำหนดไว้ จากนั้นเป็นการทำ inverse FFT เพื่อแปลงภาพกลับไปยัง spatial domain และเก็บค่าไว้ในตัวแปร image_vis สำหรับนำไปแสดงผลต่อ

7. บรรทัดที่ 1093 – 1099 คำสั่ง Python สำหรับ normalize ภาพที่ฟิลเตอร์แล้ว และปรับขนาดหน้าต่างแสดงผล

ในบรรทัดที่ 1094 คือคำสั่ง normalize ภาพที่แปลงกลับมาอยู่ใน spatial domain ซึ่งเก็บอยู่ในตัวแปร image_vis และปรับขนาดของหน้าต่างแสดงผลในบรรทัดที่ 1097 – 1099

8. บรรทัดที่ 997 – 1120 คำสั่ง Python สำหรับแสดงผลภาพ และ stop acquisition

บรรทัดที่ 1101 - 1105 คือคำสั่งที่เก็บจำนวนภาพที่ถ่ายทั้งหมด และแสดงผลภาพด้วย open cv

บรรทัดที่ 1108 – 1114 คือคำสั่งการหยุดการทำงานของ while loop ถ้าต้องการหยุดการแสดงผลและหยุดการทำงานของ loop ให้กดปุ่ม “q” เมื่อหยุดการทำงานของ loop แล้วจะเป็นการ stop acquisition

ฟังก์ชันคำสั่ง Python สำหรับบันทึกภาพ Background ที่ถ่ายโดยกล้อง sCMOS

```
1125. #Image Saving
1126. #Background saving
1127. def save_background():
1128.     global background_image
1129.     if background_image is None:
1130.         messagebox.showerror("SaveError", "Background image not found")
1131.         return
1132.
1133.
1134.     #Date and time for saving
1135.     current_datetime = datetime.now().strftime("%Y%m%d_%H%M%S")
1136.     default_filename = f"background_{current_datetime}.tiff"
1137.
1138.     #Ensure bbackground image is int for .txt save
1139.     background_save = background_image.astype(int)
1140.
1141.     #Ask user for save path
1142.     file_path = filedialog.asksaveasfilename(
1143.         defaultextension = ".tiff",
1144.         filetypes = [
1145.             ("TIFF files (.tiff)", "*.tiff"),
1146.             ("Bitmap files (.bmp)", "*.bmp"),
1147.             ("Text files (.txt)", "*.txt"),
1148.             ("All files", "*.*")
1149.         ],
1150.         title = "Save Background Image As",
1151.         initialfile = default_filename
1152.     )
1153.
```

```
1154.     if not file_path:
1155.         return
1156.
1157.     if file_path.endswith(".txt"):
1158.         try:
1159.             with open(file_path, "w") as f:
1160.                 #Write camera setting in first line
1161.                 setting_keys = [
1162.                     "acquisition" , "trigger_mode", "dynamic_range",
"number_of_accumulate",
1163.                     "exposure_time", "hbin", "vbin", "hstart", "hend", "vstart",
"vend", "data_type"
1164.                 ]
1165.                 setting_line = ", ".join([f"{key} = {camera_settings[key]}" for key in
setting_keys])
1166.                 f.write(setting_line + "\n")
1167.
1168.                 #Write image array
1169.                 for row in background_save:
1170.                     f.write(" ".join(map(str, row)) + "\n")
1171.
1172.         except Exception as e:
1173.             messagebox.showerror("Error", f"Failed to save text files {str(e)}")
1174.
1175.     elif file_path.endswith(".tiff") or file_path.endswith(".tif"):
1176.         normalize_image = cv2.normalize(background_image, None, 0, 65535,
cv2.NORM_MINMAX).astype(np.uint16)
1177.         Image.fromarray(normalize_image).save(file_path)
1178.
1179.     elif file_path.endswith(".bmp"):
1180.         normalize_image = cv2.normalize(background_image, None, 0, 255,
cv2.NORM_MINMAX).astype(np.uint8)
1181.         cv2.imwrite(file_path , normalize_image)
1182.
1183.     else:
1184.         messagebox.showerror("Error","Unsupported file type")
```

ฟังก์ชันคำสั่ง save_background สำหรับบันทึกภาพ Background

1. บรรทัดที่ 1127 – 1128 คำสั่ง Python สำหรับประกาศฟังก์ชันและตัวแปรที่ใช้

บรรทัดที่ 1127 คำสั่งเริ่มต้นฟังก์ชัน save_background สำหรับบันทึกภาพ background

บรรทัดที่ 1128 คำสั่งเรียกใช้ตัวแปร global background_image ซึ่งเป็นภาพ background ที่จะบันทึก

2. บรรทัดที่ 1129 – 1131 ตรวจสอบว่ามี background image

บรรทัดที่ 1129 – 1130 คำสั่งตรวจสอบว่าภาพ background ที่เก็บอยู่ใน background_image ว่างหรือไม่ ถ้าไม่มีภาพ ให้แสดง error message ผ่าน message box บรรทัดที่ 1131 จบฟังก์ชันทันทีด้วย return ถ้าไม่พบภาพ

3. บรรทัดที่ 1134 – 1139 สร้างชื่อไฟล์อัตโนมัติโดยใช้วันที่และเวลา

บรรทัดที่ 1135 คำสั่งสร้าง string ของวันที่และเวลาปัจจุบันในรูปแบบ YYYYMMDD_HHMMSS

บรรทัดที่ 1136 คำสั่งสร้างชื่อไฟล์เริ่มต้น เช่น background_20250710_235959.tiff

บรรทัดที่ 1139 แปลงภาพ background_image ให้เป็นชนิด int

4. บรรทัดที่ 1141 – 1155 เปิด Dialog ให้ผู้ใช้เลือกตำแหน่งและชื่อไฟล์สำหรับบันทึก

บรรทัดที่ 1142 – 1152 คำสั่งสำหรับใช้ filedialog.asksaveasfilename เปิดหน้าต่างเลือกไฟล์ กำหนดประเภทไฟล์ที่รองรับ ได้แก่ .tiff, .bmp, .txt และไฟล์ทั้งหมดตั้งชื่อไฟล์เริ่มต้นเป็นชื่อที่สร้างไว้ข้างต้น

บรรทัดที่ 1154 – 1155 ถ้าไม่ได้เลือกไฟล์ (กด cancel) ให้จบฟังก์ชันทันที

5.บรรทัดที่ 1157 – 1174 บันทึกเป็นไฟล์ .txt (บันทึกข้อมูลกล้องและค่า pixel)

บรรทัดที่ 1157 คำสั่งตรวจสอบว่าผู้ใช้เลือกไฟล์ .txt หรือไม่

บรรทัดที่ 1159 – 1170 คำสั่งเปิดไฟล์และเขียนค่าการตั้งค่ากล้องในบรรทัดแรก (เช่น acquisition, trigger mode ฯลฯ) และบันทึกข้อมูล array ของภาพ background และบรรทัดที่ 1172 – 1173 หากเกิด error ในการเขียนไฟล์ ให้แสดงข้อความ error

บรรทัดที่ 1175 – 1177 บันทึกเป็นไฟล์ .tiff

บรรทัดที่ 1175 คำสั่งตรวจสอบว่าผู้ใช้เลือกไฟล์ .tiff

บรรทัดที่ 1176 คำสั่ง normalize ค่าภาพให้อยู่ในช่วง 0–65535 แล้วแปลงเป็น uint16

บรรทัดที่ 1177 คำสั่งใช้ Image.fromarray() บันทึกเป็นไฟล์ .tiff ด้วย Pillow

ฟังก์ชันคำสั่ง Python สำหรับบันทึกภาพ Image (+BG) ที่ถ่ายโดยกล้อง sCMOS

```
1186. #Save image with background
1187. def save_image_bkg():
1188.     global signal_bg_image, background_image ,signal_image
1189.     if signal_bg_image is None:
1190.         messagebox.showerror("SaveError", "Image not found")
1191.         return
1192.
1193.     #Date and time for saving
1194.     current_datetime = datetime.now().strftime("%Y%m%d_%H%M%S")
1195.     default_filename = f"image_bkg_{current_datetime}.tiff"
1196.
1197.     #Ensure background image is int for .txt save
1198.     signal_bg_save = signal_bg_image.astype(int)
1199.
```

```
1200.     #Ask user for save path
1201.     file_path = filedialog.asksaveasfilename(
1202.         defaultextension = ".tiff",
1203.         filetypes = [
1204.             ("TIFF files (.tiff)", "*.tiff"),
1205.             ("Bitmap files (.bmp)", "*.bmp"),
1206.             ("Text files (.txt)", "*.txt"),
1207.             ("All files", "*.*")
1208.         ],
1209.         title = "Save Image As",
1210.         initialfile = default_filename
1211.     )
1212.
1213.     if not file_path:
1214.         return
1215.
1216.     if file_path.endswith(".txt"):
1217.         try:
1218.             with open(file_path, "w") as f:
1219.                 #Write camera settings in first line
1220.                 setting_keys = [
1221.                     "acquisition", "trigger_mode", "dynamic_range",
"number_of_accumulate",
1222.                     "exposure_time", "hbin", "vbin", "hstart", "hend", "vstart",
"vend", "data_type"
1223.                 ]
1224.                 setting_line = ", ".join([f"{key} = {camera_settings[key]}" for key in
setting_keys])
1225.                 f.write(setting_line + "\n")
1226.
1227.                 #Write image array
1228.                 for row in signal_bg_save:
1229.                     f.write(" ".join(map(str, row)) + "\n")
1230.
1231.         except Exception as e:
1232.             messagebox.showerror("Error", f"Failed to save text file {str(e)}")
1233.
1234.     elif file_path.endswith(".tiff") or file_path.endswith(".tif"):
1235.         normalize_image = cv2.normalize(signal_bg_image, None, 0, 65535,
cv2.NORM_MINMAX).astype(np.uint16)
1236.         Image.fromarray(normalize_image).save(file_path)
1237.
1238.     elif file_path.endswith(".bmp"):
1239.         normalize_image = cv2.normalize(signal_bg_image, None, 0, 255,
cv2.NORM_MINMAX).astype(np.uint8)
1240.         cv2.imwrite(file_path , normalize_image)
1241.
1242.     else:
1243.         messagebox.showerror("Error","Unsupported file type")
```

ฟังก์ชันคำสั่ง save_image_bkg สำหรับบันทึกภาพ Image (+BG)

1. บรรทัดที่ 1187 – 1188 คำสั่ง Python สำหรับประกาศฟังก์ชันและเรียกใช้งานตัวแปร global

บรรทัดที่ 1187 คำสั่งเรียกฟังก์ชัน save_image_bkg() เพื่อใช้บันทึกภาพที่มีปนอยู่ background

บรรทัดที่ 1188 คำสั่งเรียกใช้งานตัวแปร global ได้แก่ signal_bg_image, background_image และ signal_image

2. บรรทัดที่ 1189 – 1191 คำสั่ง Python สำหรับตรวจสอบว่ามีภาพที่จะบันทึกหรือไม่

บรรทัดที่ 1189 – 1191 คำสั่งตรวจสอบว่า signal_bg_image เป็น None หรือไม่ ถ้าใช่ จะแสดง error message แจ้งว่าไม่พบภาพ และจบฟังก์ชันทันที

3. บรรทัดที่ 1193 – 1195 คำสั่ง Python สำหรับสร้างชื่อไฟล์อัตโนมัติ

บรรทัดที่ 1193 – 1195 คำสั่งสร้างชื่อไฟล์อัตโนมัติด้วยวันที่และเวลา ณ ขณะนั้น โดยมีรูปแบบ image_bkg_YYYYMMDD_HHMMSS.tiff

4. บรรทัดที่ 1197 – 1198คำสั่ง Python สำหรับเตรียมข้อมูลภาพก่อนบันทึกไฟล์ .txt

บรรทัดที่ 1197 – 1198 คำสั่งแปลง signal_bg_image ให้เป็นชนิดข้อมูล int เพื่อให้เหมาะสมกับการบันทึกเป็น .txt

5. บรรทัดที่ 1200 – 1211 คำสั่ง Python สำหรับแสดง dialog ให้เลือกตำแหน่งและชื่อไฟล์ที่จะบันทึก

บรรทัดที่ 1200 – 1211 คำสั่งเปิด dialog ให้ผู้ใช้เลือกที่จัดเก็บไฟล์ พร้อมกำหนดนามสกุลไฟล์เริ่มต้นเป็น .tiff และมีตัวเลือกนามสกุล .tiff, .bmp, .txt

6. บรรทัดที่ 1213 – 1214 คำสั่ง Python สำหรับตรวจสอบกรณีผู้ใช้กด cancel

บรรทัดที่ 1213 – 1214 คำสั่งตรวจสอบเงื่อนไขหากผู้ใช้กด cancel โดยไม่เลือกไฟล์ จะจบฟังก์ชันทันที

7. บรรทัดที่ 1216 – 1233 คำสั่ง Python สำหรับบันทึกไฟล์ชนิด .txt

บรรทัดที่ 1216 – 1217 คำสั่งตรวจสอบว่าไฟล์ที่เลือกมีนามสกุล .txt หรือไม่

บรรทัดที่ 1218 – 1230 คำสั่งเปิดไฟล์เพื่อเขียนข้อมูล

บรรทัดที่ 1220 – 1225 คำสั่งเขียนค่าการตั้งค่ากล้องลงในบรรทัดแรก (เช่น acquisition, trigger mode ฯลฯ) และบันทึกข้อมูล array ของภาพ

บรรทัดที่ 1228 – 1229 คำสั่งบันทึกค่าพิกัดภาพในรูปแบบตัวเลขต่อบรรทัด

บรรทัดที่ 1231 – 1232 คำสั่งตรวจสอบหากเกิดข้อผิดพลาดระหว่างบันทึกไฟล์ จะแสดง error message

8. บรรทัดที่ 1234 – 1244 คำสั่ง Python สำหรับบันทึกไฟล์ชนิด .tiff

บรรทัดที่ 1234 – 1236 คำสั่งสำหรับปรับ normalize ค่าภาพให้อยู่ในช่วง 0 - 65535 และแปลงเป็น np.uint16บันทึกภาพเป็นไฟล์ .tiff ด้วย Image.fromarray

บรรทัดที่ 1238 – 1240 คำสั่งปรับ normalize ค่าภาพให้อยู่ในช่วง 0 - 255 และแปลงเป็น np.uint8 บันทึกภาพเป็นไฟล์ .bmp ด้วย cv2.imwrite

บรรทัดที่ 1242 – 1244 คำสั่งแสดง error message หากผู้ใช้เลือกไฟล์นามสกุลอื่นที่ไม่รองรับ

ฟังก์ชันคำสั่ง Python สำหรับบันทึกภาพ Image (-BG) ที่ถ่ายโดยกล้อง sCMOS

```
#Save image without background
def save_image():
    global signal_image , background_image , signal_bg_image
    if signal_image is None:
        messagebox.showerror("SaveError", "image not found")
        return

    #Date and time for saving
    current_datetime = datetime.now().strftime("%Y%m%d_%H%M%S")
    default_filename = f"image_{current_datetime}.tiff"

    #Ensure background image is int for .txt save
    signal_save = signal_image.astype(int)

    #Ask user for save path
    file_path = filedialog.asksaveasfilename(
        defaultextension = ".tiff",
        filetypes = [
            ("TIFF files (.tiff)", "*.tiff"),
            ("Bitmap files (.bmp)", "*.bmp"),
            ("Text files (.txt)", "*.txt"),
            ("All files", "*.*")
        ],
        title = "Save Image As",
        initialfile = default_filename
    )

    if not file_path:
        return

    if file_path.endswith(".txt"):
        try:
            with open(file_path, "w") as f:
                setting_keys = [
                    "acquisition", "trigger_mode", "dynamic_range", "number_of_accumulate",
                    "exposure_time", "hbin", "vbin", "hstart", "hend", "vstart", "vend", "data_type"
                ]
                setting_line = ", ".join([f"{key} = {camera_settings[key]}" for key in setting_keys])
                f.write(setting_line + "\n")

                #Write image array
                for row in signal_save:
                    f.write(" ".join(map(str, row)) + "\n")

        except Exception as e:
            messagebox.showerror("Error", f"Failed to save text files {str(e)}")

    elif file_path.endswith(".tiff") or file_path.endswith(".tif"):
        normalize_image = cv2.normalize(signal_image, None, 0, 65535, cv2.NORM_MINMAX).astype(np.uint16)
        Image.fromarray(normalize_image).save(file_path)

```

```
1298.     elif file_path.endswith(".bmp"):
1299.         normalize_image = cv2.normalize(signal_image, None, 0, 255,
cv2.NORM_MINMAX).astype(np.uint8)
1300.         cv2.imwrite(file_path , normalize_image)
1301.
1302.     else:
1303.         messagebox.showerror("Error","Unsupported file type")
1304.
```

ฟังก์ชันคำสั่ง save_image สำหรับบันทึกภาพ Image (-BG)

1. บรรทัดที่ 1248 – 1249 คำสั่ง Python สำหรับประกาศฟังก์ชันและกำหนดตัวแปร global

บรรทัดที่ 1248 คำสั่งเรียกฟังก์ชัน save_image เพื่อบันทึกภาพสัญญาณ ที่ยังไม่ลบ background

บรรทัดที่ 1249 คำสั่งประกาศตัวแปร global ได้แก่ signal_image, background_image และ signal_bg_image

2. บรรทัดที่ 1250 – 1256 คำสั่ง Python สำหรับตรวจสอบว่ามีภาพที่จะบันทึกหรือไม่

บรรทัดที่ 1250 – 1252 คำสั่งเงื่อนไขตรวจสอบว่า signal_image เป็น None หรือไม่ หากไม่พบภาพ จะแสดง error message และยกเลิกการทำงานของฟังก์ชันทันที ในบรรทัดที่ 1254 – 1256 สร้างชื่อไฟล์อัตโนมัติโดยมีวันที่และเวลาแนบไว้ (รูปแบบ image_YYYYMMDD_HHMMSS.tiff) เพื่อใช้เป็นชื่อไฟล์เริ่มต้นเวลาบันทึก

3. บรรทัดที่ 1258 – 1275 คำสั่ง Python สำหรับเตรียมข้อมูลภาพเพื่อบันทึกเป็น .txt

บรรทัดที่ 1258 – 1259 คำสั่งสำหรับแปลง signal_image ให้เป็นชนิด int

บรรทัดที่ 1261 – 1272 คำสั่งเปิดหน้าต่าง dialog ให้ผู้ใช้เลือกที่จัดเก็บไฟล์และกำหนดนามสกุลไฟล์ โดยรองรับ .tiff, .bmp, .txt

บรรทัดที่ 1274 – 1275 คำสั่งตรวจสอบหากผู้ใช้ยกเลิกการเลือกไฟล์ (ไม่ระบุ path) ฟังก์ชันจะยกเลิกการทำงานทันที

4. บรรทัดที่ 1277 – 1292 คำสั่ง Python สำหรับบันทึกภาพในรูปแบบ .txt

บรรทัดที่ 1277 – 1278 คำสั่งตรวจสอบว่านามสกุลไฟล์เป็น .txt หรือไม่

บรรทัดที่ 1279 – 1290 คำสั่งเปิดไฟล์เพื่อเขียนข้อมูลเขียนค่าการตั้งค่ากล้อง (เช่น exposure, binning ฯลฯ) ลงในบรรทัดแรก

บรรทัดที่ 1288 – 1289 เขียนค่าข้อมูล (image array) ลงไฟล์ทีละบรรทัด และในบรรทัดที่ 1291 – 1292 เป็นคำสั่งตรวจสอบหากเกิดข้อผิดพลาดขณะบันทึกไฟล์ จะแสดง error message

5. บรรทัดที่ 1294 – 1303 คำสั่ง Python สำหรับบันทึกภาพในรูปแบบ .bmp และ .tiff

บรรทัดที่ 1294 – 1296 คำสั่งสำหรับ normalize ค่าข้อมูลของภาพให้อยู่ในช่วง 0 - 65535 และแปลงเป็นชนิด np.uint16 บันทึกภาพเป็นไฟล์ .tiff โดยใช้ PIL.Image

บรรทัดที่ 1298 – 1300 คำสั่งสำหรับ normalize ค่าข้อมูลของภาพให้อยู่ในช่วง 0 - 255 และแปลงเป็นชนิด np.uint8 บันทึกภาพเป็นไฟล์ .bmp ด้วย cv2.imwrite

ฟังก์ชันคำสั่ง Python สำหรับบันทึกข้อมูลและภาพที่ถ่ายโดยกล้อง sCMOS หลังผ่านการประมวลผลด้วย FFT

```
#Image with FFT and filter saving
def save_fft_image():
    global original_image, magnitude_image, filtered_image

    if original_image is None or magnitude_image is None or filtered_image is None:
        messagebox.showerror("SaveError", "One or more required images not found")
        return

    #Date and time for image saving
    original_image = original_image.astype(int)
    current_datetime = datetime.now().strftime("%Y%m%d_%H%M%S")
    default_fft_filename = f"image_fft_{current_datetime}"

    #Ask user for save path
    file_path = filedialog.asksaveasfilename(
        defaultextension = ".tiff",
        filetypes = [
            ("TIFF files (.tiff)", "*.tiff"),
            ("Bitmap files (.bmp)", "*.bmp"),
            ("Text files (.txt)", "*.txt"),
            ("All files", "*.*")
        ],
        title = "Save Image As",
        initialfile = default_fft_filename
    )

    if not file_path:
        return

    folder = os.path.dirname(file_path)
    ext = os.path.splitext(file_path)[1]

    #Set up dictionary
    image_dict = {
        "original_image": original_image,
        "magnitude_spcetrum": magnitude_image,
        "filtered_image": filtered_image
    }
    try:
        for name, image in image_dict.items():
            filename = os.path.join(folder, f"{name}_{current_datetime}{ext}")

            if ext == ".txt":
                with open(filename, "w") as f:
                    #Write #Write camera setting in first line
                    setting_keys = [
                        "acquisition", "trigger_mode", "dynamic_range", "number_of_accumulate",
                        "exposure_time", "hbin", "vbin", "hstart", "hend", "vstart", "vend", "data_type"
                    ]
                    setting_line = ", ".join([f"{key} = {camera_settings[key]}" for key in setting_keys])
                    f.write(f"# {name}\n")
```

```
1356.                    f.write(setting_line + "\n")
1357.
1358.                    #Write image array
1359.                    for row in image:
1360.                        f.write(" ".join(map(str, row)) + "\n")
1361.
1362.            elif ext in [".tiff", ".tif"]:
1363.                normalize_image = cv2.normalize(image, None, 0, 65535,
cv2.NORM_MINMAX).astype(np.uint16)
1364.                Image.fromarray(normalize_image).save(filename)
1365.
1366.            elif ext == ".bmp":
1367.                normalized = cv2.normalize(image, None, 0, 255,
cv2.NORM_MINMAX).astype(np.uint8)
1368.                cv2.imwrite(filename, normalized)
1369.
1370.            else:
1371.                messagebox.showerror("Error", "Unsupported file extension selected")
1372.
1373.        messagebox.showinfo("Success", f"FFT images saved as {ext.upper()} in
folder:\n{folder}")
1374.
1375.    except Exception as e:
1376.        messagebox.showerror("Error", f"Failed to save files: {str(e)}")
```

```
1357.                    #Write image array
1358.                    for row in image:
1359.                        f.write(" ".join(map(str, row)) + "\n")
1360.
1361.            elif ext in [".tiff", ".tif"]:
1362.                normalize_image = cv2.normalize(image, None, 0, 65535,
cv2.NORM_MINMAX).astype(np.uint16)
1363.                Image.fromarray(normalize_image).save(filename)
1364.
1365.            elif ext == ".bmp":
1366.                normalized = cv2.normalize(image, None, 0, 255,
cv2.NORM_MINMAX).astype(np.uint8)
1367.                cv2.imwrite(filename, normalized)
1368.
1369.            else:
1370.                messagebox.showerror("Error", "Unsupported file extension selected")
1371.
1372.        messagebox.showinfo("Success", f"FFT images saved as {ext.upper()} in
folder:\n{folder}")
1373.
1374.    except Exception as e:
1375.        messagebox.showerror("Error", f"Failed to save files: {str(e)}")
```

ฟังก์ชันคำสั่ง save_fft_image สำหรับบันทึกภาพ original image, magnitude spectrum, filtered image

1. บรรทัดที่ 1306 – 1307 คำสั่ง Python สำหรับประกาศฟังก์ชันและกำหนดตัวแปร Global

บรรทัดที่ 1306 คือคำสั่งประกาศฟังก์ชัน save_fft_image สำหรับบันทึกภาพ FFT และภาพที่ผ่านการกรอง บรรทัดที่ 1307 คือคำสั่งประกาศตัวแปร global ได้แก่ original_image, magnitude_image, และ filtered_image เพื่อให้สามารถเรียกใช้งานได้ภายในฟังก์ชันนี้

2. บรรทัดที่ 1309 – 1311 คำสั่งตรวจสอบว่ามีภาพครบหรือไม่

บรรทัดที่ 1309 – 1311 คือคำสั่งตรวจสอบว่าภาพต้นฉบับ, magnitude spectrum และภาพ filtered มีข้อมูลหรือไม่ถ้าไม่มี จะขึ้น error message และหยุดการทำงานของฟังก์ชันทันที

3. บรรทัดที่ 1314 – 1316 คำสั่งเตรียมข้อมูลสำหรับการบันทึกไฟล์

บรรทัดที่ 1314 คือคำสั่งแปลง original_image ให้เป็นชนิดข้อมูล int เพื่อเตรียมสำหรับการบันทึก บรรทัดที่ 1315 คือคำสั่งสร้างวันที่และเวลาในรูปแบบ YYYYMMDD_HHMMSS เพื่อใช้เป็นส่วนหนึ่งของชื่อไฟล์ และในบรรทัดที่ 1316 คือคำสั่งกำหนดชื่อไฟล์เริ่มต้นโดยใช้วันที่และเวลาที่ได้

4. บรรทัดที่ 1319 – 1332 คำสั่งแสดงหน้าต่างให้ผู้ใช้เลือกตำแหน่งและรูปแบบไฟล์ที่จะบันทึก

บรรทัดที่ 1319 – 1329 คือคำสั่งเปิด dialog ให้ผู้ใช้เลือกตำแหน่งและชื่อไฟล์ที่ต้องการบันทึก รองรับไฟล์ .tiff, .bmp, .txt และในบรรทัดที่ 1331 – 1332 คือคำสั่งตรวจสอบว่าผู้ใช้ยกเลิกการเลือกไฟล์หรือไม่ ถ้าไม่มีไฟล์ที่เลือกฟังก์ชันจะหยุดทำงานทันที

5. บรรทัดที่ 1334 – 1342 คำสั่งเตรียมข้อมูล path สำหรับบันทึกไฟล์

บรรทัดที่ 1334 คือคำสั่งดึง path ของโฟลเดอร์ที่เลือกบันทึกบรรทัดที่ 1335 คือคำสั่งดึงนามสกุลไฟล์จาก path ที่เลือก และในบรรทัดที่ 1338 – 1342 คือคำสั่งรวมภาพทั้งหมด (original, magnitude spectrum, filtered) ไว้ใน dictionary image_dict เพื่อสะดวกต่อการบันทึกหลายไฟล์พร้อมกัน

6. บรรทัดที่ 1344 – 1373 คำสั่งบันทึกภาพตามประเภทไฟล์ที่เลือก

บรรทัดที่ 1344 – 1345 คำสั่งกำหนดชื่อไฟล์สำหรับภาพแต่ละชนิด สร้างชื่อไฟล์ใหม่สำหรับแต่ละภาพโดยเติมชื่อภาพและวันที่เวลาที่ชื่อไฟล์

บรรทัดที่ 1347 – 1360 คำสั่งบันทึกไฟล์เป็น .txt เปิดไฟล์ .txt สำหรับเขียน และบันทึกค่า setting ของกล้อง (เช่น acquisition mode, trigger, exposure time) จากนั้นบันทึกข้อมูล array ของภาพในแต่ละบรรทัด

บรรทัดที่ 1362 – 1364 คำสั่ง normalize ข้อมูล array ของภาพให้อยู่ในช่วง 0 – 65535 และแปลงเป็น uint16 ก่อนบันทึกไฟล์ภาพ TIFF ด้วย PIL

บรรทัดที่ 1365 – 1367 คำสั่งบันทึกไฟล์เป็น .bmp และคำสั่ง normalize ข้อมูล array ของภาพให้อยู่ในช่วง 0 – 255 และแปลงเป็น uint8 ก่อนบันทึกไฟล์ BMP ด้วย open cv

คำสั่ง Python ทั้งหมดของ Main GUI ในซอฟท์แวร์ AndorSDK3 Camera Control

คำสั่ง Python ทั้งหมดของ Main GUI

```
# Create the main window
root = tk.Tk()
root.title("AndorSDK3 Camera Control")
root.geometry("1400x700")

myMenu = tk.Menu()
root.config(menu = myMenu)

#Add Main Menu
myMenu.add_cascade(label = "File")
myMenu.add_cascade(label = "Edit")
myMenu.add_cascade(label = "About")
myMenu.add_cascade(label = "View")

root.grid_rowconfigure(1, weight=1)
root.grid_columnconfigure(0, weight=1)

# Toolbar under menu for icon buttons
toolbar = tk.Frame(root, bg="#F0F0F0", height = 120)
toolbar.grid(row=0, column=0, sticky="ew")

# Toolbar (Row)
toolbar_row1 = tk.Frame(toolbar, bg="#F0F0F0", height=20)
toolbar_row1.grid(row=0, column=0, sticky="ew")

# Take Background
take_bkg = tk.Button(toolbar_row1, text="Take Background", command=take_background, bg="white")
take_bkg.grid(row=0, column=0, padx=5, pady=2, sticky="w")
root.bind('<Control-b>', lambda event: take_background())  # Short cut key

# Take Acquisition (Snap)
snap_btn = tk.Button(toolbar_row1, text="Take Image", command=snap_image, bg="white")
snap_btn.grid(row=0, column=1, padx=0, pady=2, sticky="w")

# Take Aquisition (Live)
live_btn = tk.Button(toolbar_row1, text="Live Image", command=live_acquisition, bg="white")
live_btn.grid(row=0, column=2, padx=5, pady=2, sticky="w")

# Data type (Tool Bar)
data_tool_label = tk.Label(toolbar_row1, text="Data Type:", bg="#F0F0F0")
data_tool_label.grid(row=0, column=3, padx=5, pady=2, sticky="w")

# Data Type combo box
data_type_combo = ttk.Combobox(toolbar_row1, values=["Count", "Bg Corrected"], state="readonly", width=13)
data_type_combo.current(0)
data_type_combo.grid(row=0, column=4, padx=0, pady=2, sticky="w")
data_type_combo.bind("<<ComboboxSelected>>", on_data_type_change)

# Create the main frame
main_frame = tk.Frame(root, bg="#E0E0E0", padx=20, pady=20)
```

```
51. main_frame.grid(row=1, column=0, sticky="nsew")
52.
53. # Use grid layout for better positioning
54. main_frame.grid_rowconfigure(0, weight=1)
55. main_frame.grid_columnconfigure(0, weight=0)  # left panel
56. main_frame.grid_columnconfigure(1, weight=3)  # center panel
57. main_frame.grid_columnconfigure(2, weight=0)  # right panel
58.
59. # Create left panel (Left panel)
60. left_panel = tk.Frame(main_frame, bg="white", padx=10, pady=10, bd=2, relief=tk.GROOVE)
61. left_panel.grid(row=0, column=0, sticky="nsw", padx=10, pady=10)
62.
63. # Create center panel (Center panel)
64. center_panel = tk.Frame(main_frame, bg="white", padx=10, pady=10, bd=2,
relief=tk.GROOVE)
65. center_panel.grid(row=0, column=1, sticky="nsew", padx=10, pady=10)
66.
67. # Create camera settings panel (Right Panel)
68. right_panel = tk.Frame(main_frame, bg="white", padx=20, pady=20, bd=2,
relief=tk.GROOVE)
69. right_panel.grid(row=0, column=2, rowspan=1, sticky="nsew", padx=10, pady=10)
70.
71. # Create Status Bar
72. status_bar = tk.Frame(root, bg="#e0e0e0", relief=tk.SUNKEN, bd=1)
73. status_bar.grid(row=2, column=0, sticky="ew")
74.
75. camera_status_label = tk.Label(status_bar, text="Camera : Disconnected", bg="#e0e0e0",
fg="red", padx=10)
76. camera_status_label.grid(row=0, column=0, sticky="w")
77.
78. cooler_status_label = tk.Label(status_bar, text="Cooler : N/A", bg="#e0e0e0", padx=10)
79. cooler_status_label.grid(row=0, column=1, sticky="w")
80.
81. temp_status_label = tk.Label(status_bar, text="Temperature : N/A", bg="#e0e0e0",
padx=10)
82. temp_status_label.grid(row=0, column=2, sticky="w")
83.
84. data_type_status_label = tk.Label(status_bar, text="Data type : N/A", bg="#e0e0e0",
padx=10)
85. data_type_status_label.grid(row=0, column=3, sticky="w")
86. camera_settings["data_type"] = data_type_combo.get()
87. data_type_status_label.config(text=f"Data Type : {camera_settings['data_type']}")
88.
89. filter_mode_status_label = tk.Label(status_bar, text="Filter Mode : N/A", bg="#e0e0e0",
padx=10)
90. filter_mode_status_label.grid(row=0, column=4, sticky="w")
91.
92. ## Connect Camera Section
93. connect_label = tk.Label(left_panel, text="Connect Camera", font=("Arial", 10, "bold"),
bg="white")
94. connect_label.grid(row=0, column=0, sticky="w", pady=(0, 5))
95.
96. #Connect button & Disconnect button
97. dll_btn = tk.Button(left_panel, text="Import DLLS", command=set_dlls, width=18)
98. dll_btn.grid(row=1, column=0, pady=2)
99. con_btn = tk.Button(left_panel, text="Connect", command=connect_camera, width=18)
100. con_btn.grid(row=2, column=0, pady=2)
101. discon_btn = tk.Button(left_panel, text="Disconnect", command=disconnect_camera,
width=18)
102. discon_btn.grid(row=3, column=0, pady=2)
```

```
## Temperature Control Section
temp_label = tk.Label(left_panel, text="Temperature Control", font=("Arial", 10, "bold"), bg="white")
temp_label.grid(row=4, column=0, sticky="w", pady=(10, 5))

cooler_on_btn = tk.Button(left_panel, text="Cooler On", command=cooler_on, width=18)
cooler_on_btn.grid(row=5, column=0, pady=2)

cooler_off_btn = tk.Button(left_panel, text="Cooler Off", command=cooler_off, width=18)
cooler_off_btn.grid(row=6, column=0, pady=2)

set_temp_btn = tk.Button(left_panel, text="Set Temperature", command=set_temp, width=18)
set_temp_btn.grid(row=7, column=0, pady=2)

get_temp_btn = tk.Button(left_panel, text="Get Temperature", command=get_temp, width=18)
get_temp_btn.grid(row=8, column=0, pady=2)

#Image Saving Section (Left panel)
save_label = tk.Label(left_panel, text="Image Saving", font=("Arial", 10, "bold"), bg="white")
save_label.grid(row=9, column=0, sticky="w", pady=(20, 5))

#Save Image (+BG)
save_image_bkg_btn = tk.Button(left_panel, text="Save image (+BG)", command=save_image_bkg, width=18)
save_image_bkg_btn.grid(row=10, column=0, pady=2)

#Save Background
save_background_btn = tk.Button(left_panel, text="Save Background", command=save_background, width=18)
save_background_btn.grid(row=11, column=0, pady=2)

#Save Image (-BG)
save_image_btn = tk.Button(left_panel, text="Save image (-BG)", command=save_image, width=18)
save_image_btn.grid(row=12, column=0, pady=2)

#Save FFT data Label
save_fft_label = tk.Label(left_panel, text="Image Saving with filter", font=("Arial", 10, "bold"), bg="white")
save_fft_label.grid(row=13, column=0, sticky="w", pady=(20, 5))

#Save Image FFT button
save_fft_btn = tk.Button(left_panel, text="Save images (FFT)", command = save_fft_image, width=18)
save_fft_btn.grid(row=14, column=0, pady=2)

# Camera Setting Parameter Input (Right Panel Section)
camera_settings_frame = tk.LabelFrame(right_panel, text="Camera Settings", bg="white", font=("Arial", 10, "bold"))
camera_settings_frame.grid(row=0, column=0, sticky="ew", pady=5)
setting_inner_frame = tk.Frame(camera_settings_frame, bg="white")
setting_inner_frame.grid(row=0, column=0, sticky="w", pady=5)
```

```
150. # Combo box (Right panel combo box)
151. # Acquisition (Combo box)
152. acquisition_combo = ttk.Combobox(setting_inner_frame, values=["Single scan",
"Accumulate", "Accumulate (Single BG)"], state="readonly", width=18)
153. acquisition_combo.current(0)
154. create_input_row(setting_inner_frame, "Acquisition:", acquisition_combo, row=0)
155. acquisition_combo.bind("<<ComboboxSelected>>", on_acquisition_change)
156.
157. # Trigger Mode (Combo Box)
158. trigger_combo = ttk.Combobox(setting_inner_frame, values=["Internal", "External",
"Software", "External Exposure"], state="readonly", width=18)
159. trigger_combo.current(0)
160. create_input_row(setting_inner_frame, "Trigger Mode:", trigger_combo, row=1)
161.
162. # Dynamic Range (Combo Box)
163. dynamic_combo = ttk.Combobox(setting_inner_frame, values=["Low noise 12-bit", "HDR 16-
bit"], state="readonly", width=18)
164. dynamic_combo.current(0)
165. create_input_row(setting_inner_frame, "Dynamic Range:", dynamic_combo, row=2)
166.
167. # Entry Box (Right panel Entry box)
168. # Number of Accumulations
169. num_acc_entry = tk.Entry(setting_inner_frame, width=21)
170. num_acc_entry.insert(0, "1")
171. create_input_row(setting_inner_frame, "Number of Accumulate:", num_acc_entry, row=3)
172. num_acc_entry.config(state="disable")
173. acquisition_combo.bind("<<ComboboxSelected>>", on_acquisition_change)
174.
175. # Exposure Time
176. exposure_entry = tk.Entry(setting_inner_frame, width=21)
177. exposure_entry.insert(0, "0.02")
178. create_input_row(setting_inner_frame, "Exposure Time (sec):", exposure_entry, row=4)
179.
180. # Binning
181. hbin_entry = tk.Entry(setting_inner_frame, width=21)
182. hbin_entry.insert(0, "1")
183. create_input_row(setting_inner_frame, "hbin:", hbin_entry, row=5)
184.
185. vbin_entry = tk.Entry(setting_inner_frame, width=21)
186. vbin_entry.insert(0, "1")
187. create_input_row(setting_inner_frame, "vbin:", vbin_entry, row=6)
188.
189. # Region of interest
190. hstart_entry = tk.Entry(setting_inner_frame, width=21)
191. hstart_entry.insert(0, "0")
192. create_input_row(setting_inner_frame, "hstart:", hstart_entry, row=7)
193.
194. hend_entry = tk.Entry(setting_inner_frame, width=21)
195. hend_entry.insert(0, "2048")
196. create_input_row(setting_inner_frame, "hend:", hend_entry, row=8)
197.
198. vstart_entry = tk.Entry(setting_inner_frame, width=21)
199. vstart_entry.insert(0, "0")
200. create_input_row(setting_inner_frame, "vstart:", vstart_entry, row=9)
201.
202. vend_entry = tk.Entry(setting_inner_frame, width=21)
203. vend_entry.insert(0, "2048")
204. create_input_row(setting_inner_frame, "vend:", vend_entry, row=10)
205.
```

```
206. # Save Button
207. save_button = tk.Button(camera_settings_frame, text="Save Settings", command=save_data,
bg="#4CAF50", fg="white", width=12)
208. save_button.grid(row=1, column=0, pady=10, padx=10)
209.
210. # Image Processing Section
211. image_processing_frame = tk.LabelFrame(right_panel, bg="white", padx=5, pady=5,
text="Image Processing", font=("Arial", 10, "bold"), labelanchor="n")
212. image_processing_frame.grid(row=3, column=0, padx=5, pady=10, sticky="nsew")
213.
214. # Content frame for the image processing controls
215. content_frame = tk.Frame(image_processing_frame, bg="white", padx=5, pady=5)
216. content_frame.grid(row=0, column=0, sticky="nsew")
217.
218. # Configure the grid for content frame
219. image_processing_frame.grid_rowconfigure(0, weight=1) # Label column
220. image_processing_frame.grid_columnconfigure(0, weight=1) # Input widget column
221.
222. # FFT Mode Combo Box
223. fft_combo = ttk.Combobox(content_frame, values=["OFF", "ON"], state="readonly",
width=17)
224. fft_combo.current(0)
225. create_input_row(content_frame, "FFT Mode :", fft_combo, row=0)
226. fft_combo.bind("<<ComboboxSelected>>", on_fft_mode_change)
227.
228. # Filter Mode Combo Box
229. combo_select = tk.StringVar(value="Select")
230. filter_mode_combo = ttk.Combobox(content_frame, textvariable=combo_select, values=["Low
Pass (GLPF)", "High Pass (GHPF)"], state="readonly", width=17)
231. filter_mode_combo.config(state="disable")
232.
233. create_input_row(content_frame, "Filter Mode: ", filter_mode_combo, row=1)
234. filter_mode_combo.bind("<<ComboboxSelected>>", on_fft_mode_change)
235.
236. # Cutoff frequency Entry
237. cutoff_entry = tk.Entry(content_frame, width=20)
238. create_input_row(content_frame, "Cut off frequency:", cutoff_entry, row=2)
239. cutoff_entry.config(state="disable")
240. cutoff_entry.bind("<<ComboboxSelected>>", on_fft_mode_change)
241.
242. #Save setting in image processing
243. save_button = tk.Button(content_frame, text="Save values", command =
save_process_settings, bg="#4CAF50", fg="white", width=12)
244. save_button.grid(row=3, column=0, columnspan=2, pady=12)
245.
246. # Start the main loop
247. root.mainloop()
```

1. คำสั่ง python สร้างหน้าต่างหลักและเมนู

บรรทัดที่ 1 – 4 คำสั่ง Python สำหรับสร้างหน้าต่างหลักของโปรแกรม และกำหนดชื่อหน้าต่าง, ขนาดหน้าต่าง

บรรทัดที่ 6 – 7 คือคำสั่งกำหนดเมนูหลัก (Main Menu) ให้กับหน้าต่าง (สารมารถเพิ่มฟังก์ชันเข้าไปได้)

บรรทัดที่ 9 – 13 คือคำสั่งเพิ่มเมนูหลัก ได้แก่ File, Edit, About และ View (สารมารถเพิ่มฟังก์ชันเข้าไปได้)

2. คำสั่ง Python ตั้งค่า Layout หลัก

บรรทัดที่ 15 – 16 คือคำสั่งกำหนดการปรับขนาดอัตโนมัติเมื่อย่อ-ขยายหน้าต่าง

3. คำสั่ง Python สร้าง Toolbar สำหรับปุ่มฟังก์ชัน

บรรทัดที่ 18 – 20 คือคำสั่งสร้าง Toolbar ใต้เมนู

บรรทัดที่ 22 – 24 คือคำสั่งสร้าง Toolbar row1 สำหรับวางปุ่ม

บรรทัดที่ 26 – 29 คือปุ่ม Take Background สำหรับถ่ายภาพพื้นหลัง พร้อมปุ่มลัด Ctrl+B

บรรทัดที่ 31 – 33 คือปุ่ม Take Image สำหรับถ่ายภาพ

บรรทัดที่ 35 – 37 คือปุ่ม Live Image สำหรับแสดงภาพแบบ Live

บรรทัดที่ 39 – 41 คือ Label แสดงคำว่า Data Type

บรรทัดที่ 43 – 47 คือ Combobox เลือก Data Type และตั้ง Event เมื่อเลือก

4. คำสั่ง Python สร้าง Main Frame แบ่งพื้นที่โปรแกรม

บรรทัดที่ 49 – 51 คือคำสั่งสร้าง Main Frame ที่เป็นพื้นที่หลักของหน้าต่าง

บรรทัดที่ 53 – 57 คือคำสั่งแบ่ง Column layout ของ Main Frame ออกเป็น 3 ส่วน Left Panel, Center Panel, Right Panel

5. คำสั่ง Python สร้าง Panel แต่ละส่วน

บรรทัดที่ 59 – 61 คือคำสั่งสร้าง Left Panel สำหรับปุ่มควบคุมด้านซ้าย

บรรทัดที่ 63 – 65 คือคำสั่งสร้าง Center Panel สำหรับแสดงภาพ

บรรทัดที่ 67 – 69 คือคำสั่งสร้าง Right Panel สำหรับตั้งค่ากล้อง

6. คำสั่ง Python สร้าง Status Bar แสดงสถานะ

บรรทัดที่ 71 – 73 คือคำสั่งสร้าง Status Bar ที่ด้านล่างหน้าต่าง

บรรทัดที่ 75 – 90 คือคำสั่งแสดงสถานะต่าง ๆ เช่นควบคุมกล้อง, ระบบทำความเย็น, อุณหภูมิ

7. คำสั่ง Python กลุ่มคำสั่งเชื่อมต่อกล้อง (Left Panel)

บรรทัดที่ 92 – 94 คือหัวข้อ "Connect Camera"

บรรทัดที่ 97 – 102 คือปุ่มควบคุม Import DLLS, Connect, Disconnect

8. คำสั่ง Python กลุ่มคำสั่งควบคุมอุณหภูมิ (Left Panel)

บรรทัดที่ 104 – 106 คือหัวข้อ "Temperature Control"

บรรทัดที่ 108 – 118 คือปุ่มควบคุม เปิด/ปิดระบบทำความเย็น, ตั้งค่าอุณหภูมิ, ตรวจสอบอุณหภูมิ

9. คำสั่ง Python กลุ่มคำสั่งบันทึกภาพ (Left Panel)

บรรทัดที่ 120 – 122 คือหัวข้อ "Image Saving"

บรรทัดที่ 125 – 134 คือปุ่มบันทึกภาพประเภทต่าง ๆ บันทึกภาพที่มี Background ,บันทึก Background, บันทึกภาพที่ลบ Background แล้ว

บรรทัดที่ 136 – 138 คือหัวข้อ "Image Saving with filter"

บรรทัดที่ 141 – 142 คือปุ่มบันทึกภาพพร้อม FFT Filter

10. คำสั่ง Python กลุ่มตั้งค่ากล้อง (Right Panel)

บรรทัดที่ 144 – 148 คือการสร้าง Camera Settings Panel

บรรทัดที่ 151 – 155 คือ Combobox เลือก Acquisition Mode

บรรทัดที่ 157 – 160 คือ Combobox เลือก Trigger Mode

บรรทัดที่ 162 – 165 คือ Combobox เลือก Dynamic Range

บรรทัดที่ 167 – 187 คือกล่องกรอกข้อมูล เช่น Number of Accumulate ,Exposure Time, Binning

บรรทัดที่ 206 – 208 คือปุ่ม Save Settings เพื่อบันทึกค่ากล้อง

11. คำสั่ง Python กลุ่มตั้งค่า Image Processing (Right Panel)

บรรทัดที่ 210 – 212 คือการสร้าง Image Processing Panel

บรรทัดที่ 214 – 216 คือ Content Frame สำหรับวาง Element

บรรทัดที่ 219 – 220 คือคำสั่งกำหนด Grid Layout

บรรทัดที่ 222 – 226 คือ Combobox เลือก FFT Mode

บรรทัดที่ 228 – 234 คือ Combobox เลือก Filter Mode

บรรทัดที่ 236 – 239 คือกล่องกรอก Cutoff Frequency

บรรทัดที่ 242 – 244 คือปุ่ม Save values สำหรับบันทึกค่าการประมวลผล

12. คำสั่ง Python เริ่มต้นโปรแกรม

บรรทัดที่ 246 – 247 คือคำสั่งเริ่มต้น main loop ของโปรแกรม เพื่อให้ GUI ทำงาน

อ้างอิง

คู่มือต่างๆ และ Driver ที่เกี่ยวกับกล้อง Andor Sona sCMOS

[1] Andor Technology. (n.d.). *Downloads*. Oxford Instruments. https://andor.oxinst.com/downloads/

[2] Andor Technology. (n.d.). *Andor Driver Pack 3.15.30092.0 (sCMOS)* [Software]. Oxford Instruments. https://andor.oxinst.com/downloads/view/andor-driver-pack-3.15.30092.0-(scmos)

[3] Andor Technology. (n.d.). *Sona quick start guide* [PDF manual]. Oxford Instruments. https://andor.oxinst.com/downloads/uploads/Sona_Quick_Start_Guide.pdf

[4] Andor Technology. (n.d.). *Sona and Marana hardware guide B-6 models* [PDF manual]. Oxford Instruments.

https://andor.oxinst.com/downloads/uploads/Sona_and__Marana_Hardware_Guide_B-6_Models.pdf

[5] Lebedev, D. (n.d.). *Pylablib: Instrument control and data acquisition library for Python* (Latest version) [PDF documentation]. Read the Docs. https://app.readthedocs.org/projects/pylablib/downloads/pdf/latest/

[6] Lebedev, D. (n.d.). *pylablib.devices.Andor*

— *Andor camera control* [Web documentation]. Read the Docs. https://pylablib.readthedocs.io/en/stable/.apidoc/pylablib.devices.Andor.html#pylablib-devices-andor-package

[7] Klinchuen, R. (2024, March). องค์ความรู้ใหม่: โปรแกรมควบคุมการทำงานของกล้องถ่ายภาพ EMCCD และ CMOS [PDF manual]. Quantum-Atom Optics Laboratory.